

images/logo-unipa.png

UNIVERSITÀ DEGLI STUDI DI PALERMO
DIPARTIMENTO DI INGEGNERIA

*LAUREA MAGISTRALE IN INGEGNERIA INFORMATICA -
INTELLIGENZA ARTIFICIALE*

**VISION TRANSFORMER PRUNING USING
HESSIAN-GUIDED NEURAL PRUNING
SEARCH**

Tesi di Laurea di
Giacomo Lombardo

Relatore:
Prof.ssa Liliana Lo Presti

Controrelatore:
Prof. ?

Correlatore:
-

UNIVERSITÀ DEGLI STUDI DI PALERMO
DIPARTIMENTO DI INGEGNERIA

LAUREA MAGISTRALE IN INGEGNERIA INFORMATICA - INTELLIGENZA ARTIFICIALE

VISION TRANSFORMER PRUNING USING
HESSIAN-GUIDED NEURAL PRUNING SEARCH

Tesi di Laurea di

Giacomo Lombardo

Relatore:

Prof.ssa Liliana Lo Presti

Controrelatore:

Prof. ?

Correlatore:

-

Sommario

L'evoluzione dei Vision Transformer (ViT) ha permesso l'evoluzione della visione artificiale moderna, ma la loro elevata complessità computazionale ostacola l'inferenza locale su dispositivi mobili ed edge, sollevando problematiche di privacy e latenza legate all'uso di servizi cloud.

In questo lavoro si propone un framework automatico per la compressione strutturata di modelli ViT, modellando il pruning come un problema di ottimizzazione combinatoria multi-obiettivo risolto tramite un algoritmo Branch and Bound a profondità limitata. Per identificare i parametri ridondanti, viene formulata una metrica di importanza basata sulla matrice d'informazione di Fisher che approssima l'Hessiano della funzione di perdita. Questa metrica guida il taglio strutturato del modello, attraverso la selezione dei parametri meno rilevanti a fronte di azioni di pruning selezionate in fase di ottimizzazione. Il framework affronta la complessità combinatoria del problema sfruttando un set di ricerca limitato e dotato di tecniche di campionamento e augmentation volte a migliorare il processo di compressione, preservando le performance del modello.

I test sperimentali su CIFAR-100 e ImageNet-1K con modelli ViT-Small e DeiT-Small dimostrano come sia possibile ottenere una riduzione che raggiunge il 30% dei parametri a fronte di un calo di accuratezza inferiore all'1%. Infine, il deploy su dispositivi mobili Android tramite ONNX Runtime e NNAPI evidenzia come i guadagni prestazionali siano misurabili e rilevanti, soprattutto in contesti caratterizzati da risorse limitate, validando l'efficacia pratica del metodo in scenari edge reali.

Indice

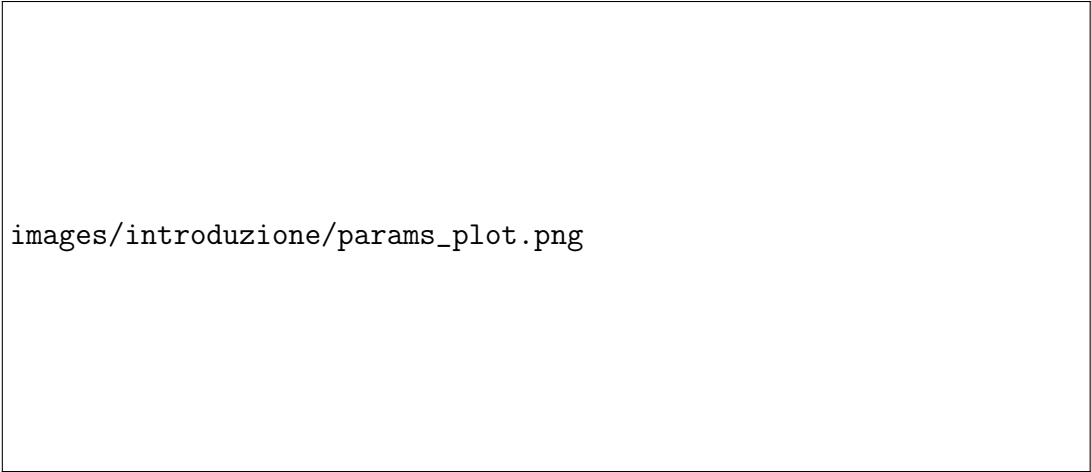
1	Introduzione	3
2	Fondamenta Teoriche	7
2.1	I modelli Transformer	8
2.2	Vision Transformer	13
2.3	Evoluzioni principali del Vision Transformer	16
3	Stato dell'arte	18
3.1	Architetture efficienti di ViT	20
3.1.1	Architetture Ibride	20
3.1.2	Architetture Edge-Optimized	25
3.1.3	Neural Architecture Search	32
3.2	Metodi di Compressione	41
3.2.1	Pruning	41
3.2.2	Quantizzazione	52
3.2.3	Knowledge Distillation	61
4	Formulazione matematica del problema	68
4.1	Formulazione della ricerca	69
4.1.1	Branching	70
4.1.2	Bound	72
4.1.3	Ottimalità dell'algoritmo di ricerca	72
4.2	Formulazione delle Azioni di ricerca	73
4.3	Derivazione del criterio di importanza	78
4.4	Fisher Information Matrix ed Hessiano	80

5	Implementazione del Framework	84
5.1	Gestione e Divisione dei dati	84
5.2	Framework di Compressione Iterativo	86
5.3	Implementazione dell'algoritmo di ricerca	87
5.4	Implementazione delle azioni di Pruning	88
6	Esperimenti e risultati	90
6.1	Dataset utilizzati	90
6.1.1	CIFAR-100	90
6.1.2	ImageNet-1K	91
6.2	Data Augmentation	92
6.3	Dettaglio implementativo	93
6.4	Metriche	96
6.5	Test con ViT-Small	96
6.5.1	Test sul dataset CIFAR100	96
6.5.2	Test sul dataset ImageNet-1K	97
6.5.3	Analisi dell'algoritmo di ricerca con il modello ViT-Small	98
6.6	Test con DeiT-Small	103
6.6.1	Test su dataset CIFAR100	104
6.6.2	Test su dataset ImageNet-1K	105
6.6.3	Analisi dell'algoritmo di ricerca con il modello DeiT-Small	107
6.7	Ablazione	111
6.7.1	Metrica di Importanza e Algoritmo di Ricerca	111
6.7.2	Il parametro λ della funzione obiettivo	112
6.8	Comparazione con lo stato dell'arte su ImageNet-1K	113
6.9	Deploy su mobile	115
6.10	Limitazioni e sviluppi futuri	120
	Conclusioni	119
	Ringraziamenti	122
	Bibliografia	123

Capitolo 1

Introduzione

A partire dall'introduzione dell'architettura *Transformer* [68] nel 2017, l'ambito dell'Intelligenza Artificiale ha vissuto una vera e propria rinascita. Questo evento ha profondamente rivoluzionato sia la ricerca accademica che il panorama industriale, portando a una diffusione su larga scala di servizi e strumenti basati su tale tecnologia. Sebbene questa rivoluzione abbia avuto inizio con i modelli linguistici di grandi dimensioni, come GPT-2 [55], essa si è rapidamente estesa a molteplici domini: dall'elaborazione delle immagini con l'avvento dei *Vision Transformer* [18], fino al riconoscimento e alla trascrizione audio con modelli come *Whisper* [54].



images/introduzione/params_plot.png

Figura 1.1: Grafico che mostra la crescita esponenziale del numero di parametri dei modelli di Intelligenza Artificiale dal 1950 ad oggi nei domini industriali e accademici. Fonte: <https://ourworldindata.org/grapher/artificial-intelligence-parameter-count>

Quotidianamente vengono rilasciati modelli sempre più capaci e versatili; tuttavia, il loro utilizzo risulta difficilmente gestibile da un hardware di fascia *consumer*. Uno dei metodi più diretti ed efficaci per migliorare le prestazioni di una rete neurale consiste, infatti, nell'aumentarne la scala (il numero di parametri), con un conseguente incremento dei requisiti computazionali. Come è possibile notare dalla Figura 1.1, il numero di parametri dei modelli di intelligenza artificiale è cresciuto esponenzialmente negli ultimi anni, passando da poche decine di milioni a centinaia di miliardi, fino a superare la soglia del trilione. Risulta quindi evidente che le risorse, non esclusivamente computazionali ma anche energetiche, necessarie per l'addestramento e l'inferenza di reti di tale portata siano proibitive e alla portata di un ristretto numero di grandi aziende tecnologiche.

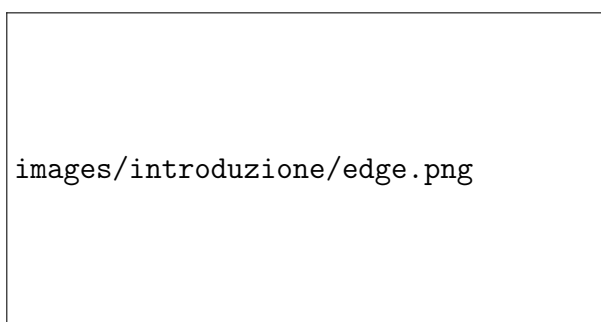


Figura 1.2: Differenza tra *Cloud AI* e *Edge AI*. Fonte: <https://www.cardinalpeak.com/blog/at-the-edge-vs-in-the-cloud-artificial-intelligence-and-machine-learning>

Per tali ragioni, la fruizione di modelli *state-of-the-art* avviene quasi esclusivamente tramite infrastrutture *cloud* (Figure 1.2), le quali delegano il carico di lavoro a cluster di hardware specializzato (GPU o TPU), con costi operativi generalmente proporzionali all'intensità di utilizzo. Questa modalità di fruizione presenta però diversi svantaggi. Oltre alla necessità di una connessione di rete, emergono notevoli problematiche legate alla privacy e alla sicurezza dei dati sensibili. Quando si fa affidamento ad un servizio di inferenza remoto, i dati dell'utente (siano essi *prompt* testuali, immagini o video) vengono trasferiti dal dispositivo locale per essere elaborati su server di terze parti, esponendo l'utilizzatore a potenziali violazioni della riservatezza o a usi impropri da parte dei fornitori del servizio. Molti dei casi d'uso di questa tecnologia, al quale si potrebbe pensare, sono irrealizzabili nella pratica a causa dell'impossibilità di gestire questi modelli di grandi dimensioni su dispositivi *Edge*, ad esempio gli smartphone. Si pensi, ad esempio, all'elaborazione di dati medici sensibili tramite dispositivi diagnostici, per il quale le normative sulla privacy impongono vincoli molto severi. Analogamente, in settori critici come

la guida autonoma o la robotica industriale, la dipendenza dalla latenza e dalla stabilità di una connessione di rete per prendere decisioni in tempo reale è un rischio inaccettabile. Tutti questi scenari applicativi risultano, di fatto, irrealizzabili nella pratica proprio a causa dell'impossibilità di eseguire localmente modelli di tali dimensioni su dispositivi *Edge*.

Per tali ragioni, la ricerca si è recentemente concentrata sia sull'introduzione di architetture più efficienti, sia su tecniche di compressione e ottimizzazione che permettono di ridurre i requisiti computazionali senza compromettere significativamente le prestazioni.

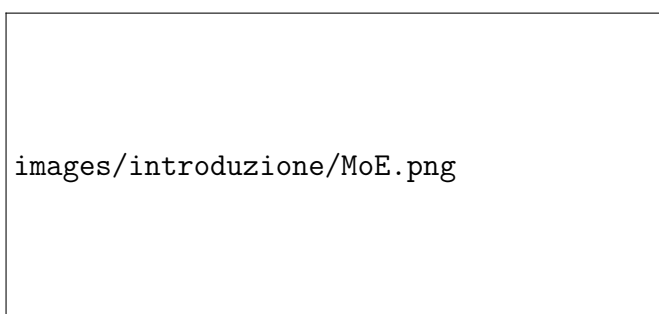


Figura 1.3: Il *MoE Layer*, alla base dei LLM a misture di esperti, posti a valle dell'attenzione invece dei tipici blocchi FFN. Fonte: [33]

Tecniche come la quantizzazione dei pesi [31], ormai standard per i *Large Language Model* (LLM), o l'adozione di architetture *Mixture of Experts* (MoE, Figura 1.3) [33], che attivano selettivamente solo un sottoinsieme di parametri, hanno garantito importanti passi avanti e godono di larga diffusione. Anche dal punto di vista hardware sono stati effettuati numerosi passi avanti. L'industria dei semiconduttori ha progettato architetture specificamente ottimizzate per i carichi di lavoro legati all'Intelligenza Artificiale. Negli ultimi anni si è assistito, ad esempio, all'introduzione dei *Tensor Core* [46] all'interno delle moderne GPU, progettati per massimizzare l'efficienza delle operazioni di moltiplicazione e somma tra tensori (MAC), fondamentali per l'inferenza delle reti neurali. Parallelamente, sono nati acceleratori dedicati, come le *Tensor Processing Unit* (TPU) per le grandi infrastrutture *cloud*, e le *Neural Processing Unit* (NPU), ormai sempre più integrate nell'hardware moderno. Tuttavia, nonostante i notevoli progressi, la sfida di rendere i modelli di intelligenza artificiale utilizzabili su hardware *consumer*, o ancor di più su dispositivi *Edge*, rimane aperta.

A partire da queste considerazioni, l'obiettivo di questa tesi è quello di contribuire alla ricerca di strategie di compressione dei modelli neurali, esplorate a fondo per quanto riguarda le reti convoluzionali, ma ancora poco studiate per quanto riguarda le architetture *Transformer*. Nella

trattazione di questo argomento ci si concentrerà sui modelli *Vision Transformer* [18] (ViT), che rappresentano ad oggi lo stato dell'arte per quanto riguarda l'elaborazione e l'estrazione di *feature* dalle immagini, e sono particolarmente diffusi come *backbone* per svariati domini applicativi, come l'*object detection* [7, 37], task multimodali [53, 80] e la segmentazione [34].

In particolare, a partire dalle idee alla base della ricerca automatica di architetture (*Neural Architecture Search*), viene proposto un framework di compressione che, dato un modello ViT pre-addestrato, specializzato su un dominio specifico, è in grado di identificare e rimuovere i parametri meno rilevanti (*pruning*), in maniera strutturata, così da poter ricostruire il modello in una sua versione ridotta, maggiormente efficiente, ma in grado di mantenere prestazioni comparabili a quelle del modello originale.

Il contributo principale di questo lavoro risiede nello sviluppo di un problema di ottimizzazione, volto a identificare le strategie di compressione ottimali per architetture *Vision Transformer*, e della sua soluzione tramite un algoritmo di ricerca *Branch and Bound*. Tale processo permette di selezionare le azioni di *pruning* più efficaci, bilanciando la riduzione dei parametri con il mantenimento delle capacità predittive del modello. A supporto di questo, introduciamo una metrica di importanza (secondo contributo) deputata, a fronte della selezione di una specifica azione di taglio, alla determinazione dei parametri candidati per il *pruning*. Notare come questo framework, nonostante sia stato testato su modelli *Vision Transformer*, sia generalizzabile anche ad altre architetture *Transformer*, ad esempio quelle utilizzate per l'elaborazione del linguaggio naturale. Potrebbe essere esteso ai modelli BERT [17], ma anche ai modelli di maggiori dimensioni, come i LLM, in quanto le architetture alla base di questi modelli condividono la stessa struttura. Anche per quanto riguarda i domini applicativi, sebbene la trattazione si concentri su task di classificazione di immagini, la strategia proposta è estendibile ad altri *task*, a patto di modificare opportunamente le funzioni di perdita e obiettivo.

Il presente elaborato è strutturato come segue. Nel **Capitolo 1** viene presentato lo stato dell'arte, analizzando le principali architetture efficienti basate su *Vision Transformer* e le più diffuse tecniche di compressione. Il **Capitolo 2** introduce la formulazione matematica del problema di ottimizzazione, descrivendo la modellazione dello spazio di ricerca tramite l'algoritmo *Branch and Bound* e la derivazione formale del criterio di importanza dei parametri. Nel **Capitolo 3** viene dettagliata l'implementazione pratica del *framework* di compressione iterativo e gli algoritmi implementati. Il **Capitolo 4** è dedicato alla discussione dei risultati sperimentali, illustrando i test condotti. Chiudono l'elaborato le **Conclusioni**, nelle quali vengono presentati gli obiettivi raggiunti e delineati i possibili sviluppi futuri.

Capitolo 2

Fondamenta Teoriche

Gran parte dell'intelligenza artificiale moderna si fonda sull'utilizzo del **Deep Learning** [36] (DL), grazie alla sua capacità intrinseca di estrarre le *features* dai dati, superando i limiti del *Machine Learning* tradizionale. Ad oggi, il DL viene utilizzato in una grande varietà di domini, come l'elaborazione del linguaggio naturale, oppure la visione artificiale, o ancora la robotica. Tuttavia, le reti neurali *feed-forward* presentano una grave limitazione, ovvero l'assenza di uno stato interno, che non ne permette l'utilizzo con dati in sequenza, come i video, gli audio o il testo. Al fine di ovviare a questa mancanza, sono state introdotte le **Reti Neurali Ricorrenti** (RNN), specificatamente pensate per l'elaborazione di dati sequenziali di qualsiasi tipologia.

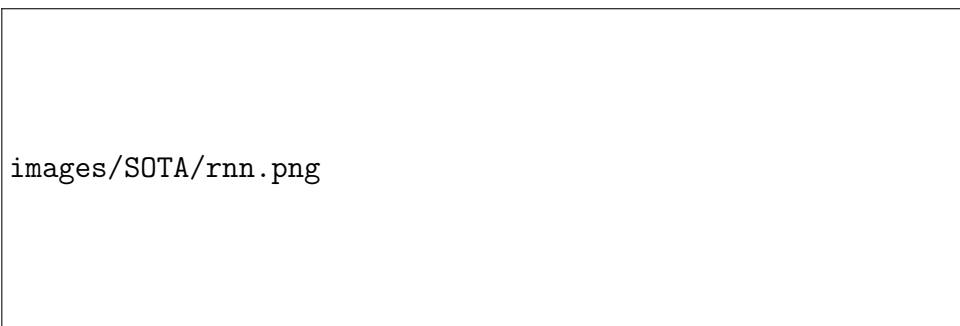


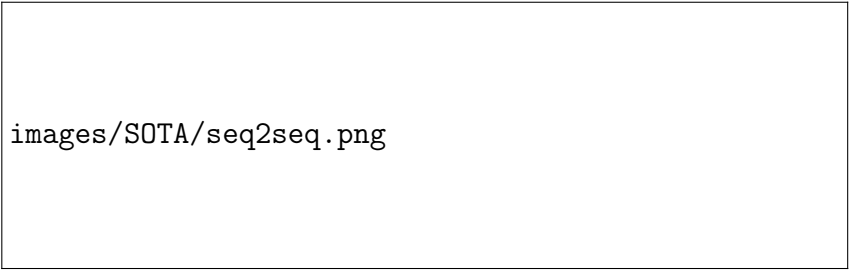
Figura 2.1: Rete Neurale Ricorrente e *unfolding*. Fonte: <https://commons.wikimedia.org/w/index.php?curid=60109157>

Nonostante la capacità teorica delle RNN di processare sequenze di lunghezza arbitraria, la loro applicazione pratica è limitata a causa di una serie di problematiche molto note. Come dimostrato matematicamente da Bengio et al. [4], l'addestramento di questi modelli è ostacolato dal

fenomeno della scomparsa o dell'esplosione del gradiente. Questa criticità rende estremamente difficile per il modello correlare informazioni distanti all'interno della sequenza temporale. Al fine di contrastare questi fenomeni nascono le reti **Long Short Term Memory** (LSTM) [30] e **Gated Recurrent Units** (GRU) [12], che introducono il concetto di *gating* per risolvere le problematiche legate al gradiente. Per anni questa tipologia di modelli ha costituito lo stato dell'arte per i dati sequenziali, pur presentando alcune limitazioni note. Tra queste possiamo individuare la necessità di elaborare i dati in modo sequenziale (parola per parola nel caso di elaborazione del linguaggio naturale), rendendo impossibile la parallelizzazione del calcolo tramite hardware moderno (GPU), oppure la difficoltà con le dipendenze a lungo termine all'interno delle sequenze.

2.1 I modelli Transformer

I modelli *Transformer* trovano le loro origini nel meccanismo di **Attenzione**, nato circa nell'anno 2014 nell'ambito dei modelli *encoder-decoder*, in particolare con le reti *seq2seq* [63], utilizzate nell'ambito della traduzione automatica. Per generare l'output, in questa tipologia di modelli il *decoder*, tipicamente costituito da una RNN, utilizza come unico vettore in input, lo stato finale dell'*encoder* (anche in questo caso si tratta solitamente di una RNN), che riassume la sequenza di input per poi generare l'output.



images/SOTA/seq2seq.png

Figura 2.2: Inferenza con modelli *seq2seq*. Fonte: <https://github.com/d2l-ai/d2l-en>

La grande limitazione di questi modelli risiede proprio nell'utilizzo di un singolo vettore statico per la fase di decodifica, che non risulta sufficiente a riassumere adeguatamente la sequenza in input. Inoltre, se quest'ultima è particolarmente lunga, dato l'utilizzo di modelli RNN o LSTM, le informazioni legate ai primi dati in ingresso possono essere perse a causa della grande distanza dallo stato finale. Allo scopo di superare i limiti del vettore di contesto e migliorare

quindi le prestazioni dei modelli *seq2seq* viene introdotto per la prima volta un meccanismo di attenzione da Bahdanau et al. [3].

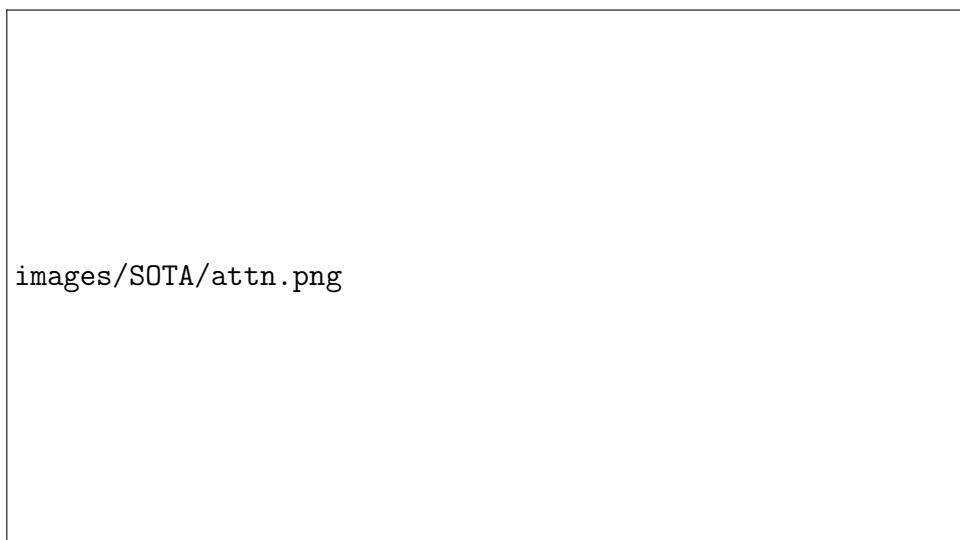


Figura 2.3: Meccanismo di attenzione nei modelli *seq2seq*. Fonte: https://lena-voita.github.io/nlp_course/seq2seq_and_attention.html

Questo meccanismo rende dinamico il vettore di contesto, che viene calcolato per ogni iterazione di decodifica pesando i contributi di tutti gli stati intermedi ottenuti dall'*encoder*. In questo modo è possibile fornire in input al modello *decoder* un vettore che riassume adeguatamente la sequenza in ingresso, dando maggiore importanza alle porzioni realmente rilevanti per l'output e migliorando nettamente le prestazioni dei modelli *seq2seq*.

Nonostante l'introduzione dell'attenzione abbia rappresentato una svolta nel superamento del limite costituito dal vettore di contesto statico, queste architetture rimangono ancora profondamente legate alla natura sequenziale delle RNN o LSTM. In tale configurazione, l'attenzione agisce solo come un meccanismo ausiliario tra l'*encoder* e il *decoder*, e l'elaborazione dei dati avviene un passo alla volta, impedendo qualsiasi tipo di parallelizzazione dei calcoli.

Una grande innovazione si verifica con l'introduzione dell'architettura **Transformer** [68], che abbandona interamente l'approccio sequenziale delle reti ricorrenti per passare a quello delle reti *feed-forward*, utilizzando la **Multi-Head Self Attention**. In particolare, l'architettura proposta da Vaswani et al. [68] è di tipo *encoder-decoder*, poiché è stata utilizzata e testata sul dominio della traduzione automatica.



Figura 2.4: Architettura Transformer originaria proposta da Vaswani et al. Fonte: [68]

Questa architettura si compone di una serie di blocchi, i cosiddetti blocchi *Transformer*, posti in cascata attraverso un flusso residuale (al fine di evitare i fenomeni della scomparsa dei gradienti) e formati da tre componenti principali:

- **Multi-Head Self Attention (MHSA):** rappresenta il cuore del blocco, permette al modello di proiettare l'input in diversi spazi di rappresentazione e catturare relazioni globali tra gli elementi della sequenza.
- **Layer Normalization (LayerNorm):** introdotta da Ba et al. [2], viene applicata prima (o dopo, a seconda della variante *Pre-Norm* o *Post-Norm* [75]) di ogni blocco funzionale, stabilizza le attivazioni e accelera la convergenza dell'addestramento, garantendo che la distribuzione dei dati rimanga consistente attraverso i vari layer.
- **Feed-Forward Network (FFN):** tipicamente costituita da due trasformazioni lineari intervallate da una funzione di attivazione non lineare (come la *GELU* [28] o la *ReLU* [1]).

Questo modulo, spesso chiamato **MLP** (*Multi-Layer Perceptron*), opera in modo indipendente su ogni posizione della sequenza, permettendo di elaborare e ricombinare le informazioni estratte dal meccanismo di attenzione.

Il nodo cruciale di questa architettura è il meccanismo di **Self Attention**, che risulta strutturalmente differente da quanto introdotto da Bahdanau et al. [3], poiché non si tratta più di un calcolo di similarità tra lo stato corrente del *decoder* (output) e gli stati intermedi dell'*encoder* (input). In questo nuovo meccanismo i singoli elementi della sequenza in input interagiscono tra di loro, permettendo di catturare le dipendenze contestuali indipendentemente dalla distanza fra gli elementi coinvolti. In particolare, detto x un elemento della sequenza in input $X = x_1, x_2, \dots, x_n$, la *Self Attention* esegue tre proiezioni lineari che producono le *query*, le *key* e i *value*:

$$\begin{aligned} q &= W_q x, \\ k &= W_k x, \\ v &= W_v x. \end{aligned} \tag{2.1}$$

Considerato l' i -esimo elemento della sequenza x_i , viene calcolato il prodotto scalare tra la *query* corrispondente q_i e le *key* di tutti gli altri elementi della sequenza. Il risultato esprime il grado di affinità o "attenzione" che l'elemento i -esimo deve porre verso gli altri elementi. Questi punteggi vengono poi normalizzati ad una distribuzione di probabilità tramite una funzione *softmax*:

$$\alpha_{ij} = \text{softmax} \left(\frac{q_i \cdot k_j}{\sqrt{d_k}} \right) = \frac{\exp \left(\frac{q_i \cdot k_j}{\sqrt{d_k}} \right)}{\sum_{l=1}^n \exp \left(\frac{q_i \cdot k_l}{\sqrt{d_k}} \right)}. \tag{2.2}$$

Infine, viene eseguita una somma pesata dei vettori *value* v per ottenere l'uscita del meccanismo di attenzione:

$$a_i = \sum_{j=1}^n \alpha_{ij} v_j. \tag{2.3}$$

Tuttavia, il vero punto di forza dell'architettura Transformer risiede nella capacità di parallelizzare questa operazione attraverso la **Scaled-Dot-Product Attention** (Figura 2.5). Invece di calcolare l'attenzione un elemento alla volta, le proiezioni di tutti gli n elementi della sequenza vengono raggruppate in matrici $Q, K, V \in \mathbb{R}^{n \times d}$, permettendo di eseguire l'intero calcolo tramite

il prodotto matriciale, facilmente parallelizzabile tramite hardware apposito (GPU oppure TPU):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V. \quad (2.4)$$

Notare che l'architettura Transformer non possiede meccanismi intrinseci per gestire l'ordine sequenziale dei dati (a differenza delle RNN). È quindi necessario introdurre il meccanismo di **Positional Encoding** [58, 61]. Si tratta di vettori che vengono sommati ai vettori di input per fornire al modello informazioni sulla posizione relativa o assoluta di ogni elemento nella sequenza. Questa formulazione evidenzia i costi computazionali dell'architettura: il prodotto QK^T genera una matrice di attenzione di dimensione $n \times n$, portando a una complessità quadratica $O(n^2)$ (temporale e spaziale) rispetto alla lunghezza della sequenza. Di conseguenza, il vantaggio rispetto alle architetture RNN, dato dalla possibilità di parallelizzare e velocizzare le operazioni sull'intera sequenza, comporta un aumento del costo computazionale (le RNN hanno costo lineare $O(n)$).



Figura 2.5: Visualizzazione dell'operazione di *Scaled-Dot-Product Attention*. Fonte: [68]

Al fine di aumentare la capacità di questa nuova architettura viene introdotto il concetto di **Multi-Head Attention**, attraverso il quale è possibile dotare il meccanismo di attenzione di molteplici "teste" che operano estraendo *features* differenti dalla sequenza in input. Quanto descritto in precedenza viene applicato all'interno di ogni *attention head* e successivamente gli output vengono concatenati e proiettati in uno spazio della dimensione originale della rappresentazione.

The image is a placeholder for a diagram titled 'images/SOTA/MHSA.png'. It is represented by a rectangular box with the text 'images/SOTA/MHSA.png' centered inside.

Figura 2.6: *Multi Head Self Attention*. Fonte: [68]

Per concludere la trattazione dei modelli *Transformer* è necessario discutere dell'utilizzo della **mascheratura** (*masking*). Come è possibile notare dalla Figura 2.4, vi sono due tipologie di blocchi *Transformer*: i blocchi *encoder* e i blocchi *decoder*. Nei primi, ogni elemento può interagire con ogni altro elemento della sequenza (attenzione bi-direzionale [17]), permettendo di estrarre una rappresentazione contestuale completa di contesto destro e sinistro. Al contrario, nei secondi (usati nei modelli generativi di linguaggio), viene applicata una maschera durante la fase di attenzione (Figura 2.5). Tale maschera pone a $-\infty$ i valori della matrice di attenzione relativi agli elementi successivi nella sequenza, garantendo che, attraverso la funzione *softmax*, i pesi di attenzione corrispondenti vengano azzerati. In questo modo la predizione (della parola successiva, nel caso dei modelli generativi del linguaggio) dipenderà esclusivamente dai termini che la precedono (contesto sinistro), preservando così la causalità necessaria durante l'inferenza.

2.2 Vision Transformer

Per quanto concerne l'ambito della visione artificiale, per anni le **Reti Neurali Convoluzionali** (CNN) hanno dominato il campo grazie alla loro capacità di estrarre le *features* spaziali delle immagini attraverso l'uso di filtri appresi che vengono applicati alle immagini. Il successo delle CNN è dovuto principalmente a due proprietà intrinseche: l'**invarianza alla traslazione** e la **località dei dati**. Grazie ai filtri convoluzionali, queste reti sono in grado di riconoscere pattern indipendentemente dalla loro posizione nell'immagine e di elaborare le relazioni tra

pixel vicini in modo estremamente efficiente. Tuttavia, questo approccio presenta un limite fondamentale: la natura locale delle convoluzioni rende difficile catturare le **relazioni a lungo raggio** tra pixel distanti, se non attraverso l'utilizzo di numerosi strati, posti in cascata, che aumentano progressivamente il campo ricettivo (*receptive field*) [44].

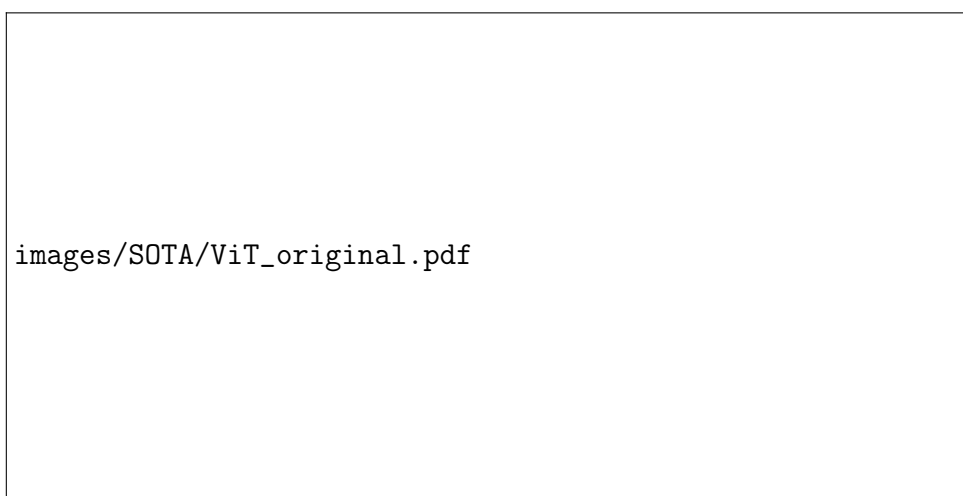


Figura 2.7: Architettura del Vision Transformer (ViT). Fonte: [18]

L'introduzione dei modelli *Transformer* ha portato alla nascita dei **Vision Transformer** (ViT) [18], che utilizzano un'architettura *Encoder-Only* per applicare il meccanismo di attenzione all'ambito della visione artificiale. In particolare, l'architettura ViT suddivide l'immagine in regioni non sovrapposte, dette *patch*. Quest'ultime, ad esempio di dimensione 16×16 pixel, vengono poi appiattite e proiettate in uno spazio di rappresentazione tramite una trasformazione lineare. Questi vettori di rappresentazione delle *patch* vengono quindi arricchiti attraverso l'*Embedding Posizionale* e forniti ad una serie di blocchi *Transformer-Encoder* simili a quelli descritti in precedenza.

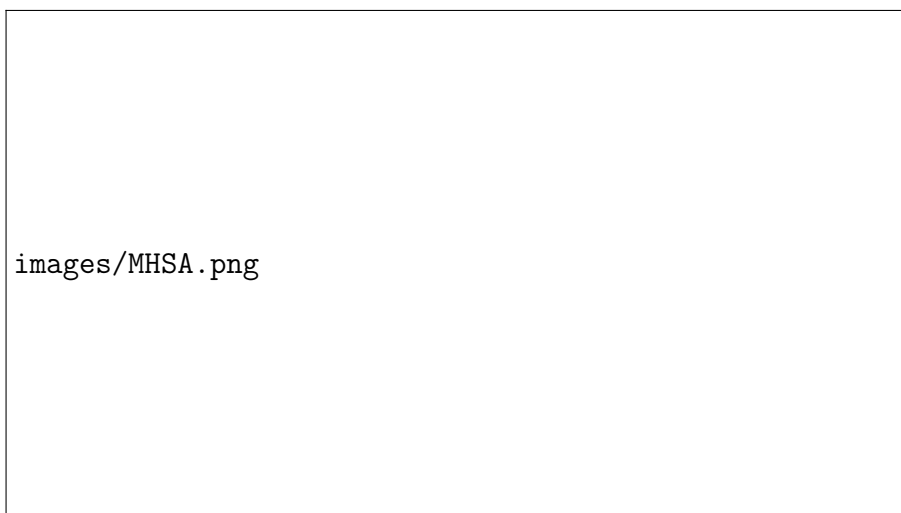


Figura 2.8: Struttura della *Multi Head Attention*.

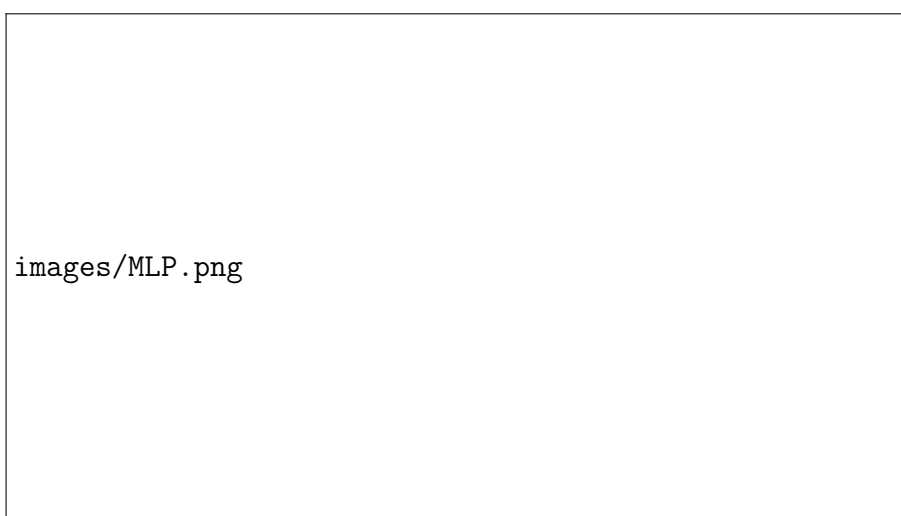


Figura 2.9: Struttura del *Multi Layer Perceptron* nei blocchi Transformer del ViT.

Sfruttando il meccanismo di attenzione, i ViT permettono alle regioni dell'immagine di interagire tra loro, catturando relazioni a lungo raggio molto più facilmente rispetto alle CNN. Nonostante i ViT abbiano dimostrato prestazioni competitive rispetto alle CNN [18, 66, 42], soprattutto su dataset di grandi dimensioni, presentano ancora alcune limitazioni, come la necessità di grandi quantità di dati per l'addestramento [60] e la complessità computazionale elevata, dovuta alla natura quadratica del meccanismo di attenzione. Tuttavia, l'utilizzo di questi modelli

sta crescendo rapidamente (soprattutto come *backbone* o *Vision Encoder* all'interno di architetture modulari [34, 53]), portando a numerose varianti e miglioramenti che cercano di mitigare queste criticità, come l'introduzione di meccanismi di attenzione più efficienti [42] o l'adozione di approcci ibridi che combinano le CNN con i ViT [47, 67].

2.3 Evoluzioni principali del Vision Transformer



Figura 2.10: Architettura del Data-efficient Image Transformer (DeiT). Fonte: [66]

Una delle prime varianti che ha migliorato l'implementazione originale del ViT è stata il **Data-efficient Image Transformer** (DeiT) [66], che ha dimostrato come l'utilizzo di una tecnica di *data augmentation* aggressiva possa evitare la necessità di utilizzare dataset di grandi dimensioni per addestrare i *Vision Transformer*. Hanno inoltre introdotto una variante dell'architettura, addestrata attraverso *knowledge distillation*, in cui un modello ("student") viene addestrato a imitare le predizioni di un modello pre-addestrato ("teacher"), ottenendo prestazioni competitive anche con dataset di dimensioni ridotte. Questa tecnica, applicata ad uno speciale *token* di distillazione, ha permesso al *DeiT-Distilled* di raggiungere risultati paragonabili a quelli del ViT originale, ma con un quantitativo di dati di addestramento significativamente minore.

images/SOTA/SWIN_arch.pdf

Figura 2.11: Architettura di *Swin Transformer*. Fonte: [42]

Successivamente, è stato proposto **Swin Transformer** [42], che introduce meccanismi di attenzione gerarchici e a finestre mobili per migliorare l'efficienza computazionale. A differenza del ViT che mantiene una risoluzione costante, lo *Swin Transformer* riduce gradualmente la risoluzione spaziale man mano che si scende in profondità nella rete, unendo le patch vicine (Patch Merging). Questo crea una struttura gerarchica simile a quella delle reti neurali convoluzionali (CNN), permettendo al modello di vedere sia dettagli piccoli sia strutture grandi. Inoltre, piuttosto che calcolare l'attenzione sull'intera immagine, lo *Swin Transformer* la calcola esclusivamente all'interno di finestre locali non sovrapposte (composte da patch), introducendo meccanismi di località simili a quelli delle CNN.

images/SOTA/SWIN_window.png

Figura 2.12: Meccanismo di Shifted Window. Fonte: [42]

Al tempo stesso viene introdotto un meccanismo di *Shifted Window* che permette alle finestre di patch di interagire tra loro nel blocco successivo, garantendo che le informazioni vengano propagate attraverso l'intera immagine. Grazie a queste innovazioni, *Swin Transformer* è in grado di raggiungere prestazioni all'avanguardia su una varietà di compiti di visione artificiale [42], come la classificazione delle immagini, il rilevamento degli oggetti e la segmentazione semantica, con un'efficienza computazionale significativamente migliorata rispetto al ViT originale.

Capitolo 3

Stato dell'arte

Data la rapida evoluzione nel campo della visione artificiale, l'analisi dello stato dell'arte risulta fondamentale per identificare le metodologie più promettenti volte a coniugare elevate prestazioni e sostenibilità computazionale. In questo capitolo verrà esaminato il panorama delle soluzioni atte a rendere i **Vision Transformer** (ViT) applicabili in scenari reali, caratterizzati da vincoli di latenza, memoria e consumo energetico. Occorre notare che nessuna soluzione esclude automaticamente le altre, spesso molti di questi approcci vengono integrati e combinati per ottenere risultati migliori.



Figura 3.1: Tassonomia delle principali metodologie per l'efficientamento dei modelli di visione artificiale.

La trattazione si articolerà su due fronti principali:

- **Ottimizzazione Architetturale:** l'analisi di modelli e architetture appositamente progettate per ridurre la complessità computazionale, i costi di memoria o anche i consumi energetici dell'inferenza. A loro volta, queste saranno suddivise in:
 - **Architetture Ibride:** architetture che integrano elementi tipici dei modelli *Transformer* con elementi tipici delle reti convoluzionali, al fine di sfruttare i punti di forza di entrambe le tipologie di modelli e ottenere un buon compromesso tra prestazioni e efficienza;
 - **Architetture Edge-Optimized:** modelli progettati specificamente per essere eseguiti su dispositivi con risorse limitate, come smartphone o dispositivi IoT, con l'obiettivo di massimizzare le prestazioni mantenendo un basso consumo energetico e ridotti requisiti computazionali;
 - **Neural Architecture Search (NAS):** ovvero l'automazione del processo di design di architetture, attraverso procedure di ricerca e ottimizzazione automatica. Questi metodi permettono di esplorare in modo efficiente un ampio spazio di architetture candidate, identificando le migliori secondo criteri di prestazioni e efficienza, eliminando la necessità di una progettazione manuale estensiva.
- **Tecniche di Compressione dei modelli:** lo studio di metodologie applicabili a reti preesistenti al fine di ridurre la dimensione e migliorarne l'efficienza, tra cui:
 - **Pruning:** ovvero la rimozione di pesi, teste di attenzione o interi blocchi ridondanti dall'architettura per snellire il modello senza compromettere significativamente le prestazioni;
 - **Quantizzazione:** cioè la riduzione della precisione numerica della rappresentazione dei parametri, ottenendo modelli molto più compatti in termini di memoria e veloci da eseguire su hardware specializzato;
 - **Knowledge Distillation:** il trasferimento di conoscenza da modelli *teacher* grandi e complessi a modelli *student* più piccoli e leggeri, aumentando in modo importante le prestazioni ottenibili da questi ultimi.

3.1 Architetture efficienti di ViT

Di seguito verranno analizzate le principali architetture efficienti di ViT, spaziando dai primi approcci applicati a seguito dell'introduzione dei ViT, fino alle soluzioni più recenti e innovative.

3.1.1 Architetture Ibride

A partire dall'introduzione dei *Vision Transformer* (ViT) da parte di Dosovitskiy et al. [18], la comunità ha compiuto numerosi sforzi per sviluppare architetture che potessero mantenere le prestazioni elevate dei ViT, ma con una complessità computazionale ridotta. Tra queste, le architetture ibride rappresentano una delle direzioni più promettenti ed esplorate.

BoTNet

Uno dei primi approcci è stato quello presentato da Srinivas et al. [59], nel quale è stata modificata una architettura ResNet [27] sostituendo le convoluzioni spaziali (3×3) negli ultimi 3 blocchi (definiti *bottleneck*) con dei blocchi Multi-Head Self-Attention.

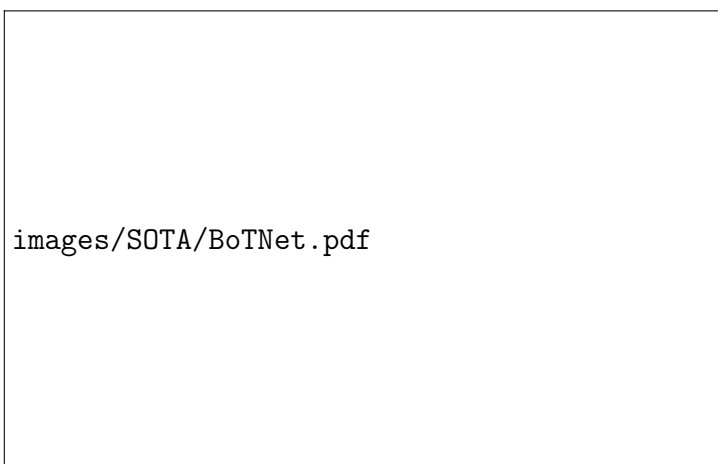
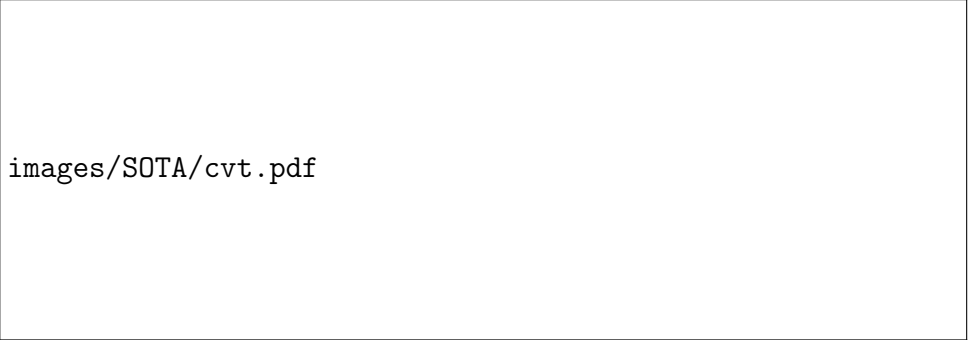


Figura 3.2: Confronto tra il blocco ResNet e il blocco BoTNet. Fonte: [59].

In particolare questa architettura ha dimostrato delle performance migliori rispetto a ResNet-50, con un numero di parametri simile, ma con una complessità computazionale più elevata data dall'introduzione dell'attenzione. Il lavoro in [59] ha dimostrato che l'integrazione delle due architetture è una direzione di ricerca promettente, ma è necessario trovare un compromesso tra prestazioni e complessità computazionale.

CvT

I pionieri dell'applicazione delle convoluzioni ad un ViT sono stati Wu et al. [72], con il loro **Convolutional Vision Transformer**, che ha introdotto delle convoluzioni per produrre le query, key e value all'interno di nuovi *Convolutional Transformer Block*, integrando profondamente le due architetture. Questo tipo di approccio ha eliminato la necessità del *Positional Encoding* tramite le proprietà delle convoluzioni.



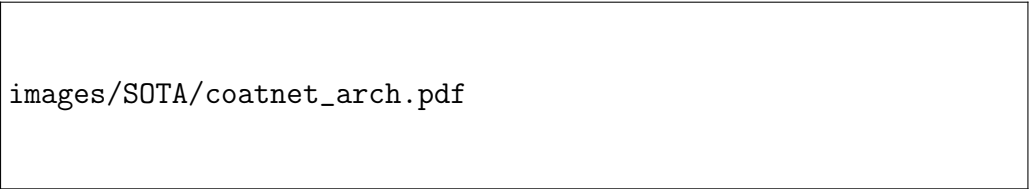
images/SOTA/cvt.pdf

Figura 3.3: A sinistra l'architettura CvT, a destra la struttura del blocco *Convolutional Transformer*.
Fonte: [72].

Questo tipo di soluzione ha permesso di raggiungere ottime performance su vari task, mantenendo contenuto il numero di parametri.

CoAtNet

Un ulteriore pilastro di questo approccio ibrido è quello di Dai et al. [14], i quali hanno proposto un'architettura CNN-*Transformer* definita **Co-AtNet**, con l'obiettivo di sfruttare i punti di forza delle due architetture. In particolare, gli autori hanno combinato blocchi convoluzionali *Depthwise* e moduli di *Self-Attention* organizzandoli in stadi successivi, con l'obiettivo di ridurre la complessità computazionale mantenendo elevate prestazioni.



images/SOTA/coatnet_arch.pdf

Figura 3.4: L'architettura di CoAtNet. Fonte: [14].

Gli autori hanno dimostrato che questo approccio permette di raggiungere prestazioni superiori rispetto a molte architetture CNN e ViT, con un numero di parametri e un numero di operazioni in virgola mobile (FLOPs) inferiori.

FastViT

Di recente, ulteriori sviluppi sono stati presentati da Vasu et al. [67] con l'architettura **FastViT**, nella quale si affronta il problema della latenza causata dai frequenti accessi in memoria necessari durante l'inferenza. In particolare, le connessioni residuali, seppur fondamentali per evitare la scomparsa del gradiente, contribuiscono pesantemente a questo problema, richiedendo di recuperare continuamente dalla memoria i valori sui quali eseguire la somma.



images/SOTA/fastvit_arch.pdf

Figura 3.5: L'architettura del modello FastViT. Fonte: [67].

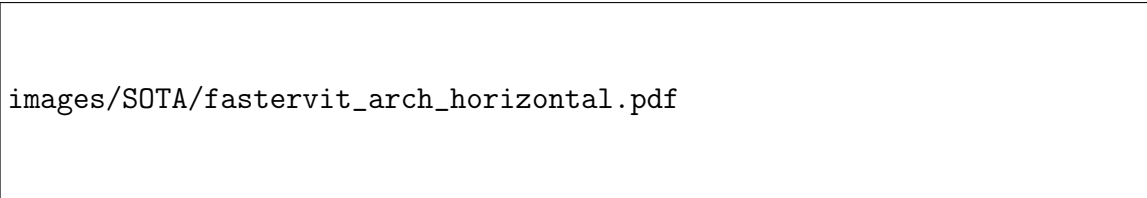
Gli autori risolvono questo problema applicando una **riparametrizzazione strutturale post-addestramento** all'interno di un modulo proprietario denominato **RepMixer**. Durante il *training*, la rete sfrutta connessioni residuali e convoluzioni su rami paralleli per massimizzare la capacità di apprendimento; in fase di inferenza, queste strutture parallele vengono fuse matematicamente in un'unica convoluzione di tipo *depthwise*. Questo approccio permette di eliminare

del tutto le operazioni di somma e i relativi accessi in memoria, mantenendo inalterate le elevate prestazioni predittive.

L'architettura di *FastViT* affida ai primi tre stadi, composti da blocchi convoluzionali, il compito di abbattere rapidamente la risoluzione. L'ultimo stadio introduce strategicamente un blocco di *Self-Attention*, che permette di ampliare il *receptive-field* e catturare le dipendenze globali solo quando la mappa delle *feature* è sufficientemente ridotta, evitando un aumento eccessivo della complessità computazionale. Grazie a queste soluzioni, il modello raggiunge prestazioni uguali o superiori alle architetture ibride di riferimento, ma con latenza e operazioni in virgola mobile (FLOPs) drasticamente inferiori.

FasterViT

Una ulteriore direzione di ricerca è quella di modificare il meccanismo di attenzione, al fine di introdurlo in una architettura basata su CNN, per limitare i costi. In questo senso Hatamizadeh et al. [26] propongono l'architettura **FasterViT**, nella quale viene concepito un nuovo meccanismo di attenzione, denominato **Hierarchical Attention**.



images/SOTA/fastervit_arch_horizontal.pdf

Figura 3.6: L'architettura del modello FasterViT. Fonte: [26].

L'approccio proposto risulta particolarmente efficace nei casi in cui vengono elaborati input ad alta risoluzione. In queste situazioni il calcolo della *Self-Attention* globale diventa rapidamente proibitivo a causa del suo costo computazionale quadratico. D'altra parte, limitare l'attenzione a finestre spaziali isolate (come avviene nello *Swin Transformer*) comporta la perdita cruciale del contesto globale. Per superare questo ostacolo, gli autori propongono una macro-architettura ibrida strutturata in quattro stadi sequenziali. I primi due stadi operano esclusivamente tramite blocchi residui convoluzionali, i quali estraggono le *feature* locali ed eseguono un sottocampionamento spaziale. Soltanto nel terzo e quarto stadio, dove la risoluzione delle *feature map* risulta notevolmente ridotta, intervengono i moduli *Transformer* al fine di catturare le dipendenze globali con un basso costo computazionale.

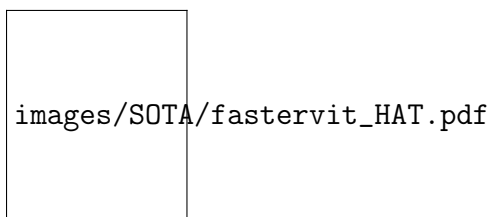


Figura 3.7: Mappa di attenzione del meccanismo di *Hierarchical Attention*. Fonte: [26].

La vera innovazione di *FasterViT* risiede nel nuovo meccanismo di **Hierarchical Attention (HAT)** utilizzato in questi ultimi stadi. Il modello introduce dei *Carrier Tokens* (token portatori) che agiscono come ponte informativo: questi speciali *token* riassumono prima le informazioni all'interno delle singole finestre locali, interagiscono poi tra di loro a livello globale scambiandosi il contesto generale dell'immagine e, infine, ritrasmettono l'informazione globale aggiornata ai *token* delle rispettive finestre locali. Questo approccio si avvicina ad un concetto di un *receptive-field* globale ma con una frazione del costo computazionale originale.

Utilizzando queste novità *FasterViT* riesce a stabilire nuovi standard di velocità ed efficienza, dimostrando che l'uso sinergico di convoluzioni iniziali e attenzione negli ultimi stadi è una strategia molto efficace per ottenere ottime prestazioni con costi computazionali contenuti.

Vision Mamba

Infine, di recente, il concetto di ibridazione si è spinto oltre la combinazione di CNN e *Transformer* classici. Per far fronte alla necessità di elaborare sequenze visive o immagini ad altissima risoluzione con complessità strettamente lineare $O(N)$, la ricerca più recente ha iniziato a sostituire i moduli di attenzione globale con i Modelli a Spazio di Stato (SSM), in particolare la loro declinazione denominata **Mamba** [24].

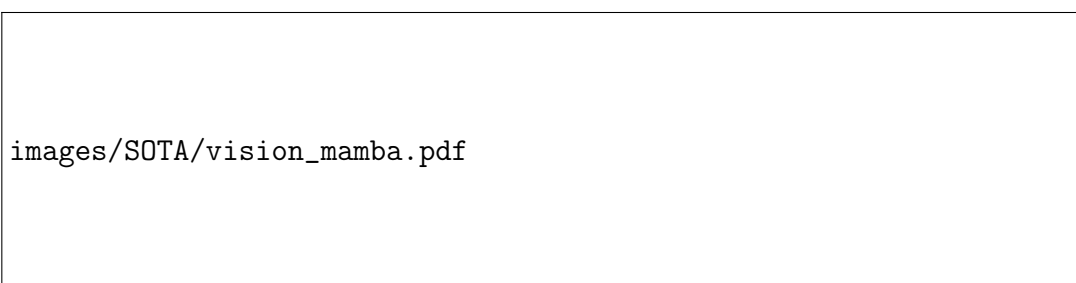


Figura 3.8: Architettura di Vision Mamba. Fonte: [82].

Architetture ibride emergenti, come le varianti convoluzionali di **Vision Mamba (Vim)** [82], integrano l'estrazione di *feature* locali tipica delle CNN con il ragionamento sequenziale bidirezionale degli SSM. Sebbene i ViT ibridi rimangano lo standard di fatto nell'industria odierna, questa nuova frontiera CNN-SSM rappresenta la naturale evoluzione per abbattere definitivamente il costo quadratico dell'attenzione.

3.1.2 Architetture Edge-Optimized

Uno dei principali ostacoli all'implementazione dei ViT in scenari reali è rappresentato dalle grandi difficoltà nell'esecuzione su dispositivi con risorse limitate, come smartphone o dispositivi IoT. Per affrontare questo problema sono stati compiuti notevoli sforzi sia in ambito hardware, implementando acceleratori specializzati per l'esecuzione di modelli neurali (NPU, *Neural Processing Unit*), sia in ambito architetturale, progettando modelli specificamente ottimizzati per i dispositivi *edge*.

MobileViT

Uno dei primi lavori in questo senso è stato quello di Mehta et al. [47], con il famoso **MobileViT** che utilizza un approccio ibrido per coniugare l'efficienza computazionale delle CNN con la capacità di catturare dipendenze globali dei ViT.

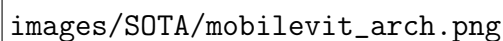
The image shows a placeholder for a diagram of the MobileViT architecture. The text 'images/SOTA/mobilevit_arch.png' is displayed within a rectangular frame, indicating the location of the original image file.

Figura 3.9: Architettura di MobileViT. Fonte: [47].

Analizzando più in dettaglio, l'architettura di MobileViT è composta principalmente da blocchi MV2 (MobileNetV2 [57]), che si occupano di eseguire il sottocampionamento e catturare

le dipendenze locali, alternati da blocchi *MobileViT*. Questi ultimi costituiscono il contributo più significativo di questo lavoro. A differenza dei classici *Vision Transformer* che appiattiscono irrimediabilmente le singole *patch* danneggiando la topologia bidimensionale dell'immagine, i blocchi *MobileViT* preservano la geometria spaziale elaborando le dipendenze globali in modo mirato.

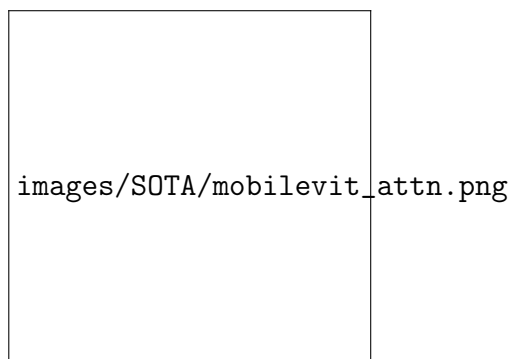


Figura 3.10: Meccanismo di attenzione di MobileViT. Fonte: [47].

Per ottenere questo risultato, il blocco esegue un'operazione matematica che gli autori definiscono *unfolding* (srotolamento). La mappa delle *feature* viene idealmente divisa in piccole finestre, o *patch*. Per costruire le sequenze monodimensionali rigorosamente richieste dal *Transformer*, il modello non fonde i pixel della singola *patch*, ma estrae esclusivamente i pixel che occupano la stessa posizione spaziale all'interno di tutte le *patch* dell'immagine (ad esempio, raggruppando tutti i pixel in alto a sinistra di ogni finestra). Ognuno di questi pixel estratti rappresenta un singolo *token*. L'insieme di questi pixel omologhi forma la sequenza su cui viene calcolata la *Self-Attention*. Questo approccio si rivela vincente perché permette di ottenere un campo recettivo globale a livello del singolo pixel senza pagarne il reale prezzo computazionale. Infatti, calcolare un'attenzione globale su tutti i pixel dell'immagine contemporaneamente comporterebbe un costo elevatissimo, insostenibile per un dispositivo mobile; svolgendo invece il calcolo in parallelo su queste specifiche sotto-sequenze posizionali, il costo totale viene abbattuto drasticamente.

Terminato il calcolo dell'attenzione, viene effettuata l'operazione inversa, definita *folding* (ripiegamento). I pixel appena aggiornati dal *Transformer* vengono ricollocati nelle loro coordinate 2D di origine, restituendo in uscita una mappa spaziale intatta e pronta per le elaborazioni successive. Tramite questo approccio, MobileViT raggiunge e supera le prestazioni sia delle CNN sia dei ViT tradizionali, risultando più efficiente di questi ultimi ma non delle prime.

Mobile-Former

In [11], Chen et al. propongono una soluzione innovativa che integra le due architetture *MobileNet* e *Transformer* in modo più profondo rispetto a *MobileViT*. In particolare, introducono un nuovo modulo, denominato **Mobile-Former Block**, che prevede due rami paralleli, uno convoluzionale (ispirato alla *MobileNet*) e uno basato su attenzione, con connessioni bidirezionali che permettono uno scambio continuo di informazioni tra i due rami.



Figura 3.11: Mobile-Former Block. Fonte: [11].

Il ramo convoluzionale, che ottiene in ingresso l'intera immagine, si occupa di estrarre le *features* locali, mentre il ramo basato sul meccanismo di attenzione, il quale ha in input dei token appresi, cattura le dipendenze globali. Le connessioni bidirezionali permettono a ciascun ramo di arricchire le proprie rappresentazioni con le informazioni apprese dall'altro, migliorando così la capacità complessiva del modello di comprendere sia i dettagli locali che il contesto globale dell'immagine. Inoltre, i token utilizzati nel ramo di attenzione non sono estratti dall'immagine, come in un ViT tradizionale, ma sono appresi durante il processo di addestramento, e sono in numero molto ridotto (tipicamente 6), così da mantenere contenuto il costo computazionale dell'attenzione. Essi agiscono inizialmente da contenitori vuoti, che verranno successivamente arricchiti di informazioni provenienti dal ramo convoluzionale, e a loro volta forniranno un contesto globale che arricchirà le rappresentazioni locali di questo.



Figura 3.12: Architettura complessiva di Mobile-Former. Fonte: [11].

A valle dei blocchi *Mobile-Former*, le *feature map* in uscita dalla porzione convoluzionale vengono sottoposte ad una operazione di global average pooling ottenendo un vettore a cui vengono concatenati i token del ramo di attenzione. Questa nuova sequenza viene poi essere processata da un classificatore finale. Grazie a questa architettura, Mobile-Former riesce a raggiungere prestazioni competitive rispetto sia ai classici modelli ViT, Swin e DeiT sia rispetto ad altre architetture basate su convoluzioni, con un numero di operazioni matematiche di moltiplicazione e somma (*MAdds*) estremamente ridotto (circa 3-4 volte inferiori rispetto ai modelli *Transformer* di riferimento), rendendolo particolarmente adatto per l'esecuzione su dispositivi mobili.

EdgeViT

Un altro esempio rilevante di architettura *edge-optimized* è quello di Pan et al. [51], con il loro modello **EdgeViT**. Gli autori, sostenendo che *MobileViT* non riesca a competere con le alternative convoluzionali ottimizzate per i dispositivi *edge* in termini di compromesso tra accuratezza e complessità computazionale, propongono una nuova architettura di tipo piramidale (all'aumentare della profondità si riduce la risoluzione spaziale e aumenta il numero di canali)

composta da 4 stadi che implementano un meccanismo definito **Local-Global-Local Bottleneck**, volto a ridurre il costo dell'attenzione.

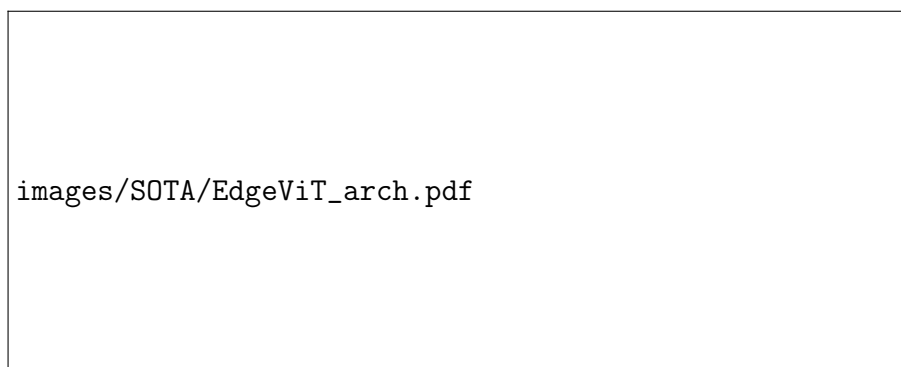
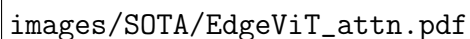


Figura 3.13: Architettura di EdgeViT. Fonte: [51].

Gli autori, partendo dal concetto di ridondanza spaziale delle immagini, che rende costosa e spesso superflua l'elaborazione di tutte le regioni dell'immagine con un meccanismo di attenzione globale, propongono una soluzione a tre fasi:

- *Aggregazione Locale*: per ogni token, vengono utilizzate delle convoluzioni per aggregare le informazioni provenienti da una finestra locale di dimensione fissata.
- *Attenzione Globale Sparsa*: invece di applicare l'attenzione globale su tutti i token, vengono campionati una serie di token rappresentativi, uno per ogni finestra locale. Il meccanismo di attenzione globale viene eseguito solo su questi ultimi token, riducendo drasticamente il costo computazionale.
- *Propagazione Locale*: Vengono utilizzate delle convoluzioni trasposte per propagare le informazioni globali contenute nei token rappresentativi a tutti i token locali (all'interno della finestra).



images/SOTA/EdgeViT_attn.pdf

Figura 3.14: Meccanismo di *Local-Global-Local Bottleneck* applicato ad un esempio. Fonte: [51].

Tramite questo approccio, *EdgeViT* riesce a ridurre il numero di token su cui viene calcolata l'attenzione, ma mantiene comunque la capacità di catturare dipendenze globali all'interno dell'immagine. Grazie a queste soluzioni, il modello riesce a superare le prestazioni di *MobileViT* e degli altri modelli *edge-optimized* e non, ottenendo un miglioramento significativo in termini di latenza.

RepViT

Reparameterized Vision Transformer (RepViT), proposto da Wang et al. [69], rappresenta un cambio di paradigma nel design dei modelli per l'*edge computing*. Gli autori sostengono che la superiorità dei *Vision Transformer* su dispositivi mobili non derivi necessariamente dal meccanismo di attenzione, ma dalle loro caratteristiche macro-architetture. Introducono quindi *RepViT*, un'architettura puramente convoluzionale che modernizza una CNN pensata appositamente per dispositivi mobili (MobileNetV3) applicando sistematicamente i principi di design dei ViT.



images/SOTA/RepViTBlock.pdf

Figura 3.15: Confronto tra il blocco di MobileNetV3 e il blocco RepViT. Fonte: [69].

Gli autori propongono una evoluzione del classico blocco *MobileNetV3*, denominato **RepViT Block**, che integra una serie di modifiche ispirate ai ViT, tra cui la riorganizzazione delle operazioni secondo lo schema di quest'ultimo. Mentre nel blocco *MobileNet* l'espansione dei canali avviene prima della convoluzione spaziale, in *RepViT* i compiti vengono separati nettamente: l'interazione tra i pixel è realizzata da una convoluzione *depthwise* all'inizio del blocco, mentre l'espansione dei canali e la relativa compressione viene affidata a un modulo finale composto da due convoluzioni 1×1 (paragonabile al componente FFN in un blocco *Transformer*). Il termine *Rep* si riferisce alla riparametrizzazione applicata alla parte iniziale del blocco: durante l'addestramento, come mostrato in Figura 3.15, questa parte dispone di tre rami paralleli (una convoluzione 3×3 , una 1×1 e una connessione residuale) per catturare informazioni più ricche; in fase di inferenza, questi rami vengono aggregati matematicamente in un'unica convoluzione 3×3 . Questa strategia elimina le connessioni residue e i rami paralleli che normalmente richiedono numerosi accessi alla memoria, con conseguenti miglioramenti in termini di efficienza.



images/SOTA/RepViT.pdf

Figura 3.16: Evoluzione dell'architettura MobileViT in RepViT da destra a sinistra. Fonte: [69].

L'evoluzione macro-architetturale da *MobileNetV3* a *RepViT* si articola attraverso una completa ristrutturazione della rete volta a eliminare le operazioni ridondanti tipiche delle vecchie CNN, ottenendo un forte semplificazione. Questo è evidenziato ulteriormente dalla Figura 3.16. A queste modifiche strutturali si aggiungono micro-ottimizzazioni come l'uso della funzione di attivazione *GeLU*, più efficiente per le applicazioni mobile. Grazie a questa architettura, RepViT-M1.0 è il primo modello a superare l'80% di accuratezza sul dataset *ImageNet* con una latenza di appena 1.0 ms su iPhone 12.

3.1.3 Neural Architecture Search

Uno dei principali problemi nell'ottimizzazione architetturale è rappresentato dalla complessità del processo di design, che richiede un'estesa sperimentazione e una profonda conoscenza del dominio. Per semplificare questo processo, sono stati sviluppati metodi di **Neural Architecture Search (NAS)**, che automatizzano il processo di ricerca di architetture ottimali in termini di costi e prestazioni.

AutoFormer

Per quanto riguarda i *Vision Transformer*, uno dei primi lavori ad affrontare il problema è stato quello di Chen et al. [8] con i loro modelli **AutoFormer**. Gli autori formulano il problema di ricerca secondo una tecnica *One Shot*, ovvero non creano molteplici reti che vengono addestrate singolarmente e successivamente valutate, invece definiscono una cosiddetta *SuperNet*, una rete sovradimensionata che contiene al suo interno tutte le possibili architetture candidate (*Subnet*) e la addestrano complessivamente, addestrando intrinsecamente tutte le candidate.

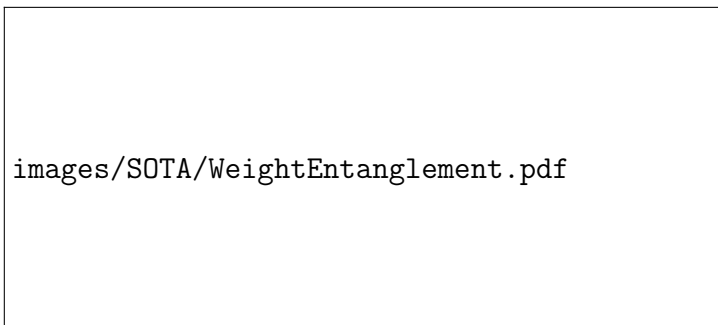
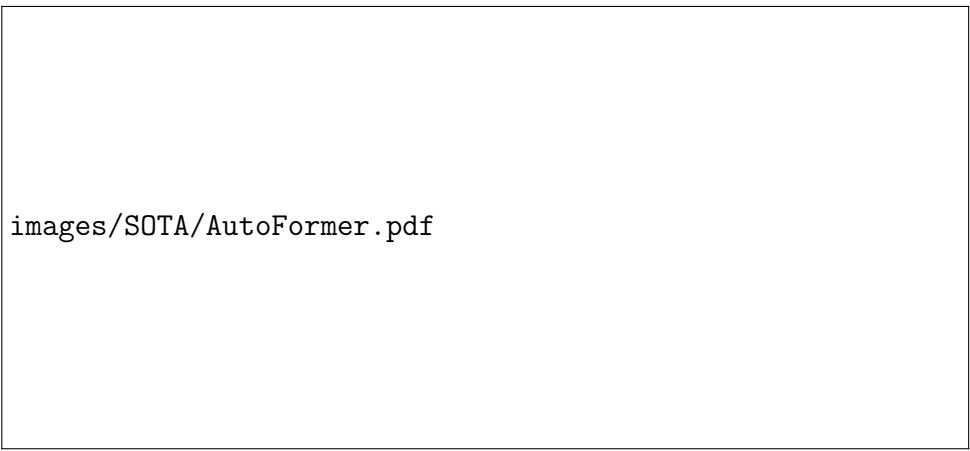


Figura 3.17: Differenza tra *Weight Sharing* e *Weight Entanglement*. Fonte: [8].

La grande introduzione di questo lavoro è proprio il meccanismo di **Weight Entanglement**, che non crea lo spazio di ricerca attraverso l'introduzione di blocchi paralleli e separati (*Weight Sharing*) che vengono attivati o disattivati a seconda dell'architettura candidata. I pesi di tutte le reti candidate all'interno di un unico blocco sono condivisi in modo che ogni blocco della *SuperNet* venga addestrato da tutte le candidate che lo contengono. Questo approccio si è dimostrato in grado di addestrare la *SuperNet* in un tempo molto più breve rispetto al metodo classico, grazie ad una convergenza più rapida e stabile, ottenendo inoltre prestazioni superiori.



images/SOTA/AutoFormer.pdf

Figura 3.18: Visualizzazione della *Supernet* di AutoFormer (A sinistra la rete complessiva, a destra il singolo blocco). Fonte: [8].

Lo spazio di ricerca definito dagli autori, contenuto all'interno della *Supernet*, prevede la possibilità di variare la dimensione dell'*embedding*, la dimensione di *query-key-value*, il numero di teste di attenzione, la dimensione del feed-forward network e il numero di blocchi. In uno spazio di questo tipo una *Subnet* è definita da una specifica combinazione di valori discreti, selezionati in fase di ricerca all'interno di intervalli predefiniti, per ognuno di questi parametri.

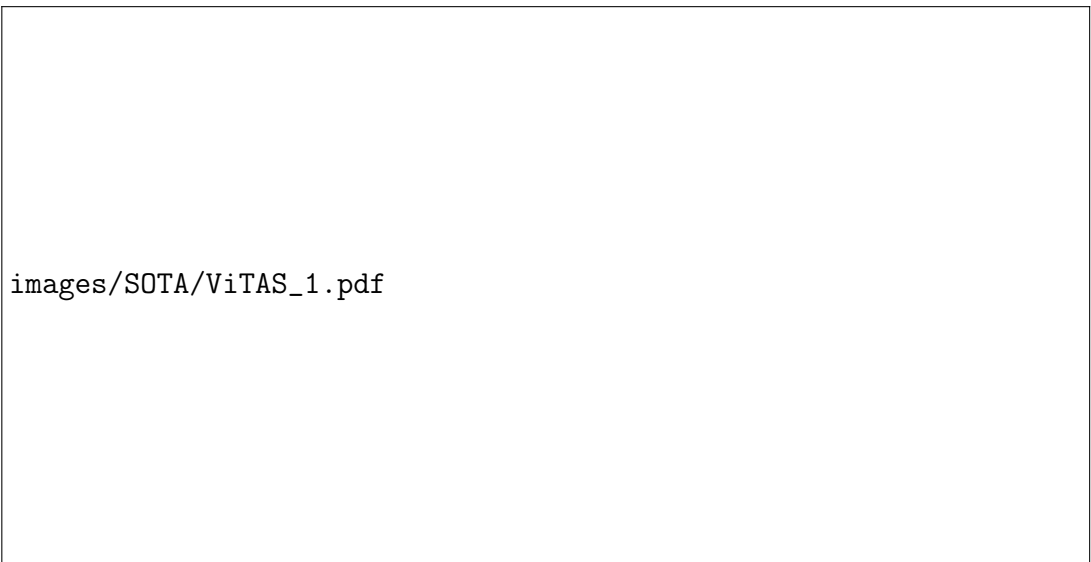
Infine, l'algoritmo di ricerca selezionato dagli autori è di tipo evolutivo ed utilizza operazioni di mutazione ed incrocio tra reti candidate per esplorare in maniera efficiente lo spazio di ricerca, guidato da una funzione obiettivo che bilancia accuratezza e numero di parametri. Notare inoltre che, a valle del processo di ricerca, non è necessario eseguire un ulteriore processo di addestramento, in quanto le *Subnet* condividono i pesi addestrati della *Supernet* e sono già in grado di raggiungere prestazioni elevate. Grazie a questo approccio, gli autori riescono a identificare architetture che raggiungono prestazioni superiori rispetto a modelli di riferimento come DeiT e ViT, con un numero di parametri e operazioni in virgola mobile (FLOPs) inferiori.

ViTAS

Un approccio radicalmente diverso è quello proposto da Su et al. con **ViTAS** [62], un framework nato dalla considerazione che l'applicazione diretta del NAS classico ai *Vision Transformer* produce spesso risultati instabili durante l'addestramento del *SuperFormer*. Secondo gli autori, la causa risiede principalmente nell'addestramento sbilanciato dei vari canali preposti

nella macro-architettura, in modo tale da danneggiare il meccanismo di *weight sharing*. Per stabilizzare questo processo, gli autori introducono due contributi principali:

- Un nuovo meccanismo di **Cyclic Weight-Sharing** che garantisce un addestramento equo di tutti i canali, indipendentemente dalla dimensione delle sottoreti campionate in fase di training.
- Una strategia di **Identity Shifting** che elimina le ridondanze e le ambiguità all'interno dello spazio di ricerca, obbligando le operazioni di identità a collocarsi solo nei livelli più profondi di ogni stadio.



images/SOTA/ViTAS_1.pdf

Figura 3.19: Confronto tra le forme di *weight sharing*. Da sinistra: Ordinale, Bilaterale e Ciclico. Fonte: [62].

Il problema principale riscontrato nel *weight sharing* tradizionale è il cosiddetto bias di addestramento dei canali. Nelle tecniche comuni, si utilizza un meccanismo ordinale: se una sottorete necessita di i canali, durante il campionamento delle sottoreti in fase di training, vengono selezionati i primi i canali da sinistra. Ciò significa che i canali posizionati all'inizio della matrice vengono addestrati molto più spesso di quelli finali, creando una disparità enorme nella loro ottimizzazione. Lo stesso fenomeno occorre nel caso di meccanismo bilaterale, in cui i canali vengono selezionati simmetricamente a partire da entrambi i lati della matrice. In questo caso, i canali centrali vengono addestrati più spesso di quelli periferici.

ViTAS risolve questo problema introducendo il *Cyclic Weight-Sharing* che punta ad una uniformità dell'addestramento: ogni canale deve ricevere lo stesso numero di aggiornamenti indipendentemente dalla dimensione della sottorete campionata. Un ulteriore obiettivo è l'uniformità dell'influenza: ogni canale deve avere un impatto equivalente sul gradiente totale. Al fine di ottenere questo bilanciamento viene formulato un problema di ottimizzazione (*Quadratically Constrained Quadratic Program*) (QCQP).

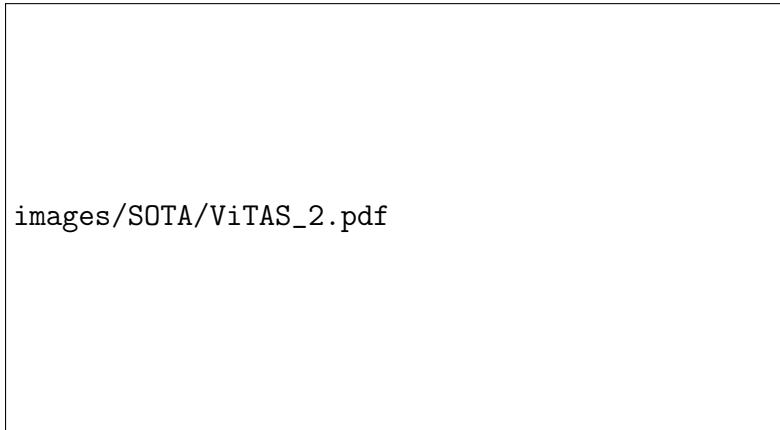


Figura 3.20: Problema della ridondanza dello spazio di ricerca, e tecnica di *Identity Shifting*. Fonte: [62].

La Strategia di *Identity Shifting* viene introdotta dagli autori al fine di risolvere la ridondanza degli spazi di ricerca NAS, osservabile in Figura 3.20. In uno spazio di ricerca è possibile che percorsi diversi all'interno del *Superformer* possano corrispondere alla stessa architettura finale (ad esempio, scambiare l'ordine di due blocchi identità non cambia il modello). Questa ambiguità arricchisce lo spazio di ricerca ma di architetture duplicate, complicando il processo e prolungando il tempo necessario. Con l'*Identity Shifting*, gli autori predispongono le operazioni di identità solo nei livelli più profondi di ogni stadio, eliminando le ridondanze e riducendo drasticamente lo spazio di ricerca (in un esempio citato, da 3^{12} a $2^{13} - 1$ combinazioni possibili [62]).

La ricerca viene guidata dall'algoritmo evolutivo multi-obiettivo NSGA-II [15], capace di ottimizzare contemporaneamente l'accuratezza e i vincoli hardware come i FLOPs. I risultati confermano la validità di questa rigorosa impostazione: con un budget di soli 3.0G FLOPs, *ViTAS* raggiunge un'accuratezza dell'82,0% sul dataset ImageNet-1k, superando anche le prestazioni di *AutoFormer*.

NASViT

Un ulteriore contributo in questo ambito è quello di Gong et al. [22] con il loro modello **NASViT**. Gli autori osservano che, provando ad addestrare una *SuperNet* (ViT) che contiene tutte le possibili architetture candidate, si riscontra un problema di instabilità e scarse prestazioni. La causa di queste difficoltà può essere individuata nel conflitto tra i gradienti delle *SubNet* e quelli della *SuperNet*. Questo fenomeno implica che, addestrando la *SuperNet* campionando le varie *SubNet*, si ottengono risultati inferiori dalla rete complessiva.

Gli autori risolvono il problema proiettando i gradienti della *SuperNet* lungo la direzione ortogonale a quelli delle *SubNet* esclusivamente quando la similarità coseno tra i due è negativa, in modo da evitare interferenze e garantire un addestramento più stabile e performante. L'unico svantaggio di questo approccio risulta il bias dei gradienti verso le *SubNet*, il quale ha avuto un impatto negativo sui tempi di convergenza.



Figura 3.21: Switchable Scaling Layer introdotti all'interno dei blocchi *Transformer*. Fonte: [62].

Viene introdotto inoltre lo **Switchable Scaling Layer** che permette di scalare dinamicamente le feature in uscita dalle componenti dei blocchi *Transformer* che vengono selezionati in fase di ricerca, come è possibile notare dalla Figura 3.21. In particolare ognuno di questi layer apprende una matrice di pesi W_{scale} che viene utilizzata per proiettare l'output (x) della *Self-Attention* o del MLP, fornendo in output $W_{\text{scale}} \cdot x$. Questo meccanismo si è dimostrato efficace per aumentare la capacità e la flessibilità delle *SubNet*.

Risulta importante notare che gli autori hanno osservato un aumento del fenomeno dei gradienti in conflitto tra le *SubNet* e la *SuperNet* quando si adottano tecniche di *Augmentation* e *Regularization* aggressive.

Per quanto concerne i risultati, gli autori utilizzano le tecniche di cui sopra con un algoritmo evolutivo per esplorare lo spazio di ricerca, guidato da una funzione obiettivo che bilancia accuratezza e FLOPs. Grazie a questo approccio, i modelli NASViT riescono a raggiungere e superare le prestazioni delle reti convoluzionali (ad esempio ResNet [27]) a parità di FLOPs.

MDL-NAS

Un'ulteriore prospettiva della NAS applicata ai ViT, focalizzata sull'efficienza e sull'apprendimento multi-dominio, è stato proposto da Wang et al. [71] con il framework **MDL-NAS**. Gli autori osservano che, per il *deploy* su dispositivi con risorse limitate (*Edge Computing*), addestrare i modelli per ogni specifico dominio visivo risulta estremamente inefficiente. D'altra parte, il semplice addestramento multi-dominio (*Multi-Domain Learning*, MDL) di un'unica rete incontra spesso due gravi ostacoli: il cosiddetto *catastrophic forgetting* (totale discostamento dal dominio originale per lasciare spazio al nuovo) e l'interferenza tra domini (*negative transfer*), i quali provocano delle prestazioni limitate.

Per risolvere questi problemi, gli autori introducono un framework unificato che integra molteplici task visivi all'interno di un'unica *SuperNet*. L'esplorazione avviene tramite uno spazio di ricerca *coarse-to-fine* (da macro a micro). A livello macroscopico, l'algoritmo seleziona l'architettura di base per i diversi task, determinando iperparametri strutturali quali la profondità della rete, il numero di teste di attenzione e le dimensioni dell'embedding.

La vera innovazione si concretizza nello spazio di ricerca a livello micro, ideato per implementare una condivisione dei parametri definita *fine-grained*. L'obiettivo è decidere in modo mirato quali pesi della rete debbano essere condivisi tra tutti i domini (per massimizzare l'efficienza in termini di memoria) e quali debbano rimanere indipendenti (per evitare che i task interferiscano negativamente tra loro). Come illustrato nella Figura 3.22, vengono proposte due politiche per la gestione dei pesi all'interno dei singoli layer:

- **Sequential Sharing Policy:** la matrice dei pesi viene suddivisa in blocchi adiacenti (sequenziali). L'algoritmo decide se intere porzioni della matrice debbano essere condivise globalmente o allocate unicamente a uno specifico task.

- **Mask Sharing Policy:** un approccio ancora più flessibile. Al posto di usare blocchi sequenziali, viene introdotta una maschera binaria apprendibile, avente la stessa dimensionalità della matrice dei pesi. Questa maschera agisce a livello di singolo parametro, determinando dinamicamente se debba essere globale o specializzato per il singolo task.

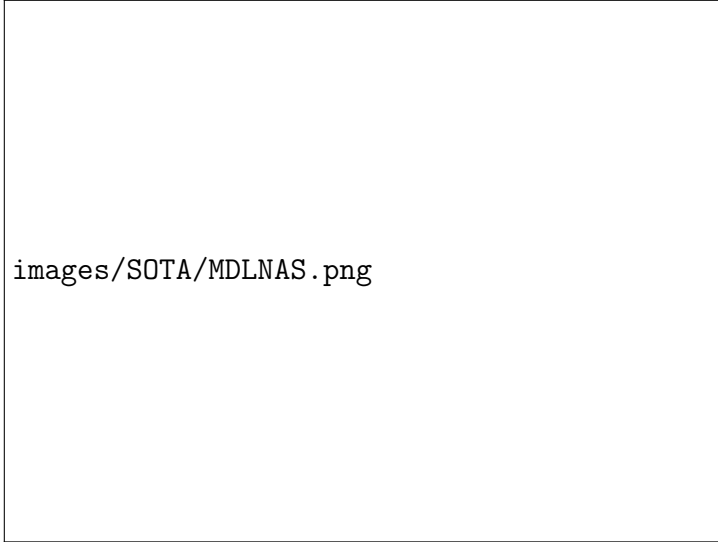


Figura 3.22: Visualizzazione delle politiche di condivisione dei parametri in MDL-NAS: *Sequential Sharing* e *Mask Sharing*. Fonte: [71].

Nel trovare una soluzione a questo problema di ottimizzazione emerge un ostacolo tecnico: decidere se un parametro debba essere condiviso o meno (scelta binaria), così come scegliere il numero di layer, sono operazioni di natura *discreta* e, di conseguenza, non differenziabili. Per aggirare questo ostacolo e permettere l'addestramento della rete tramite la discesa del gradiente, MDL-NAS sostituisce l'utilizzo degli algoritmi evolutivi, visti di frequente in precedenza, con una ricerca differenziabile basata sul *Gumbel-Softmax trick* [32, 45].

Grazie a ciò, è possibile effettuare un'ottimizzazione alternata (*alternating optimization*): ad ogni iterazione, l'algoritmo aggiorna prima i pesi effettivi della *SuperNet* tramite i dati di training, e successivamente aggiorna i parametri architetturali (ovvero le probabilità che gestiscono le maschere di condivisione e la struttura macro) tramite i dati di validazione. I risultati dimostrano che MDL-NAS riesce a ottenere prestazioni all'avanguardia su benchmark multi-dominio, generando modelli altamente efficienti in termini di memoria che abbattano drasticamente il numero di parametri totali rispetto all'utilizzo di reti ViT indipendenti.

DASViT

Un recente e rilevante contributo nell'ambito dell'ottimizzazione architetturale tramite ricerca differenziabile per i Vision Transformer è stato proposto da Wu et al. [74]. Gli autori introducono **DASViT** (*Differentiable Architecture Search for Vision Transformer*), con l'obiettivo di superare i requisiti computazionali e le tempistiche proibitive associati ai tradizionali algoritmi di ricerca evolutiva.

Gli autori osservano che l'applicazione diretta del noto algoritmo differenziabile DARTS [41] ai modelli ViT comporta due criticità fondamentali. La prima riguarda l'insostenibile consumo di memoria GPU, causato dalla necessità di istanziare e calcolare simultaneamente tutte le operazioni candidate per ogni blocco, limite ulteriormente aggravato dalla complessità quadratica della *Self-Attention*. La seconda criticità risiede in un intrinseco sbilanciamento generato dalla funzione *softmax* durante la selezione delle operazioni (*operation unfairness*). Nelle fasi iniziali dell'addestramento, le operazioni prive di parametri, come le *skip-connection* (*Identity*), risultano più facili da ottimizzare poiché offrono minor resistenza al flusso dei gradienti. La funzione *softmax* tende quindi a massimizzare prematuramente i pesi di queste operazioni a discapito di moduli complessi, come la *Self-Attention* e il MLP, i quali sono però fondamentali per la capacità rappresentativa della rete. Questo fenomeno conduce a reti dalle scarse prestazioni, in cui l'architettura degenera in una rete quasi interamente composta da connessioni dirette.

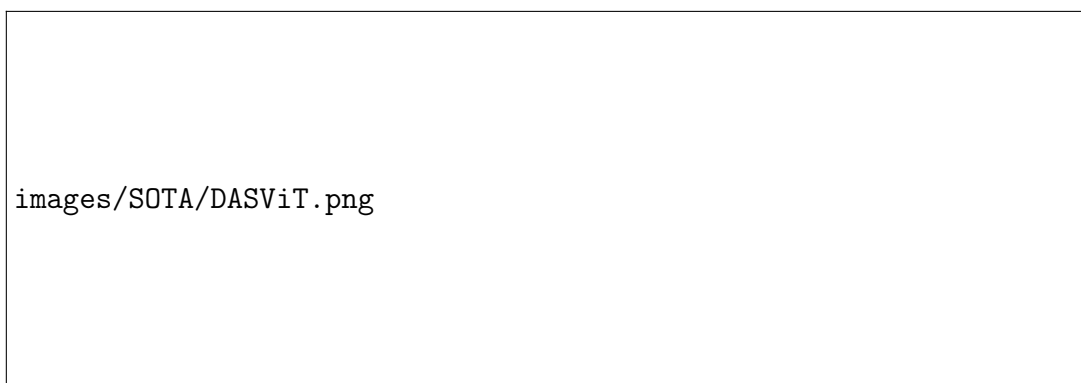


Figura 3.23: Rappresentazione dello spazio di ricerca continuo in DASViT tramite Grafo Aciclico Diretto (DAG) e selezione finale dell'architettura. Fonte: [74].

Per formulare la ricerca in modo efficiente e superare il problema della non differenziabilità delle scelte discrete, DASViT modella l'architettura dell'encoder *Transformer* come un Grafo Aciclico Diretto (DAG) (Figura 3.23). All'interno di questo grafo, i nodi rappresentano gli stati

delle *features*, mentre gli archi orientati che li collegano definiscono le trasformazioni matematiche da applicare a queste. Negli approcci standard, la scelta dell'operazione da assegnare a un arco è esclusiva e rigida. Al fine di rendere il problema ottimizzabile, DASViT applica il principio del rilassamento continuo dello spazio di ricerca (*continuous relaxation*): su ogni arco non viene posta una singola operazione, bensì una combinazione parallela di *tutte* le operazioni candidate (es. MSA, MLP, *Identity*, *Zero*), i cui output vengono fusi tramite una somma pesata.

I coefficienti di questa somma pesata costituiscono i parametri architetturali continui, che vengono appresi attivamente dalla rete. In questo modo, l'algoritmo valuta simultaneamente l'impatto di tutte le alternative su un singolo arco e, attraverso la *backpropagation*, impara ad assegnare pesi maggiori alle operazioni che minimizzano la funzione di perdita. Al termine della fase di ricerca, viene mantenuta esclusivamente l'operazione con il peso maggiore, discretizzando nuovamente il DAG in un'architettura definitiva.

Per risolvere i due limiti operativi evidenziati in precedenza (memoria e sbilanciamento) e stabilizzare questo processo per i ViT, gli autori introducono due meccanismi specifici:

- **Attention-based Partial Token Selection:** per contenere l'eccessivo consumo di memoria durante la ricerca, questo modulo riduce il numero di token elaborati. Basandosi sui punteggi calcolati dalla *Self-Attention*, il sistema seleziona dinamicamente e propaga solo i token più informativi (*top-k*), escludendo quelli ridondanti dal calcolo simultaneo delle operazioni candidate.
- **Operation Fairness Regularization:** per contrastare il dominio delle *skip-connection*, viene introdotta una regolarizzazione sulla distribuzione dei pesi architetturali. Attraverso l'imposizione di soglie e penalità calibrate per le diverse tipologie di operazioni, il framework garantisce che moduli complessi come MSA e MLP abbiano eque probabilità di essere selezionati e addestrati, garantendo un'esplorazione stabile dello spazio degli stati.

L'algoritmo viene infine integrato in una strategia di ricerca progressiva, in cui la profondità della rete viene gradualmente incrementata per ridurre la discrepanza strutturale tra la fase di ricerca e la successiva fase di addestramento (*retraining*). I risultati sperimentali confermano la validità del metodo: DASViT scopre architetture innovative che si discostano dai rigidi design tradizionali. Nelle valutazioni sui dataset CIFAR-10, CIFAR-100 e ImageNet-100, i modelli generati superano le prestazioni di un ViT-B/16 standard addestrato da zero, conseguendo al contempo una riduzione del 41% del numero di parametri e un abbattimento significativo del costo computazionale (FLOPs).

3.2 Metodi di Compressione

In questa sezione vengono presentati i principali metodi di compressione dei modelli, che mirano a ridurre la complessità computazionale e la memoria richiesta per l'esecuzione dei modelli *Transformer*, rendendoli più adatti per l'edge computing. A differenza di quanto visto nella sezione precedente, questi metodi non si basano sulla costruzione di architetture, bensì sulla riduzione e semplificazione di architetture pre-esistenti ed in alcuni casi anche pre-addestrate.

3.2.1 Pruning

Una delle possibili tecniche di compressione è il **Pruning**, che consiste nell'identificare e rimuovere i parametri ridondanti e meno importanti di un modello, come neuroni, canali o intere unità di attenzione. Il fine di questa tecnica è la riduzione dello spazio in memoria e del costo computazionale richiesti dal modello, senza compromettere significativamente le prestazioni. Esistono due tipologie principali di pruning:

- **Pruning Strutturato**: rimuove intere unità strutturali, come neuroni, canali o interi blocchi. Questo tipo di *Pruning* risulta in incrementi di velocità tipicamente importanti, in quanto le matrici dei parametri vengono effettivamente ridotte, ma è più complesso mantenere invariate le prestazioni dei modelli, in quanto vengono rimosse intere strutture e non singoli parametri.
- **Pruning Non Strutturato**: rimuove singoli parametri, portando tipicamente a matrici sparse dei pesi. Questo tipo di *Pruning* può mantenere prestazioni comparabili a quelle originali, ma spesso non porta a miglioramenti significativi in termini di latenza, in quanto le operazioni sparse non sono sempre supportate in modo efficiente dagli hardware e le dimensioni dei modelli non vengono ridotte in modo significativo.

Nello specifico, per quanto riguarda i modelli *Transformer*, un'altra tecnica di *Pruning* molto esplorata è quella del **Token Pruning**, che mira a ridurre il numero di token elaborati in fase di inferenza, eliminando quelli ritenuti meno informativi. Questa strategia risulta particolarmente efficace dato il suo diretto impatto sul meccanismo di attenzione. Quest'ultimo infatti è caratterizzato da una complessità computazionale quadratica rispetto al numero di token, di conseguenza, ridurre questo porta a significativi miglioramenti in termini di latenza. Di seguito una panoramica dei principali lavori in questo ambito.

SViTE

Tradizionalmente i metodi di *pruning* richiedono l'addestramento dell'intero modello denso prima di poterne rimuovere le ridondanze, un processo che si traduce in costi computazionali e di memoria, in particolar modo per i *Vision Transformer*. Per ovviare a questo problema, Chen et al. propongono **SViTE** (*Sparse Vision Transformer Exploration*) [9], un framework che utilizza il concetto di sparsità, addestrando dinamicamente una sottorete sparsa sin dalle prime fasi dell'inizializzazione e mantenendo fisso un budget di parametri ridotto per l'intera durata del training.

Il lavoro degli autori si articola in tre contributi fondamentali:

- **SViTE (Pruning Non Strutturato)**: Invece di addestrare un modello denso per poi compimerlo, questa variante inizializza la rete con una struttura già sparsa (molti parametri sono posti a zero fin dall'inizio). Durante l'addestramento, le connessioni attive non rimangono fisse, ma vengono aggiornate dinamicamente tramite un ciclo di *prune-and-grow*. A intervalli regolari, il sistema identifica i pesi attivi meno importanti, ovvero quelli con il valore assoluto più basso, e li disattiva. Subito dopo, per mantenere rigorosamente costante il livello di sparsità globale, la rete riattiva parametri precedentemente spenti. Queste nuove attivazioni vengono scelte in base alla magnitudo del loro gradiente: vengono cioè riattivati i pesi che promettono di ridurre maggiormente l'errore del modello. Questo approccio permette alla rete di esplorare e trovare la topologia sparsa ottimale direttamente durante l'addestramento.
- **S²ViTE (Pruning Strutturato)**: Poiché la sparsità non strutturata raramente si traduce in concreti vantaggi, misurabili in fase di inferenza, il framework viene esteso alla sua variante strutturata, *S²ViTE (Structured Sparse ViT Exploration)*. In questo caso, il pruning non agisce sui singoli pesi, ma rimuove intere teste di attenzione (*attention heads*) e specifici neuroni nei moduli MLP. L'importanza delle singole teste viene stimata tramite un'approssimazione del primo ordine (basata sull'espansione di Taylor) della funzione di perdita, mentre l'importanza dei neuroni MLP viene valutata tramite la norma L_1 , calcolata sulla base dei rispettivi parametri.
- **SViTE+ (Token Pruning)**: Per massimizzare ulteriormente l'efficienza, la sparsità architetturale viene unita alla riduzione del numero di token. A tale scopo, viene introdotto un selettore di token addestrabile che analizza le *patch* dell'immagine in ingresso per

mantenere attivi solo i *top-k* token più informativi scartando il background o eventuali ridondanze. Questo ha un forte impatto proprio sul meccanismo di attenzione, riducendo drasticamente la complessità computazionale, che è quadratica rispetto al numero di token.

I risultati si dimostrano ottimi. Applicando la variante S^2ViTE su un'architettura DeiT-Small, rimuovendo il 30% dei parametri, il modello ottiene una riduzione del 31% dei FLOPs e del 22% del tempo di inferenza. Per quanto riguarda l'accuratezza, la rete compressa non subisce un pesante degrado prestazionale (0.6%) rispetto alla sua controparte densa originale.

WDPruning

La maggior parte dei metodi di *pruning* strutturato per i *Vision Transformer* si concentra prevalentemente sulla riduzione della larghezza della rete, attraverso il taglio di canali o specifiche teste di attenzione. Tuttavia, Yu et al. fanno notare come la riduzione della profondità (ovvero la rimozione di interi blocchi Transformer) risulti spesso più vantaggiosa per l'accelerazione hardware, in quanto garantisce una maggiore efficienza di parallelizzazione e una notevole riduzione di parametri. Per colmare questa lacuna, gli autori propongono **WDPruning** (*Width & Depth Pruning*) [77], un framework in grado di comprimere simultaneamente in larghezza e in profondità un modello ViT, addestrando il modello compatto direttamente, senza necessitare di successive fasi di *fine-tuning*.



Figura 3.24: *Pruning* in larghezza proposto da Yu et al in WDPruning. Vengono compresse le teste di attenzione e i moduli MLP. Fonte: [77].

Per quanto riguarda il *pruning* in larghezza, vengono compressi i layer lineari e i moduli di *Multi-Head Self-Attention* (Figura 3.24s). In particolare, il metodo associa una maschera binaria e un *saliency score* (punteggio di importanza) addestrabile a ogni dimensione (singolo

neurone oppure testa di attenzione). Durante l'addestramento, i canali il cui punteggio risulta inferiore a una soglia vengono azzerati. Questa soglia viene resa specifica per ogni blocco Transformer e diventa inoltre un parametro differenziabile e addestrabile tramite l'ausilio di una funzione sigmoide, permettendo al modello di apprendere autonomamente un tasso di sparsità non uniforme e ottimizzato per ogni singolo layer. Per vincolare la rete a rispettare un *budget*, ovvero un obiettivo di *pruning* globale stabilito a priori, viene impiegato il metodo del Lagrangiano Aumentato direttamente nella funzione di costo.

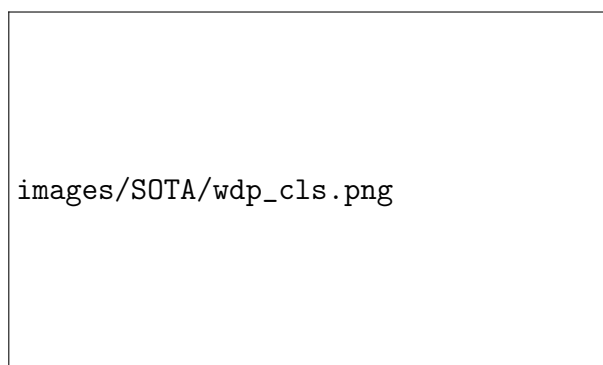


Figura 3.25: *Pruning* in profondità attraverso *shallow classifiers* in WDPPruning. A sinistra la configurazione in fase di addestramento e *pruning*, a destra quella in fase di inferenza. Fonte: [77].

Al fine di comprimere la rete in profondità, gli autori sfruttano la proprietà dei token di classificazione ('[CLS]') di accumulare rappresentazioni semantiche utili alla predizione già nei blocchi iniziali. Vengono quindi inseriti dei classificatori ausiliari (*shallow classifiers*) che hanno come input i token '[CLS]' dei blocchi intermedi (Figura 3.25). Questi classificatori vengono addestrati congiuntamente al resto della rete. In fase di inferenza, è possibile selezionare il classificatore intermedio che offre il miglior compromesso tra latenza e accuratezza, permettendo di scartare tutti i blocchi Transformer successivi.

I risultati sperimentali sul dataset ImageNet-1K dimostrano che l'ottimizzazione congiunta di queste due dimensioni porta a risultati interessanti. Applicando il metodo all'architettura DeiT-Base, gli autori sono riusciti a rimuovere circa il 20% dei parametri e oltre il 22% dei FLOPs, ottenendo un incremento del *throughput* del 10%. Sorprendentemente, questo livello di compressione ha prodotto un lieve incremento dell'accuratezza Top-1 (+0.14%) rispetto al modello denso di base. Per quanto riguarda i modelli di dimensioni ridotte (DeiT-Small e DeiT-Tiny), rimuovendo il 30% dei parametri si notano ottimi miglioramenti in termini di latenza e FLOPs, ma con un degrado in termini di accuratezza più significativo (superiore all'1%).

GOHSP

Un limite fondamentale degli approcci tradizionali di *pruning* applicati ai *Vision Transformer* risiede nell'errata assunzione che l'architettura possa essere compressa in maniera omogenea. Yin et al. [76] evidenziano come i modelli ViT siano invece intrinsecamente caratterizzati da una **sparsità strutturata eterogenea**. Nello specifico, il modulo di attenzione permette una compressione su due livelli di granularità, a livello di intere teste (*head-level*) e a livello di singole colonne delle matrici dei pesi (*column-level*); mentre il modulo MLP esclusivamente a livello di colonna. Questa multi-granularità rende inadeguate le metriche di *pruning* convenzionali: valutando l'importanza delle dimensioni in modo isolato, tali metriche falliscono nel catturare le complesse interazioni globali tra i diversi componenti della rete.



images/SOTA/gohsp.png

Figura 3.26: Rappresentazione grafica del metodo di *pruning* introdotto da Yin et al. in GOHSP. Sono visibili le tre fasi distinte (da sinistra a destra) del processo. Fonte: [76].

Per risolvere questa problematica, gli autori propongono **GOHSP** (*Graph and Optimization-based Heterogeneous Structured Pruning*, Figura 3.26) [76], un framework che affronta la compressione attraverso tre fasi sequenziali:

- **Graph-based Head Ranking:** Invece di valutare ogni testa di attenzione singolarmente, gli autori analizzano come queste interagiscono tra loro, nello specifico layer, costruendo un grafo in cui ogni nodo rappresenta una testa. Il collegamento (arco) tra due teste ha un peso proporzionale alla similarità coseno tra i loro output: maggiore è la similarità tra i risultati di due teste su un batch di dati, maggiore è la loro relazione. Questo sistema viene modellato come una **Catena di Markov**, in modo tale da poter analizzare globalmente le relazioni tra le teste. In questo contesto, la **distribuzione stazionaria** rappresenta

la probabilità finale di trovarsi su una determinata testa dopo aver effettuato un numero infinito di transizioni sul grafo. Intuitivamente, una testa con probabilità stazionaria elevata indica che questa produce un output fortemente simile a quello delle altre teste della rete. Al contrario, teste con una probabilità stazionaria minore risultano isolate o meno rappresentative dell'informazione globale, venendo quindi identificate come candidate per la rimozione. Tuttavia, eliminarle (un approccio rigido, o *hard-pruning*) causerebbe un decremento immediato dell'accuratezza. Per evitare questo problema, viene adottata una strategia graduale: ai pesi di queste teste vengono attribuite delle penalità nella funzione di costo della fase successiva. Questa operazione non azzerà i parametri bensì li costringe a tendere gradualmente verso lo zero durante le iterazioni di addestramento. In questo modo si realizza il cosiddetto *soft-pruning*.

- **Optimization-based Soft Pruning (Ottimizzazione Globale):** Questa fase ha il delicato compito di imporre la **sparsità eterogenea**. L'algoritmo non si limita infatti a penalizzare le teste suggerite dal *ranking*, ma deve simultaneamente ottimizzare la rimozione delle singole colonne, sia nei moduli di attenzione sia nei moduli MLP, al fine di rispettare il *budget* di compressione prefissato. Per gestire questi molteplici vincoli spaziali di diversa granularità all'interno di un unico problema di ottimizzazione (minimizzando al contempo la funzione di costo), viene impiegato il metodo di separazione di *Douglas-Rachford* [20]. Questo algoritmo adotta un approccio iterativo duplice: ad ogni iterazione, i pesi vengono dapprima aggiornati liberamente per minimizzare l'errore del modello; successivamente, l'algoritmo impone il rispetto del *budget* di *pruning* e, tramite le penalità sulla funzione di perdita, riduce verso lo zero i pesi che risultano meno rilevanti, gestendo attenzione ed MLP in modo indipendente. Questa alternanza scinde l'ottimizzazione di due obiettivi spesso contrastanti, evitando gradienti instabili e garantendo una convergenza stabile.
- **Fase 3: Hard-Pruning e Fine-Tuning:** Al termine del processo di ottimizzazione, i pesi e le strutture identificate come superflue avranno raggiunto valori prossimi allo zero. A questo punto viene eseguito il *pruning* strutturato definitivo, seguito da alcune epoche di *fine-tuning* per recuperare l'eventuale accuratezza persa.

I risultati sperimentali evidenziano la superiorità di questo approccio. Sul dataset ImageNet, applicando GOHSP al modello DeiT-Small con un tasso di *pruning* del 40%, il modello ottiene un'accuratezza del 79.98%, superando leggermente sia il modello base denso (79.90%) sia i

metodi strutturati concorrenti come S²ViTE. Spingendo la sparsità al 50%, il modello riesce comunque a eguagliare le prestazioni della controparte densa originale, garantendo al contempo un miglioramento del tempo di inferenza (*run-time*) superiore al 26%.

X-Pruner

Un'ulteriore problematica nell'ambito della compressione dei *Vision Transformer* è rappresentata dalla natura non spiegabile della maggior parte dei metodi di *pruning* attuali. Molti approcci, infatti, rimuovono i parametri basandosi su metriche in funzione del gradiente o della magnitudo dei pesi, ignorando del tutto la relazione semantica che intercorre tra le singole parti del modello e le classi da predire. Questo provoca l'eliminazione di componenti semanticamente rilevanti per la rete, con un conseguente calo delle prestazioni. Per affrontare questa limitazione, Yu e Xiang [78] propongono **X-Pruner** (*eXplainable Pruner*), un framework innovativo che integra i principi dell'*eXplainable AI* (XAI) all'interno di un processo di *pruning* strutturato. L'obiettivo è identificare e preservare unicamente le unità, come le teste di attenzione o le singole colonne delle matrici MLP, che contribuiscono attivamente al riconoscimento di specifiche classi.

Invece di valutare l'importanza di una dimensione in modo assoluto, l'architettura di *X-Pruner* assegna a ciascuna unità una maschera differenziabile per ogni singola classe, definita **Explainability-aware Mask**, la quale stabilisce il contributo delle singole unità alla predizione di una classe ben precisa. Durante l'addestramento, guidato dalle etichette reali (*ground-truth*), il modello aggiorna i valori di queste maschere, utilizzando inoltre una regolarizzazione tramite due vincoli. Il primo, detto di *smoothness*, penalizza la norma L_1 della derivata seconda per evitare variazioni ingiustificate e fluttuazioni tra immagini appartenenti alla stessa classe. Il secondo è invece un vincolo di sparsità, basato sulla norma L_2 delle maschere, che penalizza le distribuzioni uniformi costringendo la rete a concentrare l'importanza solo su un sottoinsieme limitato di unità.

Un ulteriore contributo importante del metodo risiede nel modo in cui queste maschere specifiche per classe vengono utilizzate per il taglio. Le maschere di ogni classe vengono sommate per ottenere un singolo punteggio di importanza per ciascuna unità. Per decidere quali unità eliminare in base a tali punteggi, i metodi tradizionali si affidano a funzioni a gradino binarie che, avendo derivata nulla quasi ovunque, ostacolano la *backpropagation*. *X-Pruner* supera questo limite introducendo un'operazione di *pruning* differenziabile basata sulla funzione tangente

iperbolica ($\tanh$), la quale agisce sulla differenza tra il valore di importanza unico dell'unità e una soglia di taglio specifica per ogni layer (anch'essa appresa). Essendo la $\tanh$ continua e differenziabile, il gradiente può fluire ininterrottamente, permettendo al framework di ottimizzare simultaneamente i pesi della rete e di apprendere dinamicamente la soglia di taglio per ogni layer. Tale processo di ottimizzazione è infine governato dal metodo della Lagrangiana aumentata, che bilancia automaticamente due obiettivi contrastanti: la minimizzazione dell'errore di predizione del modello e il rigoroso rispetto del budget di compressione globale prefissato.

I risultati sperimentali confermano l'efficacia di questa sinergia tra interpretabilità e compressione. Testato sul dataset ImageNet (ILSVRC-12), X-Pruner supera gli approcci concorrenti, come WDPruning [77]. Applicato all'architettura DeiT-Small, ad esempio, il framework riesce a ridurre i FLOPs di circa il 50% accusando un calo di accuratezza top-1 inferiore all'1%. Infine, l'analisi qualitativa tramite mappe visive (*visual explanation maps*) dimostra che, a differenza dei metodi tradizionali che tendono a danneggiare il meccanismo di attenzione della rete dopo la compressione, i modelli compressi con *X-Pruner* mantengono mappe di attivazione qualitativamente migliori.

CAIT

Un'ulteriore tipologia di *pruning* non ancora affrontata in questa sezione è quella che lavora sui token e cerca di ridurre il numero per velocizzare la fase di inferenza. Per questi metodi l'obiettivo non è esclusivamente la riduzione dei costi computazionali, ma anche mantenere l'integrità spaziale delle *feature map*, in modo tale che il modello compresso rimanga performante ed adeguato per task a livello di pixel, come la segmentazione semantica o il rilevamento di oggetti. Per risolvere questo problema, Wang et al. [70] propongono **CAIT**, un metodo di compressione congiunta che punta ad ottenere alta accuratezza, inferenza rapida e un'eccellente capacità di trasferimento verso nuovi task.

Per affrontare la ridondanza dei token preservando l'integrità spaziale dell'immagine, l'architettura di CAIT introduce una strategia chiamata **Asymmetric Token Merging (ATME)**. A differenza dei metodi di *pruning* non strutturato che scartano i token in modo sparso creando dei vuoti nelle mappe delle *feature*, ATME congiunge i token adiacenti, come è possibile notare dalla Figura 3.27.

images/SOTA/cait_atme.png

Figura 3.27: Operazioni di *pruning* dei token in CAIT tramite ATME. A sinistra l'operazione di *Horizontal Token Merging* (HTM), a destra quella di *Vertical Token Merging* (VTM). Fonte: [70].

Il processo si articola in due operazioni distinte: l'*Horizontal Token Merging* (HTM) e il *Vertical Token Merging* (VTM). Inizialmente, la sequenza monodimensionale dei token in ingresso a un blocco l , denotata come $X_l \in \mathbb{R}^{N_l \times C}$, viene riportata alla sua forma spaziale originale $\bar{X}_l \in \mathbb{R}^{H \times W \times C}$, dove H e W rappresentano altezza e larghezza dell'immagine. Nell'operazione HTM, l'algoritmo raggruppa e concatena coppie di token adiacenti lungo l'asse orizzontale, generando un tensore intermedio di dimensioni $H \times \frac{W}{2} \times 2C$. Per fondere queste informazioni, il tensore viene fatto passare attraverso un livello di normalizzazione e una proiezione lineare, $\tilde{X}_l = \text{Linear}(\text{LayerNorm}(\hat{X}_l))$, che comprime nuovamente la profondità dei canali alla dimensione originaria C . L'operazione speculare VTM agisce invece lungo l'asse verticale, concatenando i token per ottenere una griglia $\frac{H}{2} \times W \times 2C$, anch'essa successivamente riportata a C canali con la stessa metodologia. Al termine dell'operazione, la mappa viene nuovamente appiattita in una sequenza 1D di dimensioni $\frac{N_l}{2} \times C$.

La tecnica viene definita asimmetrica a causa della scelta architetturale di evitare la classica fusione dei token in blocchi quadrati e simmetrici 2×2 . Un approccio di questo tipo, infatti, comprimerebbe la risoluzione spaziale di un fattore 4 in un singolo passaggio, causando una perdita troppo drastica e improvvisa di dettagli locali. L'algoritmo ATME, invece, distribuisce questa compressione lungo la rete applicando in modo alternato le operazioni HTM (che fonde 1×2 token) e VTM (che fonde 2×1 token) in layer specifici, selezionati gerarchicamente per rispettare un preciso budget di FLOPs.

Poiché ogni singola operazione dimezza il numero di token intervenendo su un solo asse alla volta, la riduzione della griglia spaziale risulta molto più graduale e controllata. Per tale ragione, quando il modello viene trasferito su task *downstream*, queste mappe possono essere agevolmente riportate alla risoluzione originale tramite un semplice sovracampionamento (*upsampling*), evitando artefatti dovuti ad problemi di ricostruzione.

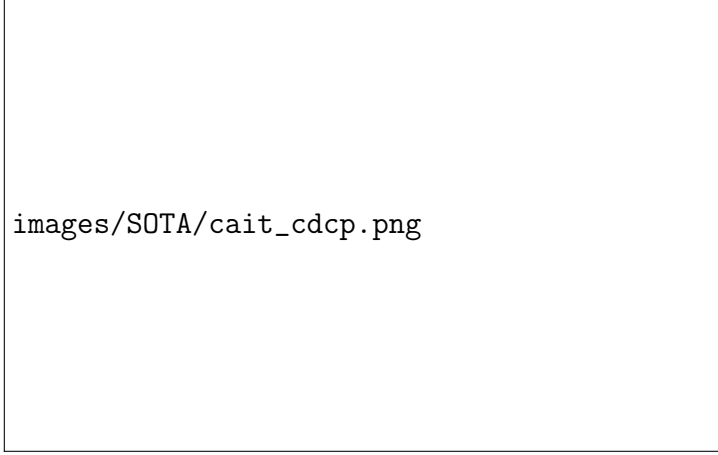


Figura 3.28: Rappresentazione grafica del meccanismo di *pruning* dei canali in CAIT tramite CDCP. A sinistra viene illustrato il funzionamento e l'aggiornamento dei *compactor*, mentre a destra vengono mostrati i vincoli a livello di teste e di attenzione. Fonte: [70].

Parallelamente alla compressione dei token, CAIT riduce la ridondanza dei parametri tramite la strategia **Consistent Dynamic Channel Pruning** (CDCP). Invece di eseguire un taglio statico e irreversibile, la CDCP valuta dinamicamente l'importanza dei canali introducendo per ciascun layer un modulo definito *compactor* (compattatore). Matematicamente, per ogni matrice di pesi originale W , viene inserita una matrice di trasformazione addestrabile M , tale per cui i pesi attivi diventano $\bar{W} = WM$. All'interno di M , ogni colonna c corrisponde a un canale di output, e la sua importanza viene quantificata calcolandone la norma L_2 ($s = \|c\|_2$). Durante il *fine-tuning*, per indurre la rete a identificare i pesi superflui, la funzione di perdita viene sommata ad un regolarizzatore *Lasso* ($L_{lasso} = \|c\|_2$), mentre l'aggiornamento del gradiente viene modulato da una maschera binaria $m \in \{0, 1\}$. Periodicamente, i canali che registrano la norma minore vengono mascherati impostando $m = 0$, forzando gradualmente e dinamicamente il loro contributo verso lo zero assoluto.

L'applicazione diretta di questo meccanismo ai moduli *Multi-Head Self-Attention* (MHSA) causerebbe, tuttavia, disallineamenti tensoriali. Per ovviare a ciò, l'algoritmo sovrascrive la selezione globale del *compactor* imponendo due vincoli. Il primo, la coerenza a livello di testa (*head-level consistency*), risolve lo sbilanciamento dimensionale tra le teste di attenzione: quando un canale viene selezionato per la mascheratura in una specifica testa h , l'algoritmo individua e vincola al taglio anche il canale con il punteggio residuo più basso all'interno di tutte le altre teste $h' \neq h$. Questo assicura che, al termine dell'operazione, ogni testa mantenga il medesimo numero di canali. Il secondo vincolo, la coerenza a livello di attenzione (*attention-level con-*

sistency), si rende necessario per garantire la compatibilità algebrica durante l'operazione di prodotto scalare interno alla *Self-Attention*. Affinché le matrici *Query* (M_q) e *Key* (M_k) possano essere moltiplicate, devono avere lo stesso numero di canali. Pertanto, se l' i -esimo canale viene contrassegnato per il *pruning* in M_q , la coerenza impone che il medesimo i -esimo canale venga forzatamente soppresso anche in M_k , scavalcando il suo punteggio di importanza originale.

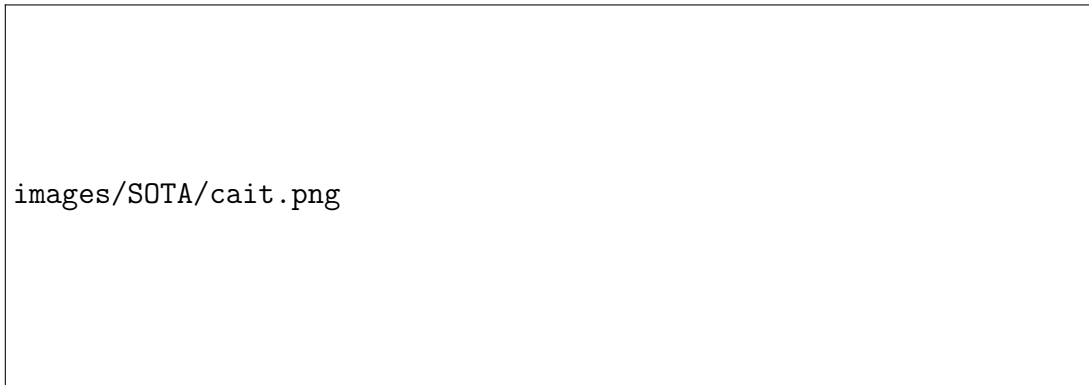


Figura 3.29: Illustrazione del framework complessivo di CAIT, che integra simultaneamente le operazioni di ATME e CDCP. Fonte: [70].

Dal punto di vista operativo, prima di iniziare l'addestramento, l'algoritmo definisce un piano di compressione basato su un budget globale di FLOPs: seleziona gerarchicamente i layer di un modello *Vision Transformer* in cui inserire in modo alternato le operazioni HTM e VTM per la riduzione dei token e, parallelamente, integra le matrici *compactor* (M) in tutti i layer della rete. Infine, CAIT applica simultaneamente CDCP e ATME attraverso un'unica fase di *fine-tuning*.

I risultati sperimentali ottenuti confermano il netto vantaggio di questo approccio congiunto. Sul dataset ImageNet, l'applicazione di CAIT all'architettura DeiT-Small garantisce un'accelerazione dell'inferenza di 1.9 volte, dimostrando un lieve incremento di accuratezza top-1 (raggiungendo l'80.2%), superando i framework di compressione visti in precedenza. Il vero punto di forza di CAIT emerge tuttavia nei task *downstream*: testato sulla segmentazione semantica (dataset ADE20k), il mantenimento dell'integrità spaziale garantito da ATME permette al modello di ottenere un netto miglioramento delle prestazioni (fino a +2.0% mIoU) e una velocità di inferenza superiore rispetto alle metodologie concorrenti.

3.2.2 Quantizzazione

La quantizzazione consiste nel ridurre la precisione (il numero di bit) per rappresentare i pesi e/o le attivazioni di un modello, con l'obiettivo di ridurre la memoria e incrementare la velocità di inferenza, tipicamente sfruttando le capacità di hardware specializzato. Esistono diverse strategie di quantizzazione, tra cui:

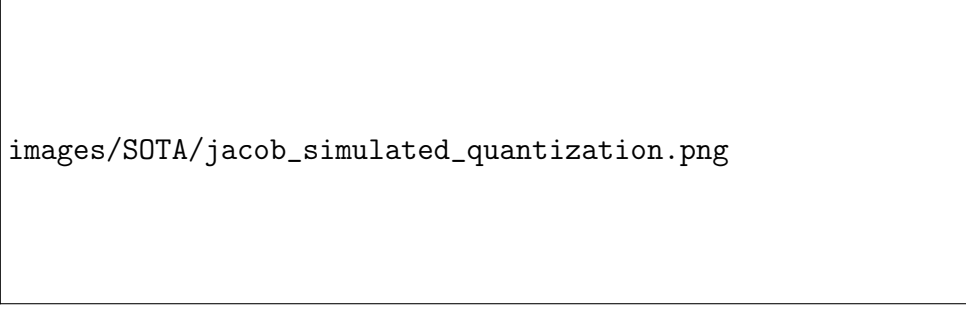
- **Quantizzazione Post-Addestramento (PTQ):** trattasi di un operazione che avviene al termine del processo di addestramento del modello e che non prevede ulteriori fasi di *training*. In questo caso, il modello viene quantizzato senza la possibilità di adattare i pesi alla nuova rappresentazione a bassa precisione, il che si traduce spesso in un maggiore degrado delle prestazioni, ma con il vantaggio della estrema semplicità e rapidità della procedura;
- **Quantizzazione con Addestramento (QAT):** strategia che prevede, dopo la quantizzazione di pesi e attivazioni, una fase di addestramento aggiuntiva volta al recupero delle prestazioni inevitabilmente perdute nel processo. In alternativa, è possibile integrare la quantizzazione durante la fase di addestramento principale, in modo da adattare il modello alla bassa risoluzione dei pesi fin dall'inizio. Questa strategia risulta più complessa e necessita di maggiore tempo, ma consente di mantenere (anche se spesso non completamente) le prestazioni del modello originale;
- **Quantizzazione Statica:** viene utilizzato un set di dati di calibrazione per determinare gli spazi dei valori delle attivazioni attesi durante l'inferenza. Grazie a questo set è possibile quantizzare le attivazioni in anticipo, senza la necessità di calcoli *on-the-fly*. Questa strategia garantisce la migliore velocità in fase di inferenza, ma può portare a un degrado delle prestazioni se il set di calibrazione non rappresenta adeguatamente la distribuzione dei dati reali;
- **Quantizzazione Dinamica:** le attivazioni vengono quantizzate dinamicamente durante l'inferenza, adattando gli spazi dei valori in tempo reale. Questo approccio può migliorare le prestazioni rispetto alla quantizzazione statica, ma introduce un overhead computazionale aggiuntivo durante l'inferenza, che potrebbe non essere accettabile in scenari di *edge computing* con risorse limitate.

A seguire una panoramica dei principali lavori in questo ambito, con un'attenzione particolare alla quantizzazione di modelli *Vision Transformer*. Risulta importante notare come tutti i metodi presentati in questa sezione siano statici, ovvero non prevedano calcoli dinamici durante l'inferenza, al fine di massimizzare la velocità di inferenza, evitando la necessità di operazioni *on-the-fly*.

Le origini della quantizzazione

Uno dei primi lavori a formalizzare il processo di quantizzazione come trasformazione dei parametri in virgola mobile in rappresentazioni intere è stato quello di Jacob et al. [31]. Fino a quel momento, i tentativi di quantizzazione si dividevano prevalentemente in due categorie. La prima comprendeva approcci che operavano solo sui pesi [25, 23], i quali si concentravano sull'abbattimento della memoria di archiviazione ma, mantenendo le attivazioni in virgola mobile, vanificavano buona parte della reale accelerazione computazionale dell'hardware. La seconda categoria esplorava approcci estremi come le reti binarie (1-bit) [56], che pur ottimizzando pesantemente l'inferenza, causavano un drastico degrado dell'accuratezza. Per risolvere questa problematica e garantire una concreta riduzione della latenza, gli autori introducono due contributi fondamentali:

- Uno schema di quantizzazione **Integer-Only**, in cui pesi e attivazioni sono rappresentati rigorosamente come interi a 8 bit (*uint8* o *int8*), mentre le operazioni di accumulo e i bias utilizzano interi a 32 bit (*int32*);
- Un framework di addestramento basato sulla **Simulated Quantization** (oggi noto come *Quantization-Aware Training*, QAT), capace di modellare l'errore di arrotondamento durante il *forward pass* per preservare l'accuratezza *end-to-end* del modello.



images/SOTA/jacob_simulated_quantization.png

Figura 3.30: Confronto tra l'inferenza Integer-Only (sinistra) e l'addestramento con Simulated Quantization (destra). Fonte: [31].

Il cuore matematico dello schema proposto si basa su una rigorosa mappatura affine tra il dominio dei numeri reali e quello degli interi. Ogni valore reale r viene approssimato ad un valore quantizzato q secondo la relazione:

$$r = S(q - Z), \quad (3.1)$$

dove S è il fattore di scala (*Scale*), ovvero un numero reale strettamente positivo, e Z , detto punto zero (*Zero-point*), è un intero dello stesso tipo di q . La genialità di questa formulazione risiede nel parametro Z : esso garantisce che il valore reale 0 sia sempre rappresentabile in modo esatto nello spazio quantizzato. Questa proprietà è cruciale per l'implementazione efficiente di operazioni come il *zero-padding* nelle convoluzioni e le attivazioni ReLU [1].

Applicando questa formulazione alla moltiplicazione tra matrici, gli autori dimostrano che il calcolo può essere eseguito interamente tramite somme e moltiplicazioni tra interi. L'unico fattore non intero risultante dall'equazione è il moltiplicatore $M = \frac{S_1 S_2}{S_3}$. Questo termine agisce come un fattore di riscaldamento necessario per mappare il prodotto delle attivazioni e dei pesi (che possiede intrinsecamente una scala pari a $S_1 S_2$) nel dominio di quantizzazione del livello successivo, definito dalla scala S_3 .

Poiché M è un valore costante determinato offline dai parametri di quantizzazione, esso può essere trattato senza ricorrere ad aritmetica a virgola mobile durante l'inferenza. Gli autori osservano empiricamente che M ricade quasi sempre nell'intervallo $(0, 1)$ e propongono di decomporlo nella forma:

$$M = 2^{-n} M_0, \quad (3.2)$$

dove $M_0 \in [0.5, 1)$ è un moltiplicatore normalizzato e n è un intero non negativo. In fase di

implementazione hardware, M_0 viene approssimato come un intero a virgola fissa (ad esempio un valore a 32 bit), permettendo di sostituire l'operazione teorica con una moltiplicazione tra interi seguita da un'operazione di *bit-shift* verso destra per applicare il fattore 2^{-n} . Questo stratagemma consente di mantenere l'intera pipeline di calcolo nel dominio delle unità aritmetiche intere (ALU), garantendo un'efficienza estrema sui processori mobile.

Per colmare il divario prestazionale causato dalla discretizzazione, il modello non viene quantizzato post-addestramento. Al contrario, il processo di addestramento viene modificato iniettando dei nodi di quantizzazione simulata (*fake quantization*) nel grafo computazionale. Durante il *forward pass*, questi nodi simulano l'effetto di troncamento e saturazione imposto dai limiti degli 8 bit; durante il *backward pass*, i gradienti fluiscono attraverso i pesi in *floating-point*, consentendo all'ottimizzatore di compensare l'errore indotto. Un'attenzione particolare viene inoltre posta nel fondere matematicamente i parametri di *Batch Normalization* all'interno dei pesi convoluzionali prima di applicare la quantizzazione simulata.

Valutato su architetture hardware *mobile-friendly* come MobileNet, questo framework dimostra un miglioramento drastico del trade-off tra latenza e accuratezza: il modello quantizzato garantisce uno *speedup* fino al 50% e una riduzione di 4x dell'occupazione di memoria, registrando un degrado prestazionale minimo, spesso inferiore al 2% rispetto alle controparti operanti in singola precisione.

Post-Training Quantization for Vision Transformer

L'applicazione diretta dei framework di *Post-Training Quantization* (PTQ) tradizionali ai *Vision Transformer* (ViT) ha rivelato limiti strutturali significativi. I metodi convenzionali, infatti, sono stati progettati primariamente per le reti convoluzionali (CNN) e non tengono conto delle peculiarità architetture dei Transformer, primo fra tutti il meccanismo di *Self-Attention*. Per colmare questa mancanza nella ricerca, Liu et al. propongono il primo approccio PTQ esplicitamente sviluppato e ottimizzato per i ViT [43]. Per preservare un'accuratezza paragonabile alla *baseline* in virgola mobile abbattendo i costi computazionali, gli autori strutturano il framework su tre contributi principali:

- **Ranking-Aware Quantization:** Un meccanismo progettato per mantenere inalterata la gerarchia relativa dei valori all'interno delle mappe di attenzione;

- **Similarity-Aware Quantization:** Un'ottimizzazione mirata degli intervalli di quantizzazione per massimizzare la somiglianza matematica tra le *feature map* originali e quelle quantizzate;
- **Mixed-Precision Quantization:** Un'allocazione dinamica dei bit guidata dalla *Nuclear Norm* delle *feature* per valutare la sensibilità intrinseca di ciascun layer alla compressione.

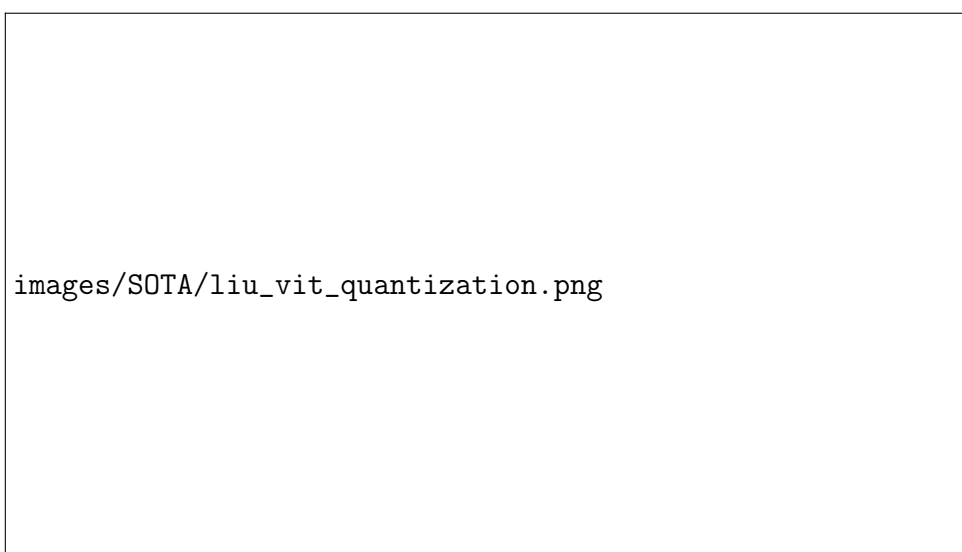


Figura 3.31: Diagramma del framework di quantizzazione per Vision Transformer. Evidenzia l'uso della *Similarity-Aware Quantization* per i moduli MLP e della *Ranking-Aware* per la Self-Attention. Fonte: [43].

Il problema fondamentale riscontrato nello studio risiede nel modulo di *Multi-Head Self-Attention* (MSA). Analizzando empiricamente il modello, gli autori osservano che l'applicazione di una discretizzazione uniforme altera gravemente il *ranking* relativo dei punteggi calcolati nella mappa di attenzione (poiché, usando meno bit, la precisione si riduce). Poiché l'attenzione modella la rilevanza globale tra i vari *token* di un'immagine, intaccare questo *ranking* genera un drastico calo prestazionale. La soluzione individuata risiede in una *Ranking Loss*: una funzione di costo che penalizza severamente le inversioni di ordine tra i pesi di attenzione originali e quelli quantizzati. Minimizzare questa funzione assicura che la gerarchia relazionale appresa dalla rete rimanga intatta.

Parallelamente, per mitigare l'errore nelle operazioni lineari dei moduli MLP, viene formulato un problema di ottimizzazione iterativa per ricercare le soglie di *clipping* ideali (ovvero

i confini degli intervalli di quantizzazione). L'obiettivo matematico è la massimizzazione del coefficiente di correlazione di Pearson tra l'output originale e la sua controparte a bassa precisione. Questa scelta metodologica è strategica: essendo invariante a traslazioni e variazioni di scala uniformi, il coefficiente di Pearson quantifica esclusivamente la dipendenza lineare tra i tensori. Ciò garantisce che le *feature map* vengano preservate fedelmente per i layer successivi.

Infine, superando i limiti degli approcci a larghezza di bit uniforme, gli autori constatano che ogni blocco del Transformer esibisce una diversa tolleranza alla quantizzazione. Attraverso la decomposizione ai valori singolari (SVD), il framework calcola la *Nuclear Norm* (ossia la somma dei valori singolari) delle mappe di attenzione e delle *feature* di output per quantificarne la diversità e la ricchezza informativa. Questa misura guida un processo di ottimizzazione che assegna precisioni miste in modo asimmetrico: i layer più critici mantengono risoluzioni elevate, mentre quelli più robusti vengono compressi maggiormente.

I risultati sperimentali confermano la solidità del framework: valutato su architetture all'avanguardia come DeiT-B sul dataset ImageNet, il modello riesce a mantenere un'accuratezza top-1 dell'81.29% operando in un regime di quantizzazione approssimabile agli 8 bit. Tale risultato annulla quasi totalmente il divario con la *baseline* ad alta precisione, garantendo allo stesso tempo un drastico abbattimento del peso in memoria e della latenza di inferenza.

FQ-ViT: Fully Quantized Vision Transformer

Nonostante i progressi ottenuti nell'adattamento della *Post-Training Quantization* ai modelli basati sull'attenzione, i framework precedenti presentavano un limite rilevante: per evitare drastici crolli di accuratezza, mantenevano i moduli di *LayerNorm* e *Softmax* in virgola mobile. Questa scelta impediva di ottenere reti completamente quantizzate (*Fully Quantized*), costringendo l'hardware a continui e onerosi passaggi di conversione dei dati tra interi e *floating-point*, limitando di fatto il reale guadagno di velocità in fase di inferenza. Per risolvere questa problematica, Lin et al. hanno introdotto **FQ-ViT** [39], il primo approccio capace di operare una quantizzazione totale dei *Vision Transformer* riducendo al minimo il degrado prestazionale (approssimabile a circa l'1%). Gli autori individuano due cause fondamentali alla base del fallimento delle tecniche tradizionali su questi specifici moduli.

In primo luogo, gli input dei layer di normalizzazione (*LayerNorm*) presentano una variabilità estrema tra i diversi canali, con fluttuazioni che possono superare di 40 volte il valore mediano. Applicare intervalli di quantizzazione uniformi a livello di intero layer in un contesto

di così forte sbilanciamento genera errori importanti. Per risolvere questa criticità, *FQ-ViT* introduce il **Power-of-Two Factor (PTF)**: per gestire le enormi differenze numeriche tra i canali della *LayerNorm*, la soluzione teorica ideale sarebbe assegnare a ciascun canale un fattore di scala specifico e indipendente, tuttavia, utilizzare parametri in virgola mobile per ogni singolo canale annullerebbe i vantaggi della quantizzazione, appesantendo e rallentando drasticamente i calcoli hardware. Il PTF aggira l'ostacolo assegnando a ogni canale un fattore di compensazione che è rigorosamente espresso come una potenza del due (ad esempio $2^1, 2^2, 2^3$). Ciò permette di utilizzare veloci operazioni hardware come il *BitShift*. Grazie a questo stratagemma, il framework riesce ad allineare l'ampiezza dei vari canali sbilanciati in modo precisissimo, operando esclusivamente nel dominio dei numeri interi e garantendo la massima velocità di inferenza.

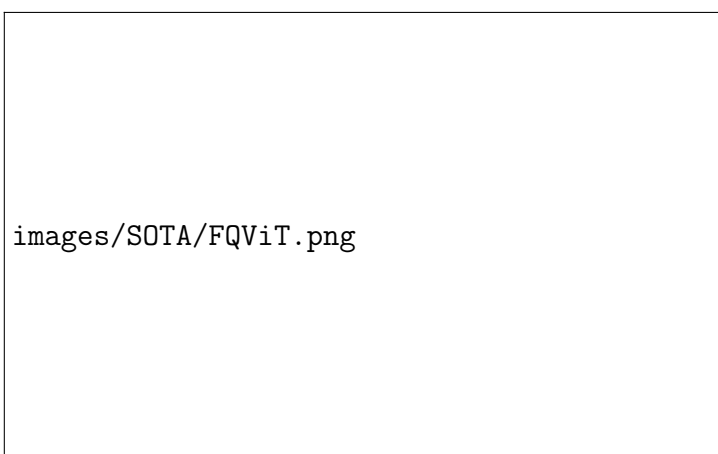


Figura 3.32: Evidenzia la differenza tra una quantizzazione incompleta e quella di *FQ-ViT*, che lavora anche sulla *Softmax* ed evita la moltiplicazione finale. Fonte: [39].

In secondo luogo, l'analisi delle mappe di attenzione rivela una distribuzione dei valori fortemente asimmetrica: la stragrande maggioranza dei punteggi ricade in un intorno piccolissimo (tra 0 e 0.01), mentre solo rare eccezioni si avvicinano ad 1. La quantizzazione lineare standard si dimostra del tutto inadeguata per tale distribuzione, poiché alloca un ridotto numero di intervalli di precisione per la fascia di valori più densa, aggregando tutti i valori indiscriminatamente. La soluzione proposta è il **Log-Int-Softmax (LIS)**, un meccanismo che adotta una quantizzazione in base logaritmica ($\log_2$). Questa trasformazione matematica garantisce una risoluzione molto più fine e granulare per i valori prossimi allo zero, preservando le sfumature della mappa di attenzione originale. Inoltre, grazie al modulo *Log-Int-Softmax*, *FQ-ViT* riesce a comprimere in modo aggressivo le mappe di attenzione portandole a soli 4 bit di precisione. Sfruttando le

proprietà dell'aritmetica logaritmica, il framework sostituisce le dispendiose moltiplicazioni matriciali a valle dell'attenzione con semplici operazioni di *BitShift*, mantenendo l'intera pipeline di calcolo della *Self-Attention* strettamente nel dominio dei numeri interi.

I test confermano l'efficacia di queste ottimizzazioni: applicato ad architetture di grandi dimensioni come ViT-L sul dataset ImageNet, *FQ-ViT* raggiunge un'accuratezza top-1 dell'84.89% ■ dimostrando che è possibile quantizzare (PTQ) completamente un *Vision Transformer* senza compromettere la capacità predittiva della rete.

Q-ViT: Accurate and Fully Quantized Low-bit Vision Transformer

Sebbene i metodi di *Post-Training Quantization* (PTQ) abbiano dimostrato risultati notevoli, non risultano efficaci quando si tenta di spingere la quantizzazione ad una precisione estremamente ridotta, come 2, 3 o 4 bit. In queste condizioni, i ViT subiscono un drastico calo delle prestazioni. Per superare questa barriera, Li et al. introducono **Q-ViT** [38], un framework basato sul *Quantization-Aware Training* (QAT).

Attraverso un'approfondita analisi empirica, gli autori identificano due colli di bottiglia fondamentali che impediscono ai ViT di apprendere efficacemente a bassa precisione. Nel *forward pass*, la quantizzazione diretta provoca una grave distorsione dell'informazione nella mappa di attenzione: le reti quantizzate perdono la capacità di variare la distanza di attenzione al variare della profondità (fenomeno in cui i layer superficiali catturano feature locali e quelli profondi feature globali), appiattendolo irrimediabilmente la gerarchia delle feature. Nella fase di *backward pass*, invece, i classici metodi di distillazione basati sull'output di classificazione si rivelano inefficaci e troppo grossolani per compensare l'enorme divario creatosi nei punteggi di attenzione quantizzati.

Per risolvere queste criticità, Q-ViT interviene con due soluzioni architetturali mirate:

- **Information Rectification Module (IRM)**: Un modulo progettato per il *forward pass* e fondato sulla teoria dell'informazione. Partendo dall'osservazione che i valori di *query* e *key* nei ViT seguono tipicamente una distribuzione gaussiana $\mathcal{N}(\mu, \sigma^2)$, gli autori dimostrano che la discretizzazione a bassa precisione ne altera irreversibilmente il profilo, facendone crollare l'entropia informativa. Poiché per una variabile gaussiana la massimizzazione dell'entropia di output equivale alla minimizzazione dell'errore medio di quantizzazione, l'IRM applica una standardizzazione modificata della funzione di quantizzazione, definita come $\tilde{q} = \frac{q - \mu(q) + \beta_q}{\gamma_q \sqrt{\sigma^2(q) + \epsilon_q}}$, dove γ e β sono parametri addestrabili. Questo

riallineamento parametrico compensa l'alterazione e ripristina la capacità del meccanismo di attenzione di catturare le dipendenze globali;

- **Distribution Guided Distillation (DGD):** Uno schema di distillazione avanzato per il *backward pass*. I metodi tradizionali, che si limitano a supervisionare l'output o le mappe di attenzione finali, si rivelano insufficienti per correggere il collasso della distanza di attenzione. La DGD interviene estraendo le matrici di similarità *patch-based* calcolando il prodotto scalare tra i vettori delle *query* e tra i vettori delle *key* e le normalizza applicando la norma L_2 . Questa normalizzazione è cruciale per eliminare le differenze di scala e le instabilità numeriche tra il modello *teacher* in virgola mobile e lo *student* quantizzato. Minimizzando la distanza L_2 tra queste matrici normalizzate, la rete riceve una supervisione di maggiore qualità.



Figura 3.33: I due principali contributi di *Q-ViT*: Information Rectification Module (IRM) e Distribution Guided Distillation (DGD). Fonte: [38].

I risultati empirici di questa impostazione stabiliscono un nuovo standard per la compressione estrema. Applicato all'architettura DeiT-S, *Q-ViT* a 4-bit raggiunge un'accuratezza top-1 dell'80.9% sul dataset ImageNet, riuscendo paradossalmente a superare la *baseline* in virgola mobile (79.9%) e garantendo al contempo un'accelerazione teorica di 6.14 volte. Ancor più notevole è il risultato a 2-bit, dove il framework riesce a comprimere i FLOPs del modello di ben 21.5 volte, mantenendo un'accuratezza del 72.0%. Con questo approccio, viene dimostrato che la QAT è una via estremamente promettente per il *deploy* dei Transformer su dispositivi a risorse limitate.

3.2.3 Knowledge Distillation

La **Knowledge Distillation** (KD) è una tecnica di compressione indiretta di modelli che consiste in un trasferimento di conoscenza da un modello *teacher* più grande e complesso a un modello *student* più piccolo e leggero. L'obiettivo è quello di addestrare il modello *student* ad emulare il comportamento del modello *teacher*, mantenendo prestazioni comparabili nonostante la riduzione delle dimensioni. Esistono tre principali strategie di distillazione:

- **Distillazione basata sui Logit:** tecniche il cui obiettivo è allineare le predizioni finali dei modelli. Il modello *student* viene addestrato a imitare le rappresentazioni finali del modello *teacher*, ad esempio minimizzando la distanza tra le distribuzioni di probabilità prodotte dai due modelli;
- **Distillazione basata sulle relazioni:** metodi avanzati, meno diffusi in letteratura, che puntano a trasferire la conoscenza strutturale e semantica della rete. Anziché replicare valori puntuali, il modello *student* impara a emulare le mutue interazioni, come le distanze vettoriali, le correlazioni o le matrici di similarità tra le feature apprese;
- **Distillazione basata sulle Feature:** approcci in cui il modello *student* viene istruito a replicare fedelmente i valori numerici delle rappresentazioni intermedie (le *feature map*) del modello *teacher*.

Di seguito una panoramica dei principali lavori in questo ambito, con un'attenzione particolare alla distillazione di modelli *Vision Transformer*.

Le origini della Knowledge Distillation

Il primo importante lavoro a formalizzare e testare il processo di trasferimento della conoscenza da modelli complessi a modelli compatti è stato quello di Hinton et al. [29]. Fino a quel punto, l'approccio diffuso per massimizzare le prestazioni di un sistema di *machine learning* consisteva nell'addestrare un *ensemble* (insieme) di reti neurali, una tecnica estremamente efficace in fase di training ma proibitiva in fase di *deploy* su larga scala a causa degli elevati costi computazionali e della latenza. Sebbene fosse già stato dimostrato da Buciluă et al. [6] che la conoscenza di un *ensemble* potesse essere compressa in un singolo modello, risultava necessario definire un framework capace di astrarre la conoscenza acquisita dal modello, svincolandola

dai soli parametri numerici per intenderla, invece, come una funzione di mappatura appresa tra input e output.

Per risolvere questa limitazione, gli autori introducono il concetto di **Distillation** (Distillazione), apportando due contributi fondamentali:

- L'utilizzo delle probabilità di output del modello complesso come **Soft Targets** per l'addestramento del modello compatto. Questi valori forniscono molta più informazione per ogni caso di addestramento rispetto ai tradizionali *hard targets* (etichette binarie), catturando la ricca struttura di similarità tra le classi appresa dal modello complesso;
- L'introduzione del parametro della **Temperatura** nella funzione Softmax. Questo espediente matematico permette di addolcire la distribuzione di probabilità, facendo emergere i valori relativi alle classi errate che, sebbene minimi, veicolano informazioni cruciali su come il modello tende a generalizzare.

Lo schema proposto interviene direttamente sull'ultimo layer della rete neurale. Ogni probabilità di classe q_i viene calcolata a partire dai logit z_i introducendo il parametro di temperatura T :

$$q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}. \quad (3.3)$$

In condizioni normali T è impostato a 1. Tuttavia, durante la distillazione, si utilizza un valore di T elevato per generare una distribuzione di target più informativa. Il modello *student* viene quindi addestrato a minimizzare una funzione obiettivo composta dalla media pesata di due componenti:

1. La *cross-entropy* calcolata rispetto ai *soft targets* generati dal *teacher*, utilizzando la medesima temperatura T per entrambi i modelli. Da un punto di vista dell'ottimizzazione, poiché la distribuzione del *teacher* è fissa, minimizzare questa componente equivale matematicamente a minimizzare la divergenza di Kullback-Leibler (KL Divergence) tra le distribuzioni in uscita dei due modelli;
2. La *cross-entropy* standard calcolata rispetto alle etichette reali (*hard targets*), computata sui logit dello *student* a temperatura unitaria.

Poiché i gradienti prodotti dai *soft targets* scalano proporzionalmente a $1/T^2$, gli autori osservano che è necessario moltiplicarli per T^2 durante l'ottimizzazione. Questo assicura che il

contributo relativo dei *soft* e *hard targets* rimanga bilanciato per qualsiasi valore selezionato di temperatura. Hinton et al. dimostrano inoltre che, per un valore di T tendente ad infinito, la distillazione equivale matematicamente alla minimizzazione della distanza quadratica tra i logit di *teacher* e *student*.

Valutato su molteplici domini, questo approccio ha dimostrato capacità di trasferimento della conoscenza notevoli. Negli esperimenti su dataset MNIST, il modello distillato è stato in grado di riconoscere correttamente cifre (come il "3") pur non avendone mai osservato alcun esempio durante l'addestramento, affidandosi esclusivamente alla conoscenza trasferita dal *teacher*. In ambito di riconoscimento vocale, un singolo modello distillato è riuscito a eguagliare quasi perfettamente le prestazioni di un *ensemble* di 10 reti neurali profonde, riducendo drasticamente l'onere computazionale in fase di inferenza senza compromettere l'accuratezza.

DeiT

L'architettura *Vision Transformer* (ViT) originale aveva dimostrato prestazioni ottime in ambito visivo, ma con un limite infrastrutturale critico: per generalizzare correttamente senza incorrere in *overfitting*, richiedeva l'addestramento su dataset dalle dimensioni proibitive (come JFT-300M, contenente 300 milioni di immagini). Con l'introduzione di **DeiT** (*Data-efficient Image Transformers* [66]), Touvron et al. superano questa limitazione, dimostrando che è possibile addestrare un ViT competitivo utilizzando esclusivamente il dataset ImageNet-1K.

Il fulcro del loro approccio non si basa su variazioni particolari dell'architettura, bensì sull'utilizzo di una tecnica di *Knowledge Distillation*. In primo luogo, gli autori osservano un fenomeno interessante: utilizzare una rete neurale convoluzionale (CNN) come modello *teacher* produce risultati nettamente superiori rispetto all'uso di un Transformer di pari dimensioni. Questo avviene perché la distillazione permette al modello *student* (il ViT) di ereditare indirettamente i *bias induttivi* della CNN, compensando la propria mancanza intrinseca di capacità spaziali e di invarianza alla traslazione.

Un'aspetto fondamentale del loro lavoro consiste nella rottura rispetto al paradigma stabilito da Hinton [29] riguardo la funzione obiettivo. Invece di utilizzare la *soft distillation* basata sulle distribuzioni addolcite, DeiT impiega la **Hard-label distillation**. In questa formulazione, la decisione finale rigida della rete *teacher* ($y_t = \operatorname{argmax} Z_t(c)$) viene considerata e utilizzata alla stregua di una vera e propria etichetta (*true label*). La funzione di costo globale diventa quindi la media di due *Cross-Entropy* distinte, una calcolata sull'etichetta reale y e l'altra sull'etichetta

del *teacher* y_t :

$$\mathcal{L}_{global}^{hardDistill} = \frac{1}{2} \mathcal{L}_{CE}(\psi(Z_s), y) + \frac{1}{2} \mathcal{L}_{CE}(\psi(Z_s), y_t). \quad (3.4)$$

L'innovazione più incisiva di DeiT risiede però nell'integrazione di questa funzione di costo all'interno della rete attraverso il **Distillation Token**. Ispirandosi al token di classificazione ('[CLS]') standard dei ViT, gli autori aggiungono alla sequenza di token, ottenuti dalle *patch* in ingresso, un nuovo vettore addestrabile. Questo token interagisce liberamente con tutti gli altri attraverso i molteplici layer di *Self-Attention*. Tuttavia, mentre la supervisione finale del token '[CLS]' mira a minimizzare l'errore rispetto all'etichetta reale (y), l'ottimizzazione del *distillation token* è esclusivamente delegata a riprodurre l'etichetta predetta dal *teacher* (y_t).

Analizzando le dinamiche interne della rete, si nota che i due token non collassano mai nello stesso vettore. Al primo layer, la loro similarità coseno è quasi nulla (0.06), per poi crescere progressivamente man mano che la rete aggrega la semantica, fino a raggiungere un valore di 0.93 all'ultimo layer. Questo comportamento dimostra che, pur puntando a predizioni affini, i due token estraggono ed elaborano informazioni visive in modo complementare.

Grazie alla sinergia di questi contributi, il modello DeiT-B non solo supera nettamente le prestazioni del ViT-B originale a parità di dati utilizzati in addestramento, ma riesce persino a battere la variante ViT pre-addestrata sul dataset JFT-300M. Questo risultato permette all'architettura Transformer di chiudere il divario con le reti convoluzionali, eguagliando architetture CNN ottimizzate e offrendo un elevato rapporto tra accuratezza e velocità di inferenza.

DearKD

Sebbene l'introduzione di metodi come DeiT abbia dimostrato la fattibilità dell'addestramento dei *Vision Transformer* senza l'ausilio di dataset di grandi dimensioni, le architetture puramente basate sull'attenzione continuano a necessitare di moli considerevoli di dati. Questo limite deriva dalla mancanza di *bias induttivo*, ovvero l'assenza di vincoli strutturali innati, come la località e la condivisione dei pesi, che rendono le reti neurali convoluzionali (CNN) estremamente efficienti nell'estrarre pattern visivi sin dalle prime fasi dell'addestramento.

I metodi di distillazione analizzati finora si limitano a trasferire la conoscenza esclusivamente attraverso i *logit* del modello *teacher*. Chen et al. introducono **DearKD** [10], evidenziando come questa pratica presenti due criticità per i Transformer: in primo luogo, rende più difficile per i layer iniziali del modello *student* apprendere il bias induttivo locale delle CNN; in secondo

luogo, forzare il ViT a imitare la CNN per l'intera durata dell'addestramento ostacola lo sviluppo delle proprie rappresentazioni intrinseche e delle dipendenze globali.



Figura 3.34: Strategia di distillazione a due stadi di *DearKD*. Fonte: [10].

Per superare questi problemi, Chen et al. propongono un framework di apprendimento a due stadi, come è possibile notare in Figura 3.34, denominato *Data-efficient EARly Knowledge Distillation* (DearKD):

- **Stadio I (Distillazione Convoluzionale Iniziale):** In questa prima fase, il modello apprende non solo dai logit, ma soprattutto dai layer intermedi della CNN *teacher*. Poiché le *feature map* convoluzionali e i token del Transformer possiedono dimensioni incompatibili, gli autori introducono un modulo di allineamento (*aligner module*). Questo modulo (Figura 3.35) rimodella e interpola spazialmente i token per poterne calcolare l'errore quadratico medio rispetto alle *feature map* della CNN. L'obiettivo dell'apprendimento di una rappresentazione spaziale è ulteriormente rinforzato dall'introduzione di un meccanismo di *Multi-Head Convolutional-Attention* (MHCA). Basandosi sulla dimostrazione teorica [13] secondo cui un layer di attenzione dotato di codifica posizionale relativa può matematicamente esprimere qualsiasi operazione convoluzionale, gli autori riformulano la matrice di attenzione A come:

$$A = \text{Softmax}(QK + v^{(h)}r_{ij}), \quad (3.5)$$

dove l'argomento della funzione Softmax è diviso in due componenti complementari. La prima componente, il prodotto scalare QK (*query-key*), elabora il contenuto del token per catturare le tradizionali dipendenze semantiche globali. La seconda componente, $v^{(h)}r_{ij}$, costituisce l'informazione posizionale relativa (RPE, *Relative Positional Embedding*). Nello specifico, r_{ij} funge da vettore geometrico fisso che descrive le distanze spaziali tra i token, mentre $v^{(h)}$ incapsula i parametri addestrabili della testa dell'attenzione. Il loro prodotto agisce come una finestra Gaussiana, permettendo a ogni *head* di emulare un filtro convoluzionale con campo ricettivo dinamico, inducendo il modello *student* a focalizzarsi sulle relazioni locali.

- **Stadio II (Apprendimento Intrinseco):** Raggiunta una solida base di comprensione spaziale e locale, la distillazione viene completamente disattivata. Il Transformer prosegue l'addestramento guidato unicamente dalla *Cross-Entropy* rispetto alle etichette reali (*ground truth*). Questo accorgimento permette alla rete di concludere l'addestramento senza la rigida guida della CNN, consentendole di sfruttare le capacità globali della *Self-Attention*.

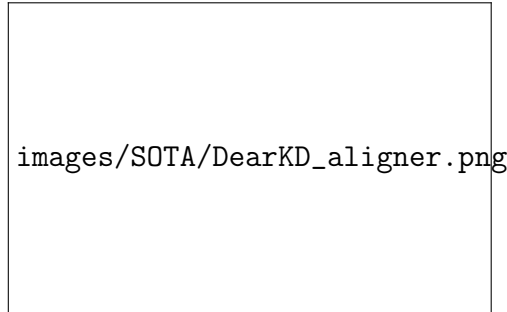


Figura 3.35: Operazione di allineamento dei token di *DearKD*, per riportarli alla dimensione delle *feature map*. Fonte: [10].

I test dimostrano che un modello DearKD, addestrato utilizzando solamente il 50% delle immagini del dataset ImageNet, riesce a superare le prestazioni della *baseline* DeiT addestrata sull'intero dataset (100%). Il framework è stato inoltre testato dagli autori con dati esclusivamente generati sinteticamente ed il modello ha registrato un calo prestazionale minimo (solo l'1.0% in meno rispetto alla controparte addestrata su dati reali), confermando l'efficacia delle soluzioni proposte.

TinyViT

Se DeiT e DearKD hanno risolto i problemi di inefficienza dei dati e di bias induttivo, l'implementazione dei *Vision Transformer* su dispositivi *Edge* pone una sfida ulteriore. La procedura standard per ottenere reti compatte ad alte prestazioni consiste nell'applicare la *Knowledge Distillation* durante la fase di pre-addestramento su dataset di grandi dimensioni (come ImageNet-21k), trasferendo la conoscenza da modelli *teacher* complessi. Tuttavia, l'esecuzione simultanea di un modello *teacher* e di uno *student* satura facilmente la memoria GPU e rallenta drasticamente il processo di addestramento.

Per superare questa limitazione, Wu et al. propongono **TinyViT** [73], un framework basato su una tecnica innovativa denominata *Fast Pretraining Distillation*. L'intuizione fondamentale è svincolare completamente il calcolo delle predizioni del *teacher* dal ciclo di addestramento dello *student*. Gli autori eseguono un *forward pass* del *teacher* in una fase preliminare (*offline*), registrando le predizioni su un dispositivo di archiviazione.

Per rendere questa procedura scalabile su milioni di immagini, il framework implementa due accorgimenti tecnici cruciali:

- **Sparse Soft Labels:** Invece di salvare l'intera distribuzione di probabilità (che su ImageNet-21k comprenderebbe oltre 21.000 valori per immagine), vengono archiviati esclusivamente i K logit con i valori più alti (*top-K*) e i rispettivi indici. Questo approccio abbatta i costi di archiviazione, senza compromettere particolarmente la qualità del trasferimento di conoscenza.
- **Data Augmentation Encoding:** Poiché le moderne tecniche di *Data Augmentation* introducono variazioni stocastiche per ogni immagine a ogni iterazione, memorizzare i parametri di ogni singola trasformazione risulterebbe impraticabile. Il sistema risolve il problema salvando un singolo parametro generatore (un *seed*), permettendo allo *student* di riprodurre esattamente la stessa sequenza di trasformazioni visive osservata dal *teacher*.

Durante la vera e propria fase di *training*, il modello *student* compatto si limita a caricare i target sparsi e il seme per il generatore random per l'*augmentation* dal disco, annullando il carico computazionale del *teacher*. Grazie a questa strategia è possibile adottare tecniche di distillazione anche in casi di limitate risorse computazionali, accelerando notevolmente il processo di addestramento.

Capitolo 4

Formulazione matematica del problema

Il problema in esame può essere formulato come una ricerca in uno spazio degli stati. A partire da una architettura iniziale preaddestrata, detta **Supernet**, si vuole identificare, tra tutte le possibili **Subnet**, quella che massimizza una determinata funzione obiettivo. In questo caso, al fine di ottenere una compressione di modelli *Transformer*, la ricerca si configura come un problema di **ottimizzazione multi-obiettivo**, tramite il quale vengono ottimizzate sia la performance del modello (attraverso l'accuratezza) sia la dimensione (attraverso il numero di parametri).

Una delle sfide principali che si presentano è proprio la dimensione dello spazio degli stati, che risulta elevatissima, poiché il numero di possibili configurazioni cresce in maniera combinatoria rispetto al numero di componenti del modello. Nei Vision Transformer, ogni blocco Transformer presenta molteplici gradi di libertà: è possibile variare il numero di teste di attenzione nell'unità di *Multi-Head Attention* (MHSA), la dimensione dei vettori *Query-Key* e *Value*, ma anche la dimensione del livello nascosto nel modulo *Multi-Layer Perceptron* (MLP). Anche considerando un numero limitato di opzioni per ogni strato, il numero totale di subnet candidate diventa rapidamente intrattabile per una ricerca esaustiva.

Proprio per rendere trattabile l'esplorazione dello spazio degli stati è stata scelta una formulazione conveniente del problema, attraverso un algoritmo di ricerca **Branch and Bound** [40] e l'utilizzo di azioni di *pruning* predefinite. L'algoritmo *Branch and Bound* affronta tale complessità decomponendo ricorsivamente lo spazio di ricerca in sottoproblemi più piccoli, organizzati in una struttura ad albero (fase di **Branch**). Per ogni nodo dell'albero, viene calcolato un limite

(Bound) del valore della funzione obiettivo: se tale limite indica che un intero ramo non può contenere una soluzione migliore di quella già individuata, il ramo viene rimosso dall'albero di ricerca. Questo meccanismo permette di navigare in modo efficiente il vasto spazio di ricerca, garantendo l'identificazione della *Subnet* ottimale attraverso l'esplorazione di una frazione ridotta dello spazio degli stati complessivo. Nella prossima sezione sarà esposta nel dettaglio la formulazione matematica proposta.

4.1 Formulazione della ricerca

Dato un modello ViT M con parametri θ , definiamo un processo iterativo di pruning. Per ogni iterazione viene eseguita una ricerca di tipo **Depth First a profondità limitata**. Durante la ricerca verranno inizialmente selezionate delle azioni da eseguire sul modello, e successivamente, attraverso un secondo problema combinatorio vengono selezionate le specifiche dimensioni da eliminare. L'**obiettivo** è massimizzare la seguente funzione rispetto alla maschera binaria m , ai parametri θ e al dataset D :

$$\max_m \text{Obj}(m, \theta, D) = \log_2(A(m, \theta, D)) - \lambda \cdot \log_2 \left(\frac{P(m, \theta)}{P_{tot}(\theta)} \right), \quad (4.1)$$

dove:

- $m \in \{0, 1\}^N$ è la maschera di pruning globale, con N numero totale di parametri del modello.
- La maschera è strutturata per blocchi: $m = \{m_{\text{emb}}, m^{(1)}, \dots, m^{(i)}, \dots, m^{(L)}\}$, con L numero di layer, dove $m^{(i)}$ rappresenta la maschera per il layer i -esimo, mentre m_{emb} rappresenta la maschera (globale) per la dimensione dell'embedding.
- Per ogni blocco i : $m^{(i)} = \{m_{\text{head}}, m_{\text{q_k}}, m_{\text{v_proj}}, m_{\text{mlp}}\}$, con m_{head} la maschera per le teste di attenzione, $m_{\text{q_k}}$ quella per i parametri della proiezione query-key, $m_{\text{v_proj}}$ per la proiezione del value, e m_{mlp} per la componente *Multi-Layer Perceptron* del blocco.
- $A(m, \theta, D)$ indica l'accuratezza del modello sul dataset D con i soli parametri attivi previsti dalla maschera m .

- $P(m, \theta)$ rappresenta il numero di parametri attivi (non azzerati da m).
- $P_{tot}(\theta)$ rappresenta il numero totale di parametri del modello originale.

Notare che tramite questa formulazione è possibile ricondurre il problema in esame ad un **Problema di Programmazione Non Lineare Binario** (PNLPB) [5]. In questo contesto, le variabili decisionali sono rappresentate dai componenti della maschera $m \in \{0, 1\}^N$, mentre la non linearità è introdotta sia dalla natura della funzione di accuratezza A (legata ai pesi della rete neurale) sia dall'operatore logaritmico utilizzato nella funzione obiettivo. Nonostante esistano in letteratura delle tecniche di *Smoothing* delle variabili binarie per risolvere questo tipo di problemi combinatori, come quelle utilizzate da Murray et al. [50], i tempi di risoluzione, con un numero ridotto di variabili binarie rispetto al caso in esame, sono molto elevati. Proprio per tale motivo si è scelto di utilizzare un differente approccio algoritmico basato sul *Branch and Bound*. Di seguito viene dettagliata l'applicazione di quest'ultimo per il problema di ricerca in esame.

4.1.1 Branching

L'esplorazione dello spazio di ricerca avviene tramite la costruzione di un apposito albero, seguendo un algoritmo di esplorazione *Depth First* a profondità limitata; di conseguenza, è possibile definire la strategia di esplorazione come **Depth Limited Depth First Branch and Bound** (DL-DFBnB).

La radice di questo albero è rappresentata dalla *Supernet*, ovvero il modello *Vision Transformer* di dimensione originale, specializzato su uno specifico dominio tramite *fine-tuning*. A partire dalla radice, viene eseguita la fase di **Branch** applicando le azioni di seguito elencate:

- **Pruning Multi-Layer Perceptron**
- **Pruning Query-Key**
- **Pruning Value-Projection**
- **Pruning Head**
- **Pruning Embedding**

Si noti che tutte le azioni, ad eccezione del *Pruning* dell'*Embedding*, sono locali a uno specifico blocco *Transformer*. La formalizzazione matematica delle singole azioni è rimandata alla sezione successiva.

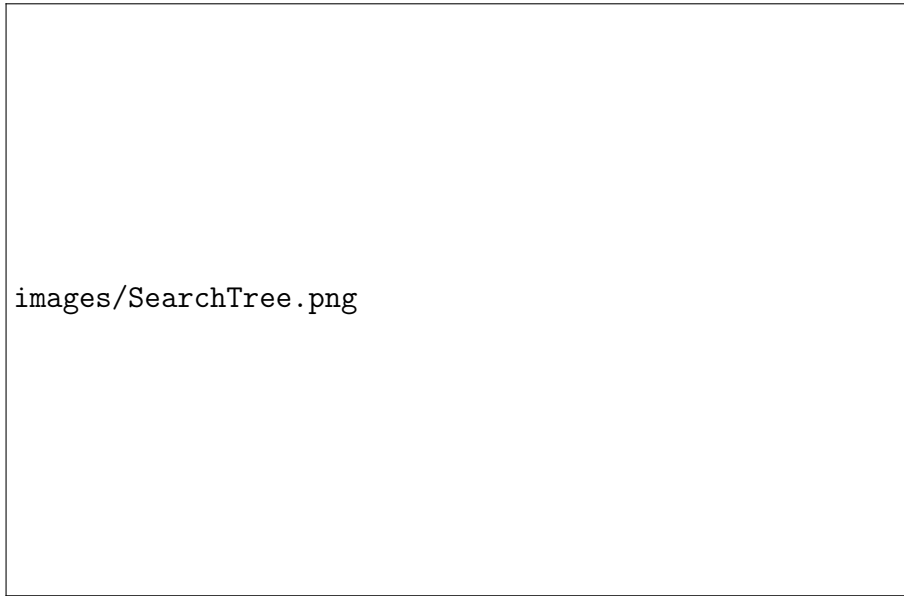


Figura 4.1: Rappresentazione schematica dell'albero di ricerca generato dalle azioni di *Branching*.

Attraverso il *branching* viene costruito un **albero 5-ario** di ricerca, dove ogni nodo rappresenta una specifica *Subnet* ottenuta a partire dal nodo genitore. Ad ogni nodo viene associato un valore della funzione obiettivo, calcolato valutando l'accuratezza su un ridotto set di immagini (*Search Set*) (utilizzando esclusivamente i parametri attivi) e il relativo numero di parametri residui. Questo valore verrà successivamente utilizzato per verificare i vincoli di *Bound* e, qualora risultasse più elevato del miglior valore individuato dalla ricerca, si aggiornerebbe quest'ultimo e si salverebbe la maschera di *Pruning* corrispondente.

Al fine di massimizzare l'efficacia del taglio dei parametri, la strategia di esplorazione *Depth First* è stata scelta per favorire il raggiungimento dei nodi foglia dell'albero di ricerca. Questo permette all'algoritmo di individuare rapidamente configurazioni caratterizzate da un elevato numero di azioni di *pruning* e, di conseguenza, da una significativa riduzione dei parametri. L'introduzione di un limite di profondità nell'albero di ricerca permette di mitigare l'impatto cumulativo del *pruning* sulle prestazioni del modello. Un'esplorazione eccessivamente profonda comporterebbe infatti una rimozione massiva di parametri in un'unica iterazione, rischiando di

degradare l'accuratezza in modo irreversibile. In tali scenari, il danno strutturale all'architettura potrebbe risultare troppo elevato, rendendo difficoltoso il recupero delle performance originali.

4.1.2 Bound

La dimensione dell'albero di ricerca risulta significativa; in particolare, detta D la massima profondità dell'albero e considerato che si tratta di un albero 5-ario, il numero totale di nodi è pari a:

$$N = \sum_{i=0}^D 5^i \quad (4.2)$$

Ad esempio, se $D = 6$, il numero totale di nodi è pari a 19.531. Considerando che per ogni nodo è necessario calcolare i gradienti per tutti i parametri attivi del modello al fine di stimarne l'importanza, il tempo necessario per una ricerca esaustiva risulterebbe proibitivo. Tale criticità viene risolta attraverso la fase di **Bound**, che permette di effettuare un *pruning on-the-fly* sull'albero di ricerca, escludendo interi rami computazionalmente onerosi ma poco promettenti dal punto di vista della funzione obiettivo.

In particolare, per ogni nodo n esplorato, viene verificato il seguente vincolo di ammissibilità:

$$\text{Obj}(n) > \text{Obj}_{best} - \varepsilon \quad (4.3)$$

Qualora il vincolo venisse violato, il nodo non verrebbe sottoposto a *Branching*, troncando di fatto l'esplorazione del suo sottoalbero. È importante sottolineare la presenza del **fattore di tolleranza** ε : esso consente alla ricerca di non arrestarsi prematuramente, permettendo l'esplorazione di subnet che, pur essendo leggermente inferiori al miglior valore individuato (Obj_{best}), si trovano in regioni dello spazio degli stati potenzialmente ottimali. Questo approccio garantisce una ricerca più robusta e completa, anche se maggiormente onerosa in termini computazionali.

4.1.3 Ottimalità dell'algoritmo di ricerca

Risulta importante sottolineare che il *Branch and Bound* è un algoritmo ottimo, come dimostrato da Lawler e Wood [19]. Tale garanzia di ottimalità globale risiede nella definizione di un *bound* che sia **ammissibile**: in un problema di massimizzazione, il valore calcolato in ogni nodo deve rappresentare un limite superiore (*upper bound*) insuperabile per qualsiasi sottoproblema

derivato da quel nodo. Formalmente, detto S l'insieme delle soluzioni nel sotto-albero del nodo n , il bound è ammissibile se:

$$\text{Obj}(n) \geq \max_{x \in S} \text{Obj}(x) \quad (4.4)$$

dove $\text{Obj}(x)$ è la funzione obiettivo. Solo sotto questa condizione l'azione di *pruning* non rischia di escludere l'ottimo globale. Tuttavia, nel contesto della presente ricerca, l'applicazione del *Branch and Bound* assume una natura più euristica per due ragioni fondamentali:

1. **Assenza di un limite superiore teorico:** La funzione obiettivo utilizzata non è intrinsecamente monotona rispetto alla profondità dell'albero. La riduzione dei parametri aumenta con la profondità ma l'accuratezza risulta variabile e non strettamente correlata a quest'ultima, non permettendo quindi di calcolare un *bound* che sia un limite superiore certo per ogni possibile combinazione futura.
2. **Introduzione di una soglia di tolleranza:** Al fine di accelerare l'esplorazione di uno spazio degli stati altrimenti intrattabile, è stata introdotta una soglia ε nelle condizioni di *pruning*. Questa scelta trasforma l'algoritmo in un metodo di ricerca ε -**ottimo**, che garantisce l'identificazione di una soluzione entro un margine di errore controllato rispetto all'ottimo teorico, ma privilegia drasticamente la velocità di convergenza.

In conclusione, sebbene si rinunci alla garanzia di ottimalità assoluta definita in letteratura [19], l'algoritmo così configurato permette di navigare efficacemente il complesso *trade-off* tra performance e dimensione del modello, individuando *Subnet* di alta qualità che una ricerca esaustiva non avrebbe potuto identificare in tempi ragionevoli.

4.2 Formulazione delle Azioni di ricerca

Per ogni nodo dell'albero di ricerca, l'applicazione delle azioni di *pruning* genera i cinque nodi figli corrispondenti. Ciascuna azione identifica un gruppo specifico di parametri W_g all'interno dell'architettura che può essere rimosso minimizzando il degrado delle prestazioni del modello.

Al fine di individuare le componenti meno rilevanti per le capacità predittive del modello, viene adottata una metrica di importanza basata sulla sensibilità della funzione di perdita rispetto

ai parametri. Per un singolo parametro w , l'importanza è definita come:

$$\mathbb{I}(w) = w^2 \cdot \mathbb{E}_{x \sim D} \left[\left(\frac{\partial L(x; \theta^*)}{\partial w} \right)^2 \right], \quad (4.5)$$

dove $L(x; \theta^*)$ è la funzione di perdita, calcolata a partire dai parametri attivi θ^* , sui campioni x (*Search Set*). Si è scelto di adottare una tipologia di *pruning strutturato*, di conseguenza è necessario definire l'importanza di un intero gruppo di parametri W_g (come un intero neurone, dove $W_g = \{w_1, w_2, \dots, w_K, b\}$, considerato $y = \sum_{i=1}^K w_i x_i + b$) che viene calcolata come la somma dei contributi individuali:

$$\mathbb{I}(W_g) = \sum_{w \in W_g} \mathbb{I}(w). \quad (4.6)$$

Le specifiche azioni di *pruning* possono essere definite come un problema di minimizzazione locale volto a identificare il gruppo di dimensioni (formate da più parametri) g la cui rimozione minimizza l'impatto sulla funzione di perdita L . Per garantire la compatibilità tra il modello compresso e l'hardware pensato per l'accelerazione di questi modelli (Tensor Core [46]), imponiamo un vincolo di cardinalità $|g| = K$ (con K multiplo di 8). In tal modo è possibile definire il metodo proposto come **Hardware-Aware Pruning**. Di seguito la formulazione matematica dettagliata per ogni azione:

- **Pruning MLP:** Sia B l'insieme dei blocchi *Transformer* e $\text{MLP}_{b_i} = \{g \subset W_{\text{MLP}, b_i} : |g| = K\}$ l'insieme dei possibili raggruppamenti di cardinalità K di dimensioni del modulo MLP nel blocco i -esimo b_i . L'azione identifica il gruppo g_{MLP}^* che minimizza:

$$g_{\text{MLP}}^* = \arg \min_{\substack{b_i \in B \\ g_K \in \text{MLP}_{b_i}}} \sum_{W_g \in g_K} \mathbb{I}(W_g). \quad (4.7)$$

Come è possibile notare dalla figura 4.2 l'azione di *Pruning MLP* comporta la rimozione di K colonne di pesi, non necessariamente contigue, dalle matrici dei parametri del modulo *Multi-Layer Perceptron* del blocco i -esimo b_i .

- **Pruning QK (Query-Key):** Sia B l'insieme dei blocchi *Transformer* e $\text{QK}_{b_i} = \{g \subset W_{\text{QK}, b_i} : |g| = K\}$ l'insieme dei raggruppamenti di K dimensioni condivise tra i moduli Query e Key di tutte le teste di attenzione del blocco i -esimo. L'azione

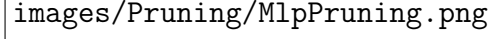


Figura 4.2: Visualizzazione delle matrici dei pesi mascherate dall'azione di *MLP Pruning*.

identifica il gruppo g_{QK}^* che minimizza:

$$g_{QK}^* = \arg \min_{\substack{b_i \in B \\ g_K \in QK_{b_i}}} \sum_{W_g \in g_K} \mathbb{I}(W_g). \quad (4.8)$$

Notare che le dimensioni dei moduli Query e Key vengono rimosse contestualmente a tutte le teste di attenzione del blocco i -esimo b_i . Questo è particolarmente evidente nella figura 4.3, ed è necessario per mantenere la consistenza matematica delle dimensioni delle matrici.

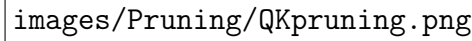


Figura 4.3: Visualizzazione delle matrici dei pesi mascherate dall'azione di *QK Pruning*, nella *MHSA*.

- **Pruning VPROJ (Value-Projection):** Sia B l'insieme dei blocchi *Transformer* e $VP_{b_i} = \{g \subset W_{VP, b_i} : |g| = K\}$ l'insieme dei raggruppamenti di cardinalità K di dimensioni dei moduli *Value* e *Output Projection* di tutte le teste di attenzione nel blocco i -esimo.

L'azione identifica il gruppo g_{VP}^* che minimizza:

$$g_{VP}^* = \arg \min_{\substack{b_i \in B \\ g_K \in VP_{b_i}}} \sum_{W_g \in g_K} \mathbb{I}(W_g). \quad (4.9)$$

Come è possibile osservare dalla figura 4.4, l'azione di *Pruning VProj* comporta la rimozione di K colonne di pesi, non necessariamente contigue, dalle matrici dei parametri dei moduli *Value* e *Output Projection* di tutte le teste di attenzione del blocco i -esimo b_i . Questo per le stesse ragioni di consistenza matematica delle dimensioni viste per il *Pruning QK*.

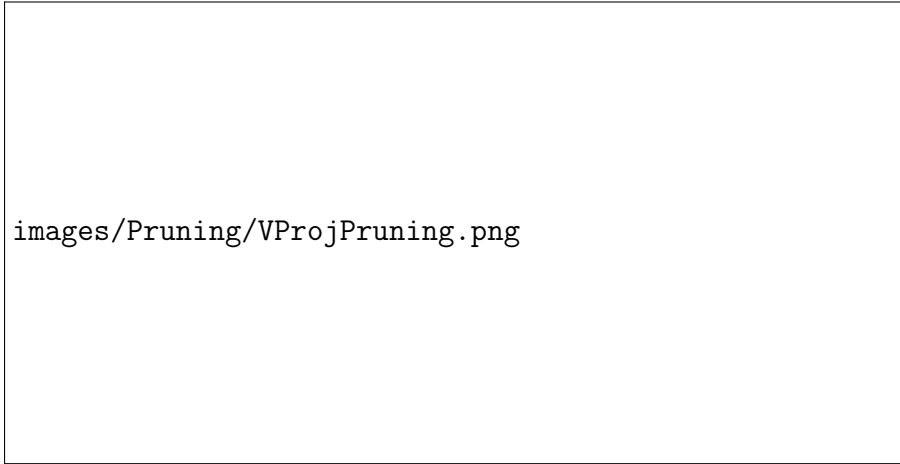


Figura 4.4: Visualizzazione delle matrici dei pesi mascherate dall'azione di *VProj Pruning*, nella *MHSA*.

- **Pruning HEAD:** Sia B l'insieme dei blocchi *Transformer* e H_{b_i} l'insieme di tutte le teste di attenzione nel blocco i -esimo. L'azione identifica la head h^* che minimizza:

$$h^* = \arg \min_{\substack{b_i \in B \\ h \in H_{b_i}}} \sum_{W_g \in h} \mathbb{I}(W_g). \quad (4.10)$$

L'azione di *Pruning Head* comporta la rimozione completa di una testa di attenzione, ovvero la mascheratura di tutte le dimensioni dei moduli Query, Key, Value e Output Projection associati alla head h^* in tutte le matrici dei parametri del blocco i -esimo b_i . Questo è particolarmente evidente nella figura 4.5.

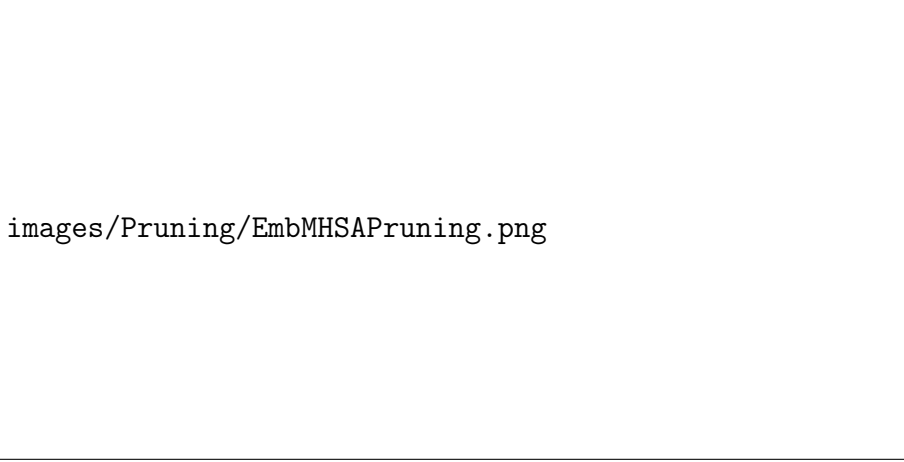


Figura 4.5: Visualizzazione delle matrici dei pesi mascherate dall'azione di *Head Pruning*, nella *MHSA*.

- **Pruning EMB (Embedding):** Sia $\text{EMB} = \{g \subset W_{\text{EMB}} : |g| = K\}$ l'insieme di tutti i raggruppamenti di cardinalità K di dimensioni dell'*Embedding* nell'intero modello *Transformer*. L'azione identifica il gruppo g_{EMB}^* che minimizza:

$$g_{\text{EMB}}^* = \arg \min_{g_K \in \text{EMB}} \sum_{W_g \in g_K} \mathbb{I}(W_g) \quad (4.11)$$

Si noti che, a differenza delle azioni precedenti, il *Pruning* dell'*Embedding* opera sulla dimensione globale del modello. La rimozione di un raggruppamento g_K in questa fase comporta una potatura della rappresentazione vettoriale che si propaga attraverso l'intera architettura, influenzando la dimensione di tutti i blocchi *Transformer*. Infine è importante osservare che questa azione comporta una eliminazione di righe delle matrici dei parametri sia nella *MHSA* che nella *FFN*, come è possibile notare dalle figure 4.6 e 4.7.



images/Pruning/EmbMHSAPruning.png

Figura 4.6: Visualizzazione delle matrici dei pesi mascherate dall'azione di *Emb Pruning*, nella *MHSA*.



images/Pruning/EmbMlpPruning.png

Figura 4.7: Visualizzazione delle matrici dei pesi mascherate dall'azione di *Emb Pruning*, nella *FFN*.

4.3 Derivazione del criterio di importanza

La metrica di importanza utilizzata combina le informazioni sulla magnitudo dei pesi e dei gradienti, al fine di misurare la sensibilità di uno specifico parametro a partire dai dati in input al modello. I parametri meno sensibili vengono interpretati come di scarso contributo all'output del modello, e di conseguenza vengono rimossi. La derivazione della metrica di importanza considera un singolo parametro w e il suo effetto sulla funzione di perdita L . In particolare si analizza la variazione di L al variare di w :

$$\Delta L = L(w + \delta w) - L(w), \quad (4.12)$$

$$L(w + \delta w) = L(w) + \Delta L. \quad (4.13)$$

Per approssimare il valore della funzione di perdita con il nuovo valore del parametro $L(w + \delta w)$ si utilizza l'espansione di Taylor al primo ordine:

$$L(w + \delta w) = L(w) + \frac{\partial L}{\partial w} \cdot \delta w + o(\delta w). \quad (4.14)$$

Dato che l'operazione effettuata è la potatura, il valore del parametro perturbato è zero (in quanto viene tagliato) e di conseguenza $\delta w = -w$, da cui possiamo ottenere il valore approssimato della variazione di L :

$$L(w + \delta w) - L(w) \approx (-w) \cdot \frac{\partial L}{\partial w}. \quad (4.15)$$

Applicato ad un gruppo di parametri W_g (ad esempio un intero neurone):

$$\Delta L(W_g) = \sum_{w \in W_g} \Delta L(w) = \sum_{w \in W_g} -w \cdot \frac{\partial L}{\partial w} = - \sum_{w \in W_g} w \cdot \frac{\partial L}{\partial w}. \quad (4.16)$$

Nonostante l'approssimazione al primo ordine sia ampiamente sfruttata in letteratura [52] per la sua semplicità computazionale, essa introduce un rischio sistematico nel caso del pruning strutturato. Si consideri un gruppo di parametri W_g contenente due parametri w_1 e w_2 . Nell'eventualità in cui le variazioni individuali della perdita siano opposte, ovvero $\Delta L(w_1) \approx -\Delta L(w_2)$, la loro somma algebrica risulterebbe prossima allo zero:

$$\Delta L(W_g) = \Delta L(w_1) + \Delta L(w_2) \approx 0. \quad (4.17)$$

Tale risultato porterebbe l'algoritmo a classificare erroneamente il gruppo W_g come irrilevante, ma in realtà è possibile che i singoli parametri possiedano un'elevata sensibilità. Poiché ΔL è una stima lineare e locale della reale superficie della funzione di perdita, fare affidamento sulla somma con segno dei contributi espone a decisioni di taglio errate, dove componenti critiche vengono rimosse a causa di una reciproca compensazione dei gradienti.

La soluzione a questo problema può essere individuata in un approccio più conservativo, utilizzando i quadrati per permettere la somma dei contributi individuali (che saranno così sempre positivi):

$$\mathbb{I}(W_g) = \sum_{w \in W_g} \Delta L(w)^2 = \sum_{w \in W_g} \left(w \cdot \frac{\partial L}{\partial w} \right)^2. \quad (4.18)$$

Notare che il calcolo del gradiente $\frac{\partial L}{\partial w}$ non è banale. Infatti accumulando semplicemente i gradienti, si rischia di incorrere in un fenomeno di cancellazione. Ad esempio detti b_1 e b_2 i due (unici) *batch* di dati in input, potrebbe accadere che:

$$\frac{\partial L}{\partial w}(b_1) \approx -\frac{\partial L}{\partial w}(b_2). \quad (4.19)$$

In fase di accumulo questo fenomeno porterebbe a un gradiente complessivo circa nullo, azzerando erroneamente l'importanza del parametro.

La soluzione a questo problema può essere facilmente individuata nell'utilizzo del valore atteso del gradiente sui batch, da cui:

$$\mathbb{I}(w) = w^2 \cdot \mathbb{E}_{b \sim D} \left[\left(\frac{\partial L}{\partial w}(b) \right)^2 \right] = w^2 \cdot \frac{1}{N} \sum_{i=1}^N \left(\frac{\partial L}{\partial w}(b) \right)^2, \quad (4.20)$$

con N numero di *batch* in input.

Date queste premesse, è possibile definire la seguente metrica di importanza per un gruppo di parametri W_g :

$$\mathbb{I}(W_g) = \sum_{w \in W_g} w^2 \cdot \mathbb{E}_{b \sim D} \left[\left(\frac{\partial L(b)}{\partial w} \right)^2 \right]. \quad (4.21)$$

4.4 Fisher Information Matrix ed Hessiano

La metrica di importanza derivata nella sezione precedente combina l'informazione fornita dalla magnitudo dei parametri con l'approssimazione diagonale della **Matrice di Informazione di Fisher** [49, 64], così definita:

$$F(w) = \mathbb{E}_{x,y} [\nabla L(y, f(x, w)) \cdot \nabla L(y, f(x, w))^T] \in \mathbb{R}^{|w| \times |w|}, \quad (4.22)$$

nella quale i singoli elementi rappresentano la correlazione tra i gradienti dei singoli parametri:

$$F(w_i, w_j) = \mathbb{E}_{x,y} \left[\frac{\partial L}{\partial w_i} \cdot \frac{\partial L}{\partial w_j} \right]. \quad (4.23)$$

Data la cardinalità dei parametri nei modelli *Transformer*, il calcolo e la memorizzazione della matrice completa risultano computazionalmente proibitivi. Si assume pertanto l'indipendenza tra i parametri, operazione che equivale a considerare non nulla esclusivamente la diagonale della matrice:

$$F(w_i, w_j) = 0 \quad \text{ed} \quad F(w_i, w_i) = \mathbb{E}_{x,y} \left[\left(\frac{\partial L}{\partial w_i} \right)^2 \right].$$

Empiricamente tale valore, detto **Informazione di Fisher**, viene stimato tramite una semplice media su un numero N di *batch* (x_k) in input:

$$F(w) = \frac{1}{N} \sum_{k=1}^N \left(\frac{\partial L(x_k)}{\partial w} \right)^2. \quad (4.24)$$

Notare che esiste un profondo legame tra l'*Informazione di Fisher* e l'*Hessiano* della funzione di perdita. A una simile conclusione sono giunti anche Theis et al. [64]. Di seguito la dimostrazione formale di questo legame, assumendo che la funzione di perdita sia la *Negative Log-Likelihood*.

Consideriamo un problema di classificazione in cui il modello definisce una distribuzione di probabilità $p(y|x, W)$. Per la proprietà di normalizzazione, la somma delle probabilità su tutte le classi $y \in Y$ è unitaria:

$$\sum_{y \in Y} p(y|x, W) = 1. \quad (4.25)$$

Derivando rispetto ai parametri W e sfruttando la linearità del gradiente:

$$\sum_y \nabla_W p(y|x, W) = 0. \quad (4.26)$$

Utilizzando una semplice proprietà delle derivate, per cui $\nabla p = p \frac{\nabla p}{p} = p \nabla \log(p)$, possiamo riscrivere l'espressione come:

$$\sum_y p(y|x, W) \cdot \nabla_W \log p(y|x, W) = 0. \quad (4.27)$$

Per brevità di notazione, assumiamo $p(y|x, W) = p(y)$ e deriviamo nuovamente rispetto a W :

$$\nabla_W \sum_y p(y) \cdot \nabla_W \log p(y) = 0. \quad (4.28)$$

Applicando la regola del prodotto:

$$\sum_y [\nabla_W p(y) \cdot \nabla_W \log p(y)^T + p(y) \cdot \nabla_W^2 \log p(y)] = 0. \quad (4.29)$$

Sostituendo nuovamente $\nabla_W p(y) = p(y) \nabla_W \log p(y)$, otteniamo:

$$\sum_y [p(y) \nabla_W \log p(y) \cdot \nabla_W \log p(y)^T + p(y) \cdot \nabla_W^2 \log p(y)] = 0. \quad (4.30)$$

Mettendo in evidenza $p(y)$ è possibile notare che costituisce la distribuzione di probabilità dei dati nel valore atteso $\mathbb{E}_{x,y}$. Sostituendo quindi con quest'ultimo otteniamo:

$$\mathbb{E}_{x,y} [\nabla_W \log p(y) \nabla_W \log p(y)^T + \nabla_W^2 \log p(y)] = 0. \quad (4.31)$$

Da questa relazione, si può notare come $\nabla_W \log p(y) \nabla_W \log p(y)^T$ sia proprio il termine legato alla Matrice di Fisher, mentre $\nabla_W^2 \log p(y)$ è il termine legato all'Hessiano. In particolare, si evince che il valore atteso della Fisher Information e dell'Hessiano sono legati dalla relazione:

$$\mathbb{E}_{x,y}[\text{Fisher}] + \mathbb{E}_{x,y}[\text{Hessiano}] = 0 \implies \mathbb{E}_{x,y} [\nabla \log p \nabla \log p^T] = -\mathbb{E}_{x,y} [\nabla^2 \log p]. \quad (4.32)$$

Considerato che è assunto l'utilizzo della funzione di perdita Cross-Entropy, che corrisponde alla Negative Log-Likelihood (NLL) nel caso della classificazione:

$$L = -\log p(y_{true}|x, W). \quad (4.33)$$

Le derivate prima e seconda rispetto ai parametri sono dunque:

$$\nabla L = -\nabla_W \log p, \quad \nabla^2 L = -\nabla_W^2 \log p. \quad (4.34)$$

Sostituendo queste identità nella (4.32), otteniamo l'equivalenza finale:

$$\mathbb{E}_{x,y} [\nabla L \cdot \nabla L^T] = \mathbb{E}_{x,y} [\nabla^2 L] \quad (4.35)$$

, il che implica, per le singole componenti diagonali:

$$\mathbb{E}_{x,y} \left[\left(\frac{\partial L}{\partial W_i} \right)^2 \right] = \mathbb{E}_{x,y} \left[\frac{\partial^2 L}{\partial W_i^2} \right]. \quad (4.36)$$

Considerazioni teoriche

Sebbene la teoria garantisca che la matrice di Fisher approssimi l'Hessiano, è importante sottolineare che tale equivalenza è strettamente valida solo quando il modello rappresenta correttamente la distribuzione dei dati reali, ovvero in condizioni di convergenza, come descritto approfonditamente da Kunstner et al. [35]. Nel metodo proposto, questa assunzione è considerata verificata poiché l'estrazione della metrica di importanza avviene su modelli che sono stati sottoposti ad una fase di fine-tuning. In tale stato di stabilità e convergenza, la *Fisher Information* fornisce una stima affidabile della curvatura della funzione di perdita, permettendo di identificare con precisione i parametri la cui rimozione minimizza l'impatto sulle prestazioni del Transformer.

Capitolo 5

Implementazione del Framework

In questo capitolo viene esposto l'intero framework di compressione proposto, costruito sulle fondamenta teoriche esposte nel Capitolo 4.

5.1 Gestione e Divisione dei dati

Nella fase iniziale il framework di compressione prevede di costruire il dataset. In particolare, inizialmente viene eseguita la suddivisione in *Dati di Train* e *Dati di Test*. Successivamente i primi vengono ulteriormente divisi in *Training Set* e *Validation Set*. Il secondo di questi verrà utilizzato esclusivamente per monitorare l'accuratezza del modello durante le iterazioni di compressione. Per quanto concerne il *Training Set*, questo viene utilizzato alla fine di ogni iterazione per eseguire una serie di epoche di *Fine Tuning*, al fine di permettere un riadattamento dei parametri del modello.

A partire dal *Training Set*, viene ulteriormente ricavato il *Search Set*, formato da un numero ben preciso di campioni per classe N , che costituisce un iper-parametro del processo di compressione; in questo modo la sua dimensione corrisponde ad $N \times |Classes|$. Dagli esperimenti effettuati emerge un valore ottimale di $N = 25$ campioni per classe. Questo permette una buona approssimazione dei gradienti, ma allo stesso tempo consente di mantenere contenuto il tempo necessario alla fase di ricerca. Infatti, il *Search Set* viene utilizzato su ogni nodo dell'albero di ricerca (ovvero su ogni *Subnet*) sia per il calcolo della metrica di importanza per ogni parametro (che prevede quindi un passo di *backpropagation*, dal grande costo computazionale), sia per la valutazione dell'accuratezza.



Figura 5.1: Processo di suddivisione del dataset adottato.

Per evitare che la fase di ricerca, utilizzando un unico *Search Set* per tutte le iterazioni, provochi un ***Overfitting Architettuale***, con conseguenti difficoltà di generalizzazione del modello compresso, il *Search Set* viene **campionato a ogni iterazione** a partire dal *Training Set*, fornendo campioni differenti all'algoritmo, di iterazione in iterazione. Questo permette di ottenere un modello risultante che è in grado di generalizzare al meglio sul problema di classificazione, proprio perché l'architettura finale è il risultato ottenuto con una maggiore quantità e varietà di dati.

Un altro contributo fondamentale al mantenimento della capacità di generalizzazione del modello è dato dall'applicazione sul *Search Set* del medesimo processo di **Data Augmentation** che viene applicato ai dati del *Training Set*. Si potrebbe pensare che questo introduca rumore nei gradienti, e quindi influenzi la precisione e l'affidabilità della metrica di importanza. Nella pratica però il problema del rumore è fortemente limitato dall'utilizzo della media dei gradienti sui batch per il calcolo della metrica di importanza, com'è possibile osservare nella Formula (4.5). La *Data Augmentation* applicata al *Search Set* è risultata molto efficace, nei test effettuati, per impedire che venissero sacrificate delle dimensioni del modello essenziali per mantenere la sua capacità di generalizzazione, traducendosi in questo modo in valori di accuratezza sul *Test Set* e sul *Validation Set* più elevati durante tutta l'esecuzione del framework.

Notare che è possibile interpretare il framework di compressione come un **ottimizzazione a due livelli** del modello. In particolare il livello superiore è occupato dal processo di ricerca, che esegue un vero e proprio **training architettuale**, mentre il livello inferiore è quello del *Fine Tuning*, che invece si occupa di **ottimizzare i parametri**, date le modifiche architeturali

operate dal livello superiore. Questa visione del metodo permette di comprendere al meglio le motivazioni dietro la specifica gestione dei dati, ed in particolare del *Search Set*. Infatti, l'applicazione di tecniche di *data augmentation* a quest'ultimo implica una dinamicità che contribuisce a scongiurare il fenomeno dell'*overfitting architetturale*, analogamente a come tecniche quali l'*early stopping* o i *Learning Rate Scheduler* vengono impiegate per prevenire l'*overfitting* dei parametri del modello.

5.2 Framework di Compressione Iterativo

Una volta definita la strategia di gestione dei dati, è necessario definire la struttura del framework di compressione iterativo, in particolare l'intero processo che costituisce una singola iterazione.



Figura 5.2: Diagramma di flusso del processo di compressione iterativo.

Inizialmente viene campionato il *Search Set*, da utilizzare in fase di ricerca. Successivamente ha inizio la fase di **Pruning**. La prima operazione eseguita è l'algoritmo di ricerca *BnB*, che verrà analizzato nella sezione 5.3, il quale restituisce lo stato migliore individuato, ovvero una maschera di *pruning* per l'intero modello.

Quest'ultima viene applicata al modello, e questo viene successivamente ricostruito istanziando tutti i tensori dei parametri ed effettuando una copia solo di quelli non mascherati in fase di ricerca. Viene quindi creato un nuovo modello *ViT*, del quale sono state effettivamente ridotte le dimensioni e questo risulta fondamentale perché implica che il framework proposto non si limita ad applicare delle maschere di *pruning*, che non avrebbero alcun impatto sulle prestazioni effettive del modello sull'hardware, bensì produce dei nuovi modelli di dimensioni realmente minori, con prestazioni realmente superiori. In particolare viene ridotta la latenza in fase di inferenza ma anche la memoria necessaria (VRAM) per utilizzare il modello.

Una volta ricostruito il modello, dato che la fase di *Pruning* avrà introdotto un danno alle performance del modello, è necessario passare alla successiva fase di **Recovery Fine Tuning**. Si tratta di una serie di epoche di addestramento, volte a consentire ai parametri rimasti del modello di riadattarsi, per poter recuperare l'accuratezza perduta. Risulta fondamentale in questa fase la possibilità di introdurre la *Knowledge Distillation* [29] tramite il modello originale non compresso, che permette tipicamente un recupero più efficace della performance del modello.

Algorithm 1 Framework di Compressione Iterativo

```

1: Input: Pre-trained model  $M_0$ , Dataset  $\mathcal{D}$ , Iterations  $N$ , Fine Tuning epochs  $K$ 
2: Output: Compressed and optimized model  $M$ 
3:  $M_{teacher} \leftarrow M_0$  ▷ Original model used for distillation
4:  $M \leftarrow M_0$ 
5: for  $n \leftarrow 1$  to  $N$  do
6:    $\mathcal{D}_{search} \leftarrow \text{sample\_search\_set}(\mathcal{D}_{train})$  ▷ Dynamic sampling of the search set
7:    $m_{pruning} \leftarrow \text{pruning\_algorithm}(M, \mathcal{D}_{search})$  ▷ obtain pruning mask
8:    $M_{pruned} \leftarrow \text{compress\_ViT}(M, m_{pruning})$  ▷ Rebuilding ViT
9:    $M \leftarrow M_{pruned}$ 
10:   $M \leftarrow \text{FineTuning}(M, M_{teacher}, \mathcal{D}_{train}, \mathcal{D}_{val}, K)$ 
11: end for
12: return  $M$ 

```

5.3 Implementazione dell'algoritmo di ricerca

In questa sezione viene dettagliata l'implementazione della fase di ricerca con algoritmo *Branch and Bound*, che viene eseguita all'inizio di ogni iterazione di compressione.

Al fine di evitare un eccessivo utilizzo di memoria, il processo di ricerca fa utilizzo di stati costituiti come strutture dati a dizionario, invece che vere e proprie copie del modello originale. All'interno di ogni stato è presente una struttura che, per ogni blocco *Transformer* contiene tutte le dimensioni rimosse per ogni tipologia di struttura (MLP, QK, ecc.), oltre a una informazione generale su quali dimensioni di *Embedding* sono state eliminate globalmente.

A partire da questa struttura dati è possibile agire su una serie di maschere di *pruning* che vengono applicate al modello, permettendo così di ottenere la vera e propria *subnet*. Per garantire la massima efficienza queste maschere vengono applicate ad un'unica copia del modello iniziale, che, alla fine di ogni esplorazione dello stato vede le proprie maschere reimpostate, per poter applicare i tagli previsti dal successivo stato della ricerca.

Algorithm 2 Ricerca via Depth-First Branch and Bound a profondità limitata

```
1: Input: Base Model  $M$ , Depth Limit  $D_{max}$ 
2: Output: Optimal State  $s_{best}$ , Optimal State value  $v_{best}$ 
3:  $s_{init} \leftarrow \text{build\_initial\_state}()$ 
4:  $stack \leftarrow [s_{init}]$  ▷ Stack setup
5:  $M_{work} \leftarrow M$ 
6:  $s_{best} \leftarrow s_{init}$ 
7:  $v_{best} \leftarrow -\infty$ 
8: while  $stack$  is not empty do
9:    $s_{curr} \leftarrow stack.pop()$ 
10:   $\text{reset\_masks}(M_{work})$ 
11:   $\text{apply\_masks}(M_{work}, s_{curr})$ 
12:   $\text{accuracy}, \text{active\_params} \leftarrow \text{evaluate}(M_{work})$  ▷ Subnet evaluation
13:   $v_{curr} \leftarrow \text{compute\_obj}(\text{acc}, \text{active\_params})$ 
14:  if  $v_{curr} > v_{best}$  then
15:     $v_{best}, s_{best} \leftarrow v_{curr}, s_{curr}$ 
16:  end if
17:  if  $D_{max}$  reached OR  $\text{Bound}(v_{curr}, v_{best})$  then
18:    continue ▷ Bound or depth limit reached
19:  end if
20:  // Branching
21:   $\text{next\_states} \leftarrow \text{Branch}(s_{curr}, M_{work})$ 
22:  for  $s$  in  $\text{next\_states}$  do
23:     $stack.push(s)$ 
24:  end for
25: end while
26: return  $s_{best}, v_{best}$ 
```

5.4 Implementazione delle azioni di Pruning

Data la formulazione matematica delle azioni di ricerca presentata nella sezione 4.2 e considerato l'elevatissimo numero di dimensioni nel modello, in fase di implementazione, viene adottato un approccio più efficiente. In particolare, non vengono considerate realmente tutte le possibili combinazioni di dimensioni a K a K di un componente del modello, poiché, dato che è di interesse la somma delle importanze delle singole dimensioni, è sufficiente considerare le K dimensioni le cui importanze sono minime. In particolare, per tutte quelle azioni che agiscono all'interno di un singolo blocco (ovvero tutte tranne il *Pruning* della dimensione di *Embedding*),

vengono identificate per ogni blocco *Transformer* le K dimensioni con il minor valore di importanza. Successivamente, viene selezionato il blocco il cui gruppo di dimensioni ottimali hanno una importanza complessiva minore. Questo approccio semplifica e rende la risoluzione del problema più efficiente.

Per quanto riguarda il *Pruning* della dimensione di *Embedding*, l'approccio adottato è il medesimo, con la differenza che, essendo questa una dimensione globale, non vengono considerati singolarmente i blocchi bensì la totalità, riducendo l'algoritmo all'individuazione dei K minimi valori di importanza sull'intero modello.

Algorithm 3 Selezione del gruppo di dimensioni ottimale per il taglio

```

1: Input: Model  $M$ , Chunk size  $K$ , Target component  $T$ 
2: Output: (block, [dimension indices]) Target dimension group
3:  $v_{best\_imp} \leftarrow \infty$ 
4:  $target \leftarrow \text{null}$ 
5: for every block  $b$  in  $M.blocks$  do
6:   if number of dimensions in  $T(b) < K$  then
7:     continue
8:   end if
9:    $candidates \leftarrow \emptyset$ 
10:  for every dimension  $d$  in  $T(b)$  do
11:    if  $d$  is already pruned then
12:      continue
13:    end if
14:     $imp_d \leftarrow \text{importance\_score}(d)$ 
15:     $candidates.append((d, imp_d))$ 
16:  end for
17:   $candidates \leftarrow \text{sort}(candidates, \text{ascending} = \text{True})$ 
18:   $chunk \leftarrow candidates[0 : K]$ 
19:   $v_{block\_imp} \leftarrow \sum_{(d,i) \in chunk} i$ 
20:  if  $v_{block\_imp} < v_{best\_imp}$  then
21:     $v_{best\_imp} \leftarrow v_{block\_imp}$ 
22:     $target \leftarrow (b, chunk.indices)$ 
23:  end if
24: end for
25: return  $target$ 

```

Capitolo 6

Esperimenti e risultati

In questo capitolo vengono presentati i risultati ottenuti dal framework proposto, tramite una serie di esperimenti che coinvolgono modelli e dataset differenti. Tutti gli esperimenti vengono svolti con modelli *Vision Transformer*, preaddestrati sul dataset **ImageNet**, che utilizzano *patch* di dimensione 16×16 e immagini in *input* di tipo **RGB** con risoluzione 224×224 .

6.1 Dataset utilizzati

Al fine di valutare l'efficacia del framework sono stati selezionati due dataset differenti, di cui il primo più semplice e di dimensioni ridotte, mentre il secondo molto più vasto e complesso. Questa scelta è stata effettuata al fine di mostrare il comportamento dell'algoritmo di compressione in contesti differenti.

6.1.1 CIFAR-100

Il primo dataset utilizzato per testare il framework è stato il **CIFAR100**, che è stato selezionato poiché molto diffuso in letteratura, con il ruolo di benchmark standard per compiti di classificazione. Risulta notevolmente più complesso rispetto al *CIFAR10*, ma mantiene dimensioni ridotte e quindi permette una ottima rapidità di *training*. In particolare *CIFAR100* contiene 60.000 immagini totali, suddivise in 50.000 immagini di addestramento e 10.000 di validazione, tutte della risoluzione di 32×32 e di tipo **RGB**.

Per i test sul dataset *CIFAR100* è stata effettuata una ulteriore suddivisione del set di addestramento con rapporto **90/10**, in modo tale da ottenere un **Train Set** finale di **45.000** immagini

e un **Held-Out Set** di **5.000** immagini, quest'ultimo utilizzato per il monitoraggio del processo di compressione.

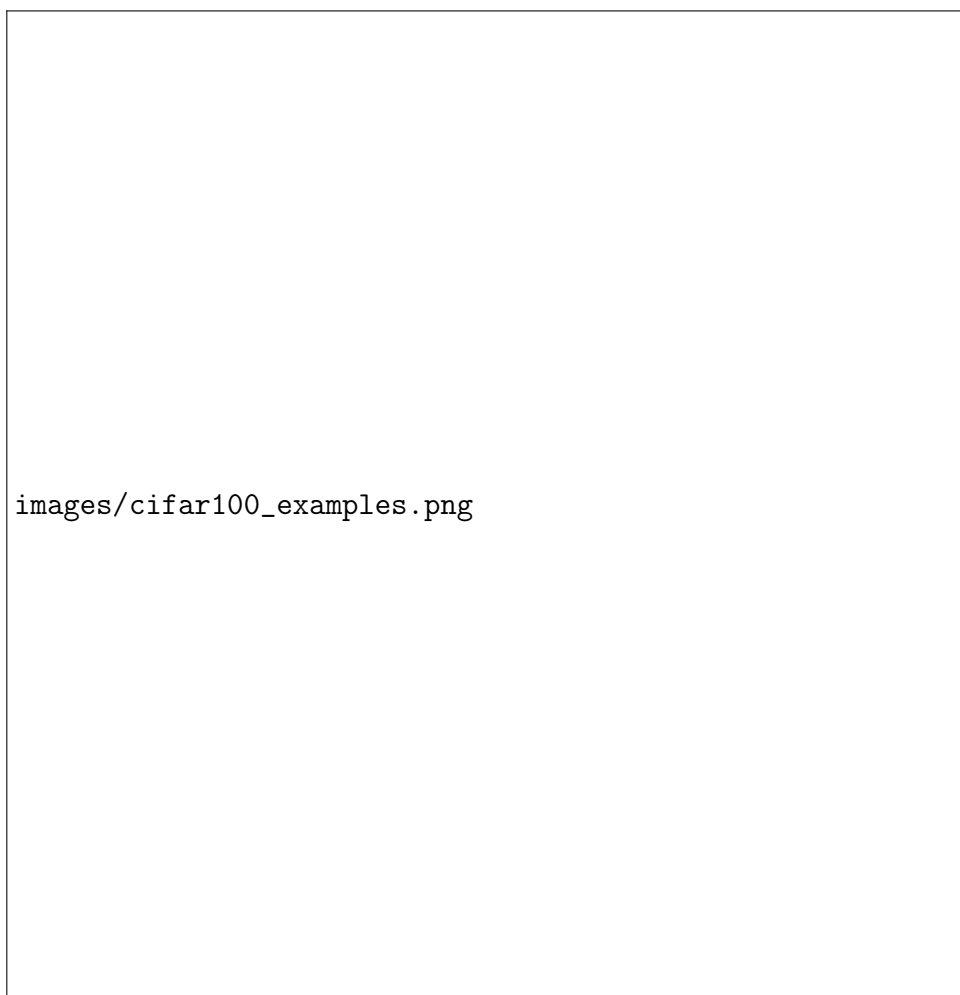


Figura 6.1: Esempio di immagini dal dataset CIFAR100.

6.1.2 ImageNet-1K

Il secondo dataset adottato per testare il framework è **ImageNet-1K** [16], molto diffuso in letteratura e molto più complesso rispetto al precedente. Si tratta di un *benchmark* utilizzato sin dalle prime reti convoluzionali, come *Alex-Net*, per task di *Image Classification* e *Object Detection*. In questo caso la dimensione è di gran lunga maggiore rispetto al precedente, poiché

contiene circa 1.300.000 immagini di addestramento, e 50.000 immagini di validazione, tutte di tipo **RGB** ma con risoluzione variabile (tipicamente molto superiore a quella del CIFAR-100).

Per quanto concerne i test su *Imagenet-1K*, il set di addestramento è stato suddiviso con rapporto **97/3** al fine di ottenere un **Held-Out Set** di circa 38.000 immagini, per la validazione del processo di compressione ed eventuale *Early Stop*.



Figura 6.2: Esempio di immagini del dataset ImageNet-1K.

6.2 Data Augmentation

Data la la grande capacità dei modelli *Vision Transformer*, per scongiurare il rischio di *overfitting*, sono state adottate delle tecniche di **Data Augmentation**, in particolare, per quanto riguarda i test su modelli ViT:

- **Random Resized Crop:** Questa tecnica estrae una porzione casuale dell'immagine originale con un'area compresa tra l'**80%** e il **100%** della dimensione iniziale. Successivamente, il ritaglio viene ridimensionato alla risoluzione *target* di 224×224 pixel.
- **Random Horizontal Flip:** Consiste nel riflettere l'immagine orizzontalmente con una probabilità del **50%**.

- **Color Jitter:** Questa trasformazione applica variazioni casuali alla luminosità, al contrasto e alla saturazione dell'immagine, con un fattore di distorsione impostato a **0.3** per ciascun parametro.

Per i modelli DeiT invece, sono state adottate le modalità di *data augmentation* proposte dagli autori[66], che prevedono:

- **RandAugment:** Viene adottata la policy *rand-m9-mstd0.5-inc1*. Questa versione avanzata di *RandAugment* applica trasformazioni casuali con una intensità media di 9 e una deviazione standard di 0.5, introducendo una forte variabilità nei dati al fine di favorire una migliore generalizzazione.
- **Color Jitter:** A differenza della configurazione precedente, il fattore di distorsione per luminosità, contrasto e saturazione viene elevato a 0.4, aumentando la robustezza del modello alle variazioni cromatiche.
- **Random Erasing:** La probabilità di applicazione del *random erasing* è impostata a 0.0 per il dataset CIFAR-100, e a 0.25 per il dataset ImageNet-1K. Come suggerito dagli autori per configurazioni specifiche o dataset a risoluzione ridotta, risulta utile evitare di mascherare porzioni dell'immagine per non perdere informazioni spaziali essenziali durante il processo di apprendimento.

6.3 Dettaglio implementativo

Di seguito i dettagli implementativi delle fasi di addestramento del framework, precedentemente e contestualmente alla compressione. Tutti i test sono stati condotti su una GPU NVIDIA RTX 2070, per quanto riguarda invece i valori di *throughput* e latenza, questi sono stati misurati su una GPU NVIDIA RTX 4080 mobile. Per la prima fase di *Fine Tuning* dei modelli ViT, è stata adottata la seguente configurazione:

- **Ottimizzatore AdamW** con un fattore di **decadimento dei pesi** pari a **0.05** e con **learning rate discriminativi**: 0.5×10^{-5} per il *backbone* del *Transformer* e 0.5×10^{-4} per la testa di classificazione.
- **Scheduler** dei *learning rate* di tipo **Cosine Annealing**.

- Dimensione dei **batch** di *training* pari a **128** campioni.
- **30 epoche** di addestramento con **early stopping**.

Nel caso dei modelli DeiT è stata adottata una configurazione di addestramento simile ma più articolata, seguendo le indicazioni fornite dagli autori Touvron et al. [66], che prevedono l'utilizzo di tecniche avanzate di training, in particolare:

- **Ottimizzatore AdamW** con un fattore di **decadimento dei pesi** pari a **0.05** e con **learning rate discriminativi** raddoppiati rispetto ai modelli ViT.
- **Scheduler** dei *learning rate* di tipo **Cosine Annealing**.
- Dimensione dei **batch** di *training* pari a **128** campioni.
- **50 epoche** di addestramento con **early stopping** (pazienza pari a 10).
- **CutMix e Mixup**: applicati a livello di *batch*, con una probabilità di applicazione del 50% per entrambi (vengono adottati in maniera esclusiva, non combinata). Queste tecniche di regolarizzazione avanzate prevengono l'*overfitting* alterando le immagini e le rispettive etichette: il *Mixup* [81] sovrappone due immagini in trasparenza interpolandone le probabilità target, mentre il *CutMix* [79] ritaglia una porzione di un'immagine e la sovrappone parzialmente ad un'altra. L'adozione di queste strategie ha portato all'utilizzo della funzione di perdita **Soft-Target Cross Entropy**, in aggiunta ad un valore di **label smoothing** pari a 0.1, che ammorbidisce le etichette target, evitando che il modello diventi troppo sicuro delle sue predizioni e migliorando così la generalizzazione.
- Non sono state utilizzate tecniche quali il *Random Erasing* oppure il *Drop Path* poiché, come suggerito dagli autori stessi [66], risultano controproducenti per eseguire *Fine Tuning* come nel caso in esame.

Per quanto riguarda invece la fase di **Recovery Fine-Tuning**, volta a ripristinare le prestazioni dopo il taglio dei parametri, viene adottata la medesima configurazione di ottimizzazione del *Fine-Tuning* iniziale, con alcune variazioni.

Per quanto concerne i test sui modelli ViT è stato impiegato un *learning rate* univoco pari a 0.5×10^{-5} per tutta la rete e un limite massimo di 20 epoche di addestramento, sufficienti per ripristinare e adattare i pesi della *subnet* individuata.

Nel caso dei modelli DeiT invece, oltre alle stesse variazioni adottate per i modelli ViT, è stata disattivata l'applicazione delle tecniche di *CutMix* e *Mixup*, e di conseguenza anche il *label smoothing*. Questa scelta implementativa è stata effettuata considerando che le *subnet* individuate si trovano già in condizioni vicine alla convergenza, e non risulta necessario introdurre ulteriori tecniche di regolarizzazione, inoltre è possibile considerare il processo di ricerca stesso, con i suoi tagli progressivi, come una forma di regolarizzazione finalizzata a prevenire l'*overfitting*.

Infine, per quanto riguarda i test con la **Knowledge Distillation (KD)**, è stata implementata una versione *logit-based* (Equazione 6.1) seguendo l'approccio proposto da Hinton et al. [29]. Il modello teacher, in tutti i test, consiste nel modello originale, ovvero non sottoposto ad alcuna compressione, ma specializzato sul dominio di interesse. La funzione di perdita utilizzata durante il *Recovery Fine-Tuning* integra un termine di distillazione con temperatura $\tau = 4.0$. A tale componente è stato assegnato un peso relativo pari a 0.9, privilegiando così il trasferimento della conoscenza dal modello *teacher* alla *subnet* compressa.

$$L_{\text{total}} = (1 - \alpha)L_{\text{CE}}(y, z_s) + \alpha\tau^2 L_{\text{KL}}(z_s^\tau, z_t^\tau), \quad (6.1)$$

dove z_s e z_t denotano rispettivamente i logit in uscita dai modelli *student* e *teacher*. Il termine L_{CE} rappresenta la *Cross-Entropy* calcolata rispetto alle etichette reali y , mentre L_{KL} indica la divergenza di Kullback-Leibler. I logit scalati dal parametro di temperatura τ sono definiti come $z^\tau = \text{softmax}(z/\tau)$, e $\alpha \in [0, 1]$ funge da coefficiente di bilanciamento tra le due componenti della funzione obiettivo.

Nonostante Touvron et al. [66] mostrano la superiorità della *hard distillation* rispetto alla *soft distillation* per i modelli *Vision Transformer*, l'adozione della seconda è stata preferita a causa di un diverso contesto di applicazione. Infatti, se nel caso degli autori del modello DeiT, la distillazione viene utilizzata in fase di addestramento iniziale, nel caso in esame viene adottata in fase di *Recovery Fine-Tuning*, nel quale la *subnet* individuata si trova già in condizioni di convergenza vicine a quelle del modello originale. In tale situazione il trasferimento della conoscenza attraverso l'intera distribuzione di probabilità del modello *teacher*, piuttosto che alla sola classe target, risulta più informativa, e di conseguenza utile, per guidare il processo di recupero della *subnet* compressa,

6.4 Metriche

Per valutare le prestazioni dei modelli nei test effettuati, e di conseguenza per valutare l'efficacia del framework di compressione, sono state utilizzate le seguenti metriche:

- **Numero di Parametri:** consiste nella quantità di parametri che costituiscono il modello, prima e dopo l'esecuzione del processo di compressione.
- **Accuracy:** al fine di valutare le prestazioni del modello sul task di riferimento, viene utilizzata la *Top-1 Accuracy*, valutata sui dati di test del dataset utilizzato.
- **GFLOPs:** si tratta del numero di *Floating Point Operations* (operazioni in virgola mobile) che sono necessarie per eseguire un singolo passaggio di inferenza (*forward pass*) sul modello. In particolare 1 GFLOP corrisponde ad 1×10^9 operazioni in virgola mobile. A differenza della latenza, che risulta legata all'hardware utilizzato, i GFLOPs forniscono una misura deterministica della complessità computazionale del modello.
- **Throughput:** consiste nel numero di immagini al secondo che vengono processate dal modello, e viene calcolato costruendo un batch di immagini ottenute tramite la generazione di un tensore randomico, e successivamente misurando il tempo necessario al modello per eseguire l'inferenza su tale tensore.
- **Latency:** viene calcolata congiuntamente alla precedente, e misura (in ms) il tempo necessario per l'inferenza di un singolo batch di immagini.

6.5 Test con ViT-Small

I seguenti test sono eseguiti sul modello *vit_small_patch16_224.augreg_in21k_ft_in1k* della libreria **timm**. Trattasi di un modello preaddestrato sul dataset ImageNet-21K, e successivamente specializzato su ImageNet-1K tramite *fine tuning*.

6.5.1 Test sul dataset CIFAR100

In questi test è stato utilizzato il *ViT-Small* che, con l'aggiunta della testa di classificazione da 100 unità, ha raggiunto **21.7** Mln di parametri. Il modello è stato successivamente sottoposto alla fase di *fine tuning* iniziale sul dataset in oggetto. Per quanto riguarda il framework, tutti i

test sono stati effettuati con **15 iterazioni** di compressione. Ad ogni iterazione, la **profondità** dell'albero di ricerca è stata limitata a **6**, questo implica che al modello possono essere applicate fino a **6 azioni consecutive** di *pruning* prima di essere nuovamente addestrato. Viene inoltre utilizzata una **soglia di tolleranza** della fase di *Bound* pari a **0.005**, in modo tale da permettere una più approfondita esplorazione dello spazio di ricerca, al costo di un numero lievemente maggiore di nodi esplorati. Infine, la **funzione obiettivo** utilizza un valore di λ pari a **1.0**, attribuito al contributo dei parametri.

Model	Params (M) ↓	Top-1 (%) ↑	FLOPs (G) ↓	Throughput (imgs/s) ↑	Latency ($\Delta\%$) ↓
ViT-Small	21.70	90.42%	4.60	1137.5	112.5
Pruned	17.59 (-19.0%)	88.92% (-1.50%)	3.74 (-18.67%)	1308.6 (+15.04%)	97.8 (-13.06%)
Pruned+KD	15.78 (-27.3%)	89.86% (-0.56%)	3.39 (-26.19%)	1406.0 (+23.60%)	91.0 (-19.11%)

Tabella 6.1: Nella tabella sono presentati i risultati comparativi utilizzando il *ViT-Small* e il dataset *CIFAR100*, in particolare vengono evidenziati i valori delle metriche e le variazioni rispetto al modello *Baseline*.

I risultati illustrati dalla Tabella 6.1 dimostrano chiaramente un eccesso di parametri del modello, che infatti possono essere rimossi senza intaccare in maniera significativa le prestazioni. Il framework è riuscito a comprimere il modello di circa il **27%**, con una perdita di accuratezza inferiore di circa lo **0.5%**, e un aumento del throughput superiore al **23%**. L'aggiunta della *Knowledge Distillation* ha permesso una maggiore compressione del modello, con una accuratezza significativamente superiore (quasi 1.0% in più) rispetto al test senza KD.

6.5.2 Test sul dataset ImageNet-1K

In questo caso il modello *ViT-Small* ha raggiunto i 22 Mln di parametri con l'aggiunta della testa di classificazione da 1000 unità. Per questi test il modello *baseline* è stato ottenuto sottoponendo a *fine tuning* sul *train set*, ma con la *backbone* bloccata. I parametri del framework, per tutti i test, sono stati configurati a **15 iterazioni** di compressione, la **profondità** dell'albero di ricerca è stata limitata a **6**, la **soglia di tolleranza** della fase di *Bound* pari a **0.005**, la **funzione obiettivo** con λ pari a **1.0** per i parametri.

Model	Params (M) ↓	Top-1 (%) ↑	FLOPs (G) ↓	Throughput (imgs/s) ↑	Latency (Δ%) ↓
ViT-Small	22.05	81.20%	4.60	1142.6	112.0
Pruned	18.77 (-14.9%)	79.17% (-2.03%)	3.90 (-15.13%)	1270.5 (+11.20%)	100.7 (-10.07%)
Pruned+KD	17.18 (-22.1%)	81.12% (-0.08%)	3.58 (-22.26%)	1321.9 (+15.70%)	96.8 (-13.57%)

Tabella 6.2: Nella tabella sono presentati i risultati comparativi utilizzando il *ViT-Small* e il dataset *ImageNet-1K*, in particolare vengono evidenziati i valori delle metriche e le variazioni rispetto al modello *Baseline*.

Come è possibile notare, i risultati ottenuti su *ImageNet-1K* rispecchiano la maggiore complessità del dataset, con una diminuzione più marcata dell'accuratezza, a fronte di una minore percentuale di compressione. Nonostante ciò, il framework, con l'ausilio della *Knowledge Distillation*, ha portato ad un risultato rilevante, con una perdita di accuratezza inferiore allo **0.1%**, a fronte di una compressione superiore al **22%**, e un aumento del throughput superiore al **15%**.

La caratteristica della KD di agevolare il taglio di un numero maggiore di parametri, osservata anche nel caso del test precedente (Tabella 6.1), è giustificata dalla natura iterativa del framework. Agendo come forte regolarizzatore durante le fasi di fine-tuning intermedie, la KD permette al modello di mantenere le prestazioni più elevate ed uno stato di convergenza più solido. Ciò permette alle iterazioni di compressione successive di sfruttare al meglio la metrica di importanza, la cui efficacia aumenta con la vicinanza del modello allo stato di convergenza, come discusso nella sezione 5.4, permettendo tagli più profondi.

Notare inoltre che la compressione inferiore ottenuta rispetto al test su CIFAR-100, sarebbe facilmente colmabile aumentando il numero di iterazioni a disposizione del framework. Nel caso in esame è stato scelto di mantenere quest'ultimo uguale per entrambi i dataset, in maniera tale da agevolare un confronto, ma soprattutto per dimostrare come l'algoritmo di ricerca adottato sia sensibile e capace di adattarsi alla complessità dei dati, selezionando in maniera più conservativa le azioni di *pruning* da eseguire, al fine di mantenere elevate le prestazioni del modello.

6.5.3 Analisi dell'algoritmo di ricerca con il modello ViT-Small

In questa sezione viene illustrato il funzionamento dell'algoritmo di ricerca oltre agli interessanti *pattern* individuati nel taglio dei parametri.

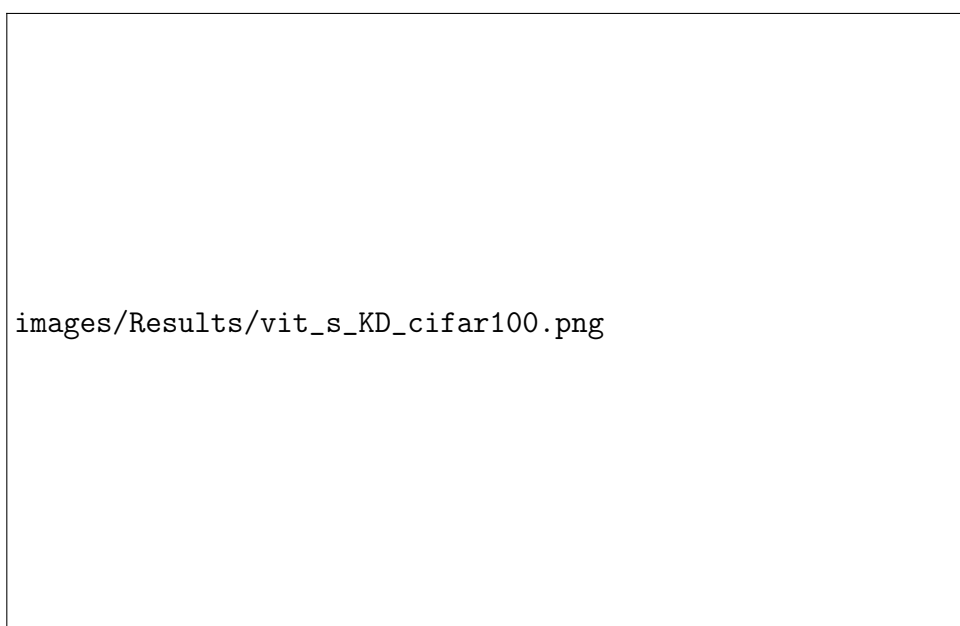


Figura 6.3: Illustrazione del processo di ricerca che ha portato alla compressione di ViT-Small sul dataset CIFAR100, con *Knowledge Distillation* attiva. In alto è visibile l’andamento della accuratezza (top-1) pre e post *fine tuning*, al centro il numero di *subnet* (nodi) esplorati per ogni iterazione, in basso la compressione progressiva in termini di numero di parametri.

Guardando la Figura 6.3, è possibile notare una serie di particolarità. Partendo dal primo grafico in alto, è importante osservare come nelle prime iterazioni la compressione corrisponda ad un effettivo aumento della *accuracy*. In corrispondenza dell’iterazione 5 infatti, il modello risulta ridotto di circa il 10% con il livello di accuratezza massima raggiunta sul set di validazione. Questo risultato si verifica spesso nei test eseguiti e conferma il metodo di compressione proposto come tecnica efficace di **regolarizzazione**.

Per quanto riguarda il secondo grafico, è possibile notare come il numero di *subnet* esplorate sia spesso vicino a 40, con alcune eccezioni per i quali l’algoritmo ha approfondito la ricerca fino ad oltre 80 nodi. Nonostante questi picchi, dal terzo grafico si evince che il framework produce una compressione sempre graduale e vicina ad un comportamento lineare. Questo si traduce in cali di *accuracy* mai eccessivi al termine della ricerca, che infatti non superano il 2%, e scendendo al massimo a circa il 3% dalla *baseline* del modello originale.



Figura 6.4: Illustrazione delle percentuali di dimensioni tagliate nei vari blocchi *Encoder* del ViT-Small, nel caso del test sul dataset CIFAR100 con l'utilizzo di *Knowledge Distillation*.

Per quanto riguarda invece la Figura 6.4, emergono dei dettagli interessanti sui **pattern di compressione** a vari livelli di profondità. Questi ultimi rimangono inalterati al variare del dataset e anche dell'utilizzo della distillazione. In particolare, è possibile notare come il framework di compressione abbia prediletto il taglio del meccanismo di attenzione nei primi livelli (specialmente il primo, al quale sono state rimosse 5 delle 6 teste). Nei livelli più profondi invece, sono stati osservati dei tagli più marcati nelle strutture MLP. Molto interessante è invece la decisione di lasciare inalterati i blocchi centrali della rete.

Al fine di confrontare i due processi di compressione, effettuati utilizzando i due dataset differenti, di seguito vengono illustrati ed analizzati i risultati ottenuti utilizzando lo stesso modello (ViT-Small) sul dataset ImageNet-1K.

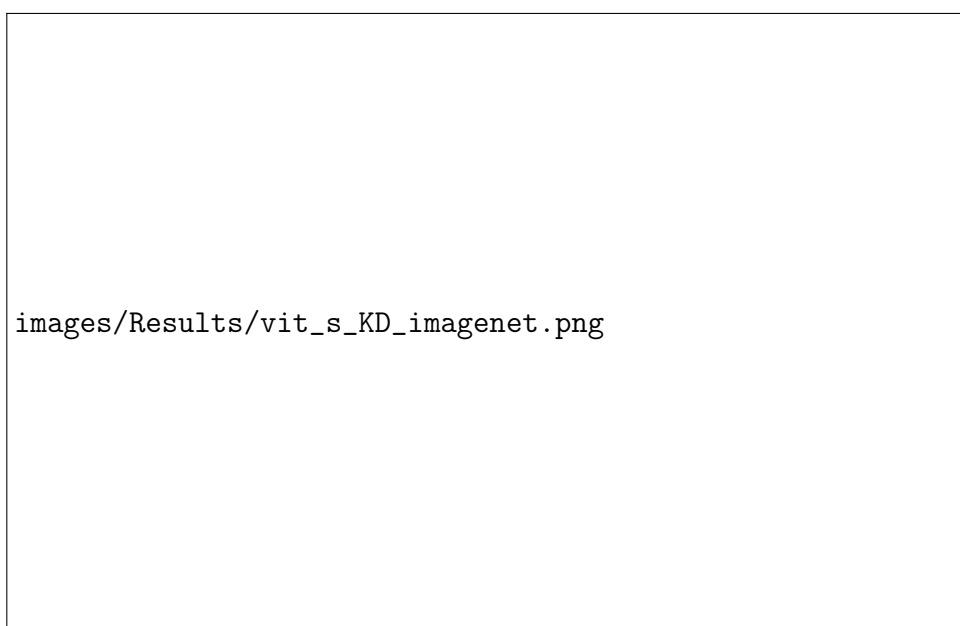


Figura 6.5: Illustrazione del processo di ricerca che ha portato alla compressione di ViT-Small sul dataset ImageNet-1K, con *Knowledge Distillation* attiva. In alto è visibile l'andamento della accuratezza (top-1) pre e post *fine tuning*, al centro il numero di *subnet* (nodi) esplorati per ogni iterazione, in basso la compressione progressiva in termini di numero di parametri.

Osservando la Figura 6.5 è possibile notare che, se le prime iterazioni hanno esplorato un numero di nodi simile a quanto osservato in Figura 6.3, a causa della maggiore difficoltà del dataset utilizzato per questo test, le iterazioni successive hanno esplorato un numero di *subnet* molto superiore, superando in certi casi i 300 nodi. Questo comportamento conferma la **flessibilità** del framework proposto, che ha eseguito delle fasi di ricerca molto più approfondite rispetto a quanto visto nei risultati precedenti, proprio a causa della natura del problema, in questo caso intrinsecamente più complesso.

Per quanto concerne il grafico in alto (Figura 6.5) è necessario precisare che le *accuracy* osservate risultano nettamente maggiori di quella finale ottenuta (Tabella 6.2), poiché queste sono misurate a partire dal *validation set*. Questo set di dati è stato ricavato dal *train set* durante la prima fase di *fine tuning* iniziale (che ha interessato esclusivamente il classificatore finale a valle del ViT), di conseguenza i campioni sono stati interessati dalla fase di *pre-training* del modello. Osservando l'andamento dell'accuratezza è possibile affermare che questo fattore non ha intaccato in modo significativo il processo, data la forte similitudine del comportamento con quanto visto in Figura 6.3. In particolare, sebbene sia possibile notare una decrescita più mar-

cata del valore di accuratezza, rispetto a quanto osservato nel test sul CIFAR100, questo risulta fisiologico considerando che questi campioni, precedentemente utilizzati in fase di *pre-training*, vengono ora utilizzati esclusivamente per la valutazione. Il modello, durante il processo di compressione, dimostra di perdere la sua conoscenza mnemonica di questi campioni, mantenendo però una robusta capacità di generalizzazione, come confermato dalle metriche osservate sul *test set*.

Infine, per quanto riguarda il terzo grafico, come nel test precedente, notiamo un decremento del numero di parametri graduale e lineare, che non raggiunge però lo stesso grado di compressione visto in Tabella 6.1. Questo risultato è comprensibile considerando la maggiore complessità del task e del dataset impiegato, che ha reso più complicato il taglio dei parametri del modello.



Figura 6.6: Illustrazione delle percentuali di dimensioni tagliate nei vari blocchi *Encoder* del ViT-Small, nel caso del test sul dataset ImageNet-1K con l'utilizzo di *Knowledge Distillation*.

Per quanto riguarda la Figura 6.6 è possibile osservare i pattern di taglio prodotti dal framework di compressione. Questi grafici mostrano dei trend molto simili a quanto visto per i test sul dataset CIFAR100. In particolare i blocchi MLP vengono compressi solo nei *layer* più profondi, al contrario del meccanismo di attenzione, che viene prevalentemente ridotto nei *layer* iniziali, più vicini all'input. Nonostante le differenti percentuali di compressione, dovute alle

differenze tra i dataset utilizzati, sono visibili delle sostanziali somiglianze con quanto osservato nella Figura 6.4, suggerendo l'esistenza di **ragioni architettoniche** dietro alle scelte effettuate dal framework. In effetti, quanto osservato in questi test trova riscontro e giustificazione in diversi studi sulle dinamiche interne ai modelli Transformer.

Per quanto riguarda i moduli di *attenzione*, Michel et al. [48] dimostrano l'estrema ridondanza del meccanismo di *Multi-Head Self-Attention*, provando che la rimozione di gran parte delle teste, o addirittura la riduzione di determinati layer a una singola testa, non comporta un degrado rilevante delle prestazioni. Il framework di compressione sfrutta questa tolleranza, mostrando un pattern ricorrente in cui i tagli al meccanismo di attenzione si concentrano prevalentemente sui layer iniziali e finali. Questo risulta coerente con le dinamiche di elaborazione dell'informazione nei Vision Transformer descritte da Tian et al. [65], che hanno individuato una *struttura ad "U"* del meccanismo di elaborazione delle informazioni interno al ViT. In particolare, attraverso un'analisi delle attivazioni di ogni blocco *encoder*, vengono identificati i blocchi centrali come i responsabili di elaborare le feature a basso livello che sono state ripulite dal rumore e dalla ridondanza nei blocchi precedenti. Ai layer finali viene invece attribuito il ruolo di espandere lo spazio delle features per individuare delle rappresentazioni di alto livello, introducendo però al contempo della ridondanza. Questa interpretazione spiega e trova sostegno nei tagli attuati dal framework di compressione, che evita il più possibile i blocchi centrali poiché essi costituiscono il **nucleo semantico** della rete.

Analizzando la struttura della componente **MLP** nei *Transformer*, Geva et al. [21] evidenziano che questi moduli operano come memorie chiave-valore, lavorando in sinergia con il flusso residuale della rete per raffinare le rappresentazioni interne al modello. Con l'avanzare della profondità, l'informazione accumulata all'interno del flusso residuale diventa progressivamente più ricca e quindi in grado di guidare la predizione. In questo contesto, il MLP assume un ruolo sempre più marginale con l'aumentare della profondità, spiegando così i tagli marcati nei blocchi finali del *Vision Transformer*.

6.6 Test con DeiT-Small

I seguenti test sono eseguiti sul modello *deit_small_patch16_224.fb_in1k* della libreria **timm**. Trattasi di un modello addestrato sul dataset ImageNet-1K secondo la configurazione proposta da Touvron et al. [66].

6.6.1 Test su dataset CIFAR100

Per i seguenti risultati è stato utilizzato il *DeiT-Small* che, con l'aggiunta della testa di classificazione da 100 unità, ha raggiunto **21.7** Mln di parametri. Il modello è stato successivamente sottoposto alla fase di *fine tuning* iniziale sul dataset in oggetto secondo le modalità illustrate nella Sezione 6.3. Per quanto riguarda il framework, tutti i test sono stati effettuati con **25 iterazioni** di compressione. Ad ogni iterazione, la **profondità** dell'albero di ricerca è stata limitata a **8**. Viene inoltre utilizzata una **soglia di tolleranza** della fase di *Bound* pari a **0.01**. Infine, la **funzione obiettivo** utilizza un valore di λ pari a **1.25**, attribuito al contributo dei parametri.

Model	Params (M) ↓	Top-1 (%) ↑	FLOPs (G) ↓	Throughput (imgs/s) ↑	Latency (Δ%) ↓
DeiT-Small	21.70	87.51%	4.60	1134.7	112.8
Pruned	15.95 (-26.5%)	87.09% (-0.42%)	3.36 (-27.00%)	1443.6 (+27.22%)	88.7 (-21.40%)
Pruned + KD	16.44 (-24.3%)	88.34% (+0.83%)	3.49 (-24.20%)	1347.3 (+18.74%)	95.0 (-15.78%)

Tabella 6.3: Nella tabella sono presentati i risultati comparativi utilizzando il *DeiT-Small* e il dataset *CIFAR100*, in particolare vengono evidenziati i valori delle metriche e le variazioni rispetto al modello *Baseline*.

Per quanto riguarda i risultati ottenuti sul dataset CIFAR100, è possibile notare una minore accuratezza di DeiT rispetto a quanto osservato con ViT. Questo è dovuto ai modelli utilizzati, infatti i ViT (Sezione 6.5) sono stati ottenuti attraverso un addestramento che includeva anche il dataset ImageNet-21K, invece i modelli DeiT sono ottenuti esclusivamente tramite addestramento sul dataset ImageNet-1K. Questo implica che il primo è più robusto del secondo, poiché i parametri sono stati ottenuti con una mole molto maggiore di dati di *training* (13 Mln. contro 1.3 Mln). La conseguenza di ciò è proprio una *accuracy* minore a parità di dataset utilizzato (CIFAR100), fin dai modelli *baseline* non compressi.

Un dato interessante è che, a differenza di quanto osservato in Tabella 6.1, il declino di accuratezza è risultato minore, a fronte di un fattore di compressione molto simile. In particolare, senza l'ausilio della *distillation* (KD), il framework è riuscito a ridurre di oltre il **26%** il numero di parametri con una perdita di *accuracy* trascurabile (0.4%). Ma il risultato di maggiore interesse è sicuramente il secondo, ottenuto nelle stesse condizioni del precedente e con l'aggiunta della KD, il quale ha portato a quasi il **25%** di compressione e un **guadagno** significativo di

accuracy (0.8%). Quanto osservato conferma quindi come l'utilizzo della distillazione permetta di mantenere più elevate le prestazioni del modello.

Confrontando questi risultati con quelli ottenuti dal modello ViT-Small, una ulteriore osservazione è che in questo caso i tagli di parametri si traducono in un maggiore incremento nel **throughput** e decremento di **latenza**. La spiegazione può essere individuata nei trend di compressione alle varie strutture del modello DeiT, che verranno approfondite maggiormente nella Sezione 6.6.3. In questi test, infatti, è stata osservata una maggiore tendenza al taglio di interi blocchi *Multi-Head Self Attention* all'interno dei *layer*, fenomeno che non si osservava nel caso del modello ViT. Intuitivamente, il taglio di un intero blocco di MHSA produce un incremento in velocità molto importante e maggiore rispetto al taglio di una singola testa di attenzione, o anche di una grossa percentuale di MLP in un *layer*, portando così ai risultati osservati in questo test.

Infine, dalla Tabella 6.3 è possibile notare una discrepanza tra gli incrementi in velocità dei due modelli risultanti. Il modello ottenuto utilizzando la *distillation*, nonostante una simile compressione, ha un'incremento in velocità di molto inferiore all'altro. La spiegazione risiede nel numero di MHSA interamente eliminate, che nel caso del modello senza KD è pari a 2, mentre nell'altro si limita ad 1. Si noti inoltre che questa discrepanza si registra a causa dei fattori di compressione molto simili, e non si osserverebbe nel caso di modelli con una maggiore differenza in termini di numero di parametri.

6.6.2 Test su dataset ImageNet-1K

In questo test è stato utilizzato il *DeiT-Small* che, con l'aggiunta della testa di classificazione da 1000 unità, ha raggiunto **22** Mln di parametri. Il modello è stato successivamente sottoposto alla fase di *fine tuning* iniziale sul dataset in oggetto secondo le modalità illustrate nella Sezione 6.3. Per quanto riguarda il framework, tutti i test sono stati effettuati con **15 iterazioni** di compressione. Ad ogni iterazione, la **profondità** dell'albero di ricerca è stata limitata a **6**. Viene inoltre utilizzata una **soglia di tolleranza** della fase di *Bound* pari a **0.01**. Infine, la **funzione obiettivo** utilizza un valore di λ pari a **1.0**, attribuito al contributo dei parametri. In questo caso, inoltre, è stato adottato un *learning rate* per il *recovery fine tuning* più elevato, pari a 2×10^{-5} .

Model	Params (M) ↓	Top-1 (%) ↑	FLOPs (G) ↓	Throughput (imgs/s) ↑	Latency (Δ%) ↓
DeiT-Small	22.05	79.85%	4.60	1137.0	112.6
Pruned	18.31 (-17.0%)	77.16% (-2.69%)	3.81 (-17.14%)	1279.8 (+12.56%)	100.0 (-11.19%)
Pruned+KD	16.49 (-25.2%)	79.15% (-0.70%)	3.46 (-24.85%)	1392.6 (+22.48%)	91.9 (-18.36%)

Tabella 6.4: Nella tabella sono presentati i risultati comparativi utilizzando il *DeiT-Small* e il dataset *ImageNet-1K*, in particolare vengono evidenziati i valori delle metriche e le variazioni rispetto al modello *Baseline*.

I risultati ottenuti su ImageNet-1K, mostrati in Tabella 6.4, confermano quanto già osservato nei test con il modello ViT-Small (Tabella 6.2). Anche in questo caso, il framework di compressione ha beneficiato dell'utilizzo della *Knowledge Distillation*, che ha permesso di tagliare un maggior numero di parametri, e anche di mantenere molto più elevata l'accuracy.

In particolare, senza l'ausilio della KD è stato possibile comprimere il modello solo del 17% con una perdita di accuratezza pari al 2.7%, invece con l'aggiunta di questa si è riusciti a comprimere di oltre il 25% con una perdita di accuratezza molto inferiore, equivalente allo 0.70%.

Le metriche di velocità rispecchiano quanto descritto sopra, con un incremento del throughput superiore al 22% e una riduzione della latenza superiore al 18%. Questi risultati, sono superiori a quelli ottenuti con il modello ViT-Small, nonostante una compressione simile, e questo è dovuto alla maggiore tendenza al taglio di interi blocchi di MHSA osservata nei test con DeiT-Small, che come già descritto nella sezione precedente, produce un incremento in velocità molto importante.

Infine, una considerazione interessante è che, a fronte di un numero equivalente di iterazioni e del parametro λ , il framework è riuscito a comprimere di più il modello DeiT-Small rispetto al ViT-Small.

Questo risultato potrebbe essere giustificato dalla scelta di utilizzare una soglia di tolleranza più elevata (0.01 contro 0.005), che ha permesso al framework di esplorare maggiormente, accettando anche delle azioni di *pruning* più aggressive, che altrimenti sarebbero state scartate sulla base dell'*accuracy*. Questa scelta è stata dettata dalla minore robustezza del modello DeiT-Small, che non ha beneficiato di un addestramento su un dataset di grandi dimensioni come ImageNet-21K, a differenza del modello ViT-Small, e quindi necessita di una maggiore tolleranza da parte del framework per comprimere in maniera più efficace il modello.

6.6.3 Analisi dell'algoritmo di ricerca con il modello DeiT-Small

Per quanto riguarda i risultati ottenuti con il modello DeiT-Small, è possibile notare una somiglianza importante con quanto osservato nei test con ViT-Small.



Figura 6.7: Illustrazione del processo di ricerca che ha portato alla compressione di DeiT-Small sul dataset CIFAR100, con *Knowledge Distillation* attiva. In alto è visibile l'andamento della accuratezza (top-1) pre e post *fine tuning*, al centro il numero di *subnet* (nodi) esplorati per ogni iterazione, in basso la compressione progressiva in termini di numero di parametri.

Analizzando inizialmente i test effettuati sul dataset CIFAR100 (Figura 6.7), è possibile notare un andamento simile a quanto osservato in Figura 6.3 per il modello ViT-Small. In particolare, si nota un incremento ancora più marcato dell'accuratezza nelle prime iterazioni, che raggiunge un picco di quasi il 2% superiore al modello di partenza. Questo dimostra e conferma l'effetto di regolarizzazione osservato precedentemente.

Per quanto riguarda il numero di nodi esplorati, in questo caso si osserva una media di poco inferiore ma comunque comparabile a quanto ottenuto nei test con ViT-Small. Infine, la compressione si dimostra anche in questo caso graduale e lineare, anche se distribuita su un numero maggiore di iterazioni, che in questo caso è pari a 25, contro le 15 dei test con ViT-Small.

La scelta di adottare un numero maggiore di iterazioni è stata dettata da quanto già affermato nella sezione 6.6.1. In particolare, la motivazione risiede nella minor robustezza del modello

DeiT-Small, che non ha beneficiato di un addestramento su un dataset di grandi dimensioni come ImageNet-21K, a differenza del modello ViT-Small. Di conseguenza, è stato necessario fornire al framework un numero maggiore di iterazioni per comprimere in maniera più graduale il modello.

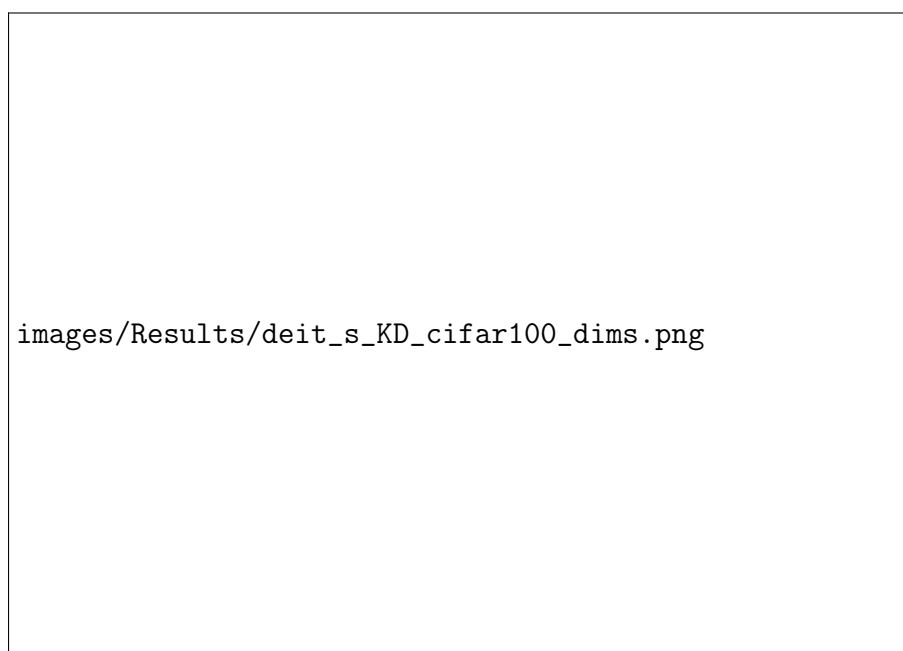


Figura 6.8: Illustrazione delle percentuali di dimensioni tagliate nei blocchi *Encoder* del DeiT-Small, nel caso del test sul dataset CIFAR100 con l'utilizzo di *Knowledge Distillation*.

Per quanto concerne invece la struttura dei tagli, osservando la figura 6.8 è possibile rinforzare ulteriormente le osservazioni precedentemente effettuate (Figura 6.4). Risulta infatti possibile notare in modo ancora più evidente la struttura ad "U" del meccanismo di attenzione, con i tagli più marcati nei layer iniziali e finali, e una quasi totale conservazione dei blocchi centrali.

Anche dal punto di vista del MLP si può osservare lo stesso comportamento visto nei test precedenti, dimostrando così la ridondanza di questi moduli nei layer più profondi.

Considerazioni simili possono essere fatte per i test sul dataset ImageNet-1K, che presentano anche in questo caso forti somiglianze con quanto osservato nei test con ViT-Small sullo stesso dataset.

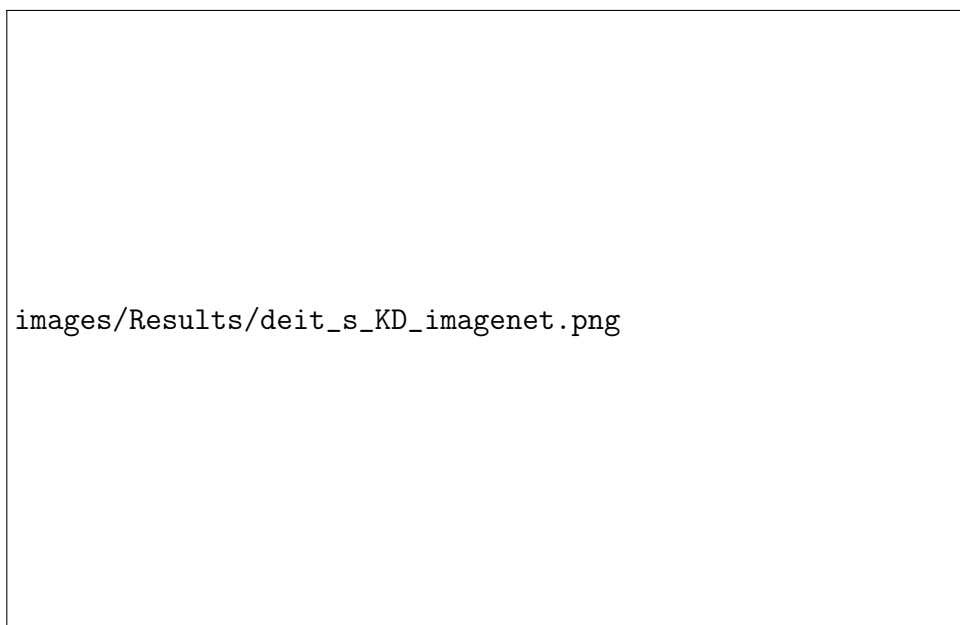


Figura 6.9: Illustrazione del processo di compressione che ha portato alla compressione di DeiT-Small sul dataset ImageNet-1K, con *Knowledge Distillation* attiva. In alto è visibile l’andamento della accuratezza (top-1) pre e post *fine tuning*, al centro il numero di *subnet* (nodi) esplorati per ogni iterazione, in basso la compressione progressiva in termini di numero di parametri.

In Figura 6.9 è possibile visionare l’andamento del processo di compressione. Notare che, per quanto riguarda l’accuratezza, questa è mostrata sul *validation set*, che viene prodotto secondo le stesse modalità viste nei test con ViT-Small. Questo porta all’osservazione di valori di accuratezza superiori e distanti da quanto osservato sul *test set*, ma che comunque mostrano un andamento simile a quanto osservato nei test con il modello precedente (Figura 6.5).

Il numero di nodi esplorati è risultato in questo caso inferiore in media a quanto osservato nei test con ViT-Small, ma comunque con picchi molto elevati, che superano nettamente i valori sul dataset CIFAR100. Questo dimostra e conferma quanto osservato nella sezione 6.5.3, ovvero che il *framework* di compressione è in grado di adattarsi dinamicamente alla complessità del task sottoposto.

La scelta di differenziare il numero di iterazioni, 15 per ImageNet-1K e 25 per CIFAR100, riflette la differente stabilità delle rappresentazioni latenti nei due domini. Nel caso di ImageNet-1K, essendo il dataset di pre-addestramento originario del modello, DeiT-Small opera su un dominio di feature già consolidato. Questa stabilità consente al framework di comprimere il modello in modo più rapido. Al contrario, sul dataset CIFAR100, il modello si trova in uno

situazione di *transfer learning*, nel quale le feature originali sono state riadattate tramite un *fine-tuning* iniziale su un task differente, portando ad una minore robustezza. Tale fenomeno non si registrava nel caso del ViT-Small, poiché era stato addestrato su un dataset di grandi dimensioni (ImageNet-21K) e quindi aveva già sviluppato delle rappresentazioni latenti più robuste e generali, che hanno permesso al framework di comprimere il modello in modo più rapido anche sul dataset CIFAR100.



Figura 6.10: Illustrazione delle percentuali di dimensioni tagliate nei blocchi *Encoder* del DeiT-Small, nel caso del test sul dataset ImageNet-1K con l'utilizzo di *Knowledge Distillation*.

In merito alla distribuzione dei tagli (Figura 6.10) è possibile riconfermare quanto osservato nei test precedenti, in quanto vengono osservati gli stessi pattern di compressione, sia per quanto riguarda il meccanismo di attenzione, sia per quanto riguarda i moduli MLP.

Avendo osservato queste strutture su modelli e dataset differenti, è possibile confermare che siano presenti delle motivazioni intrinseche all'architettura dei modelli Transformer. In particolare, le osservazioni di Tian et al. [65] sulla struttura ad "U" del meccanismo di attenzione, e le osservazioni di Geva et al. [21] sulla marginalità dei moduli MLP nei layer più profondi, trovano conferma nei pattern di compressione individuati in questi test, suggerendo che queste siano delle proprietà intrinseche di questa architettura, e non semplicemente dei risultati casuali o dovuti alla natura dei dataset utilizzati.

Queste osservazioni forniscono anche un importante spunto per futuri lavori, che potrebbero indagare più a fondo queste dinamiche interne ai modelli Transformer, in particolare valutando altri task sia visivi che testuali (ad esempio tramite modelli BERT [17]). In tal modo, sarebbe possibile verificare la presenza delle stesse strutture nei tagli, e di conseguenza sviluppare una nuova famiglia di modelli Transformer maggiormente efficienti, con architetture basate sulle osservazioni in oggetto.

6.7 Ablazione

Al fine di verificare la bontà del metodo sono stati effettuati dei test per isolare il contributo delle due principali componenti del *framework*, l'algoritmo di ricerca e la metrica di importanza.

I seguenti risultati sono stati ottenuti sul dataset CIFAR100 e sul modello ViT-Small, utilizzando la stessa configurazione riportata nella sezione 6.5 e nella sezione 6.3. I test sono stati eseguiti senza l'utilizzo della *Knowledge Distillation* per isolare al meglio l'effetto delle singole componenti del framework.

6.7.1 Metrica di Importanza e Algoritmo di Ricerca

Model	Params (M) ↓	Top-1 (%) ↑	FLOPs (G) ↓	Throughput (imgs/s) ↑	Latency (Δ%) ↓
ViT-Small	21.70	90.42%	4.60	1137.5	112.5
ViT-S Pruned	17.59 (-19.0%)	88.92% (-1.50%)	3.74 (-18.67%)	1308.6 (+15.04%)	97.8 (-13.06%)
Azione Casuale	12.28 (-43.4%)	81.17% (-9.25%)	2.64 (-42.66%)	1679.9 (+47.68%)	76.2 (-32.27%)
Metrica Disattivata	19.38 (-10.7%)	89.07% (-1.35%)	4.09 (-11.09%)	1147.9 (+00.90%)	111.5 (-00.90%)

Tabella 6.5: Nella tabella sono presentati i risultati dell'applicazione del *framework* di compressione nella sua interezza e con le componenti disattivate, sul dataset *CIFAR100* e sul modello *ViT-Small*.

Come è possibile notare dalla Tabella 6.5, entrambi i test di ablazione hanno dato risultati peggiori rispetto a quanto ottenuto con il *framework* di compressione nella sua interezza. In particolare nel test con la **ricerca disattivata**, per ogni iterazione, considerando d la massima profondità della ricerca, vengono eseguite casualmente d azioni sul modello. In questo caso il

framework si è trovato costretto ad eseguire dei tagli su dimensioni importanti, e quindi poco conservativi dal punto di vista delle performance. La metrica di importanza, mantenuta attiva nel test, non è riuscita a limitare i danni portati dai tagli molto aggressivi e indiscriminati, portando così ad una grande diminuzione di accuratezza. Questo primo test ha dimostrato che l'algoritmo di ricerca *BnB* ha un grande contributo nel selezionare accuratamente le azioni, da un lato limitando il livello di compressione a parità di iterazioni di *pruning*, dall'altro consentendo di mantenere dei livelli di prestazioni molto vicini al modello originale.

Nel secondo test invece, la **metrica di importanza** è stata disattivata, di conseguenza per ogni azione selezionata dall'algoritmo di ricerca, viene prima selezionato casualmente il blocco *Encoder* su cui agire, e successivamente vengono campionate casualmente le dimensioni da tagliare delle specifiche componenti (QK, MLP, ...) di quel blocco (tranne per la dimensione di *Embedding*, per il quale non è necessario selezionare prima il blocco su cui agire). I risultati indicano chiaramente una difficoltà dell'algoritmo di ricerca nell'esplorare lo spazio degli stati, causata proprio dalla mancanza dell'azione della metrica di importanza. Selezionando in maniera casuale le dimensioni da tagliare, la maggioranza degli stati (*subnet*) che vengono esplorati risultano poco promettenti e peggiorano il valore della funzione obiettivo, con un conseguente aumento del numero di nodi esplorati, ed una grande fatica a comprimere il modello. Durante i test che fanno uso della metrica di importanza, si assiste invece ad una progressione stabile e rapida del valore della funzione obiettivo, che si dimostra quindi molto efficace nel guidare le azioni di *pruning* e la ricerca.

In questo secondo test è importante notare come le prestazioni del modello si sono mantenute elevate, a conferma del grande lavoro di filtraggio eseguito dal processo di ricerca, che evita di selezionare quegli stati la cui selezione casuale dei tagli ha danneggiato gravemente il modello.

6.7.2 Il parametro λ della funzione obiettivo

Considerando la funzione obiettivo definita nella Sezione 4.1 è possibile notare la presenza di un parametro che bilancia il peso attribuito al fattore di compressione. Agendo su tale parametro è possibile modificare il comportamento dell'algoritmo di ricerca. In particolare all'aumentare del valore di λ verranno considerati più facilmente stati la cui accuratezza è minore ma presentano una maggiore riduzione del numero di parametri. Per dimostrare il contributo di questo parametro sono stati eseguiti dei test sul modello ViT-Small, utilizzando

il dataset CIFAR100 (la configurazione è la medesima rispetto ai test 6.5, senza l’ausilio della *Knowledge Distillation*) i cui risultati sono mostrati in Tabella 6.6.

Modello	Parametri (M)	Top-1 Acc. (%)	GFLOPs	Throughput	Latency
ViT-Small	21.70	90.42%	4.60	1137.5	112.5
ViT-S Pruned $\lambda = 0.5$	18.89 (-13.0%)	89.84% (-0.58%)	4.00 (-13.13%)	1256.4 (+10.45%)	101.9 (-9.42%)
ViT-S Pruned $\lambda = 1.0$	17.59 (-19.0%)	88.92% (-1.50%)	3.74 (-18.67%)	1308.6 (+15.04%)	97.8 (-13.06%)
ViT-S Pruned $\lambda = 1.5$	15.86 (-26.9%)	86.82% (-3.60%)	3.43 (-25.47%)	1406.3 (+23.63%)	91.0 (-19.11%)

Tabella 6.6: Nella tabella sono presentati i risultati dell’applicazione del *framework* di compressione al variare del valore di λ , ovvero il peso della componente di compressione nella funzione obiettivo, sul dataset *CIFAR100* e sul modello *ViT-Small*.

Quanto osservato in Tabella 6.6 risulta in accordo con le aspettative. I tre valori testati producono modelli nettamente differenti:

- Utilizzando $\lambda = 0.5$ l’algoritmo di ricerca preserva efficacemente l’accuratezza, ma questo porta ad una compressione ridotta;
- Con $\lambda = 1.0$ è possibile notare una situazione di equilibrio: il taglio di parametri è significativo, con una perdita di *accuracy* maggiore ma tollerabile;
- Infine, adottando $\lambda = 1.5$, si ottiene una compressione importante (oltre il 25% dei parametri) ma con perdita di prestazioni molto più accentuata.

Dai risultati ottenuti, è possibile concludere che il parametro λ può essere configurato sulla base degli obiettivi da raggiungere. Ad esempio, nel caso di un *deploy* su dispositivi con capacità computazionali molto ridotte, è possibile incrementarlo al fine di ottenere un modello decisamente più compatto, accettando però un compromesso in termini di *performance* predittive.

6.8 Comparazione con lo stato dell’arte su ImageNet-1K

Al fine di confrontare i risultati del *framework* proposto con gli approcci presenti in letteratura, è stato ulteriormente compresso il modello DeiT-Small mostrato nella Tabella 6.4, nello specifico la versione ottimizzata tramite *Knowledge Distillation*.

Al modello sono state applicate tre ulteriori iterazioni di compressione, mantenendo i medesimi dettagli implementativi del test originale ma introducendo due variazioni negli iperparametri: il valore di λ è stato incrementato a 1.25 e il numero di epoche di *fine-tuning* per singola iterazione è stato fissato a 12 (rispetto alle 10 originarie). Questa configurazione si è resa necessaria per spingere l’architettura verso una maggiore compressione, rendendo il metodo direttamente confrontabile con i concorrenti e garantendo l’equità della comparazione.

Modello	Parametri (M)		Top-1 Acc.		GFLOPs		Throughput	Latency
	Valore ($\downarrow$)	$\Delta\%$	Valore ($\uparrow$)	$\Delta\%$	Valore ($\downarrow$)	$\Delta\%$	$\Delta\%$ ($\uparrow$)	$\Delta\%$ ($\downarrow$)
DEiT-S	22.00	—	79.80%	—	4.6	—	—	—
GOHSP [76]	14.40	-35.0%	79.98%	+0.18%	NA	-35.0%	NA	-24.6%
S2ViTE [9]	14.60	-33.6%	79.22%	-0.58%	NA	-31.6%	NA	-22.6%
WDP [77]	15.00	-31.8%	78.55%	-1.25%	3.1	-32.6%	+10.7%	-11.2%
X-Pruner [78]	NA	NA	78.93%	-0.87%	2.4	-47.9%	NA	NA
CAIT [70]	18.40	-16.4%	80.20%	+0.40%	2.1	-54.4%	+90.0%	NA
Ours	15.40	-30.0%	79.26%	-0.54%	3.2	-30.4%	+27.4%	-21.5%

Tabella 6.7: Tabella comparativa delle prestazioni su ImageNet-1K utilizzando DeiT-Small come architettura di partenza. Vengono riportati i valori assoluti e le variazioni percentuali ($\Delta\%$) rispetto alla baseline per i parametri, l’accuratezza e la complessità computazionale, limitando l’analisi hardware al solo delta percentuale di Throughput e Latency.

Come si evince dai dati riportati in Tabella 6.7, il metodo proposto si colloca in una posizione competitiva rispetto allo stato dell’arte, mostrando un ottimo bilanciamento tra compressione e mantenimento delle prestazioni in accuratezza.

Il modello ottenuto ha sorprendentemente raggiunto un livello di compressione maggiore (pari al 30% dei parametri) ma aumentando il valore di *accuracy*. A fronte di una riduzione netta del 30.0% del volume dei parametri e del 30.4% dei GFLOPs complessivi, il modello compresso dal framework proposto mostra un’accuratezza Top-1 del 79.26% (a differenza di 79.15% osservato nella Tabella 6.4). Questo valore ha subito un decremento di soli 0.54 punti percentuali rispetto alla baseline (79.80%).

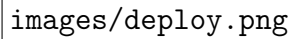
Questo risultato trova una spiegazione nell’aumento del numero di epoche di *fine tuning* adottato nelle 3 iterazioni aggiuntive, che hanno quindi consentito al modello di recuperare maggiormente le sue prestazioni originali. Si noti che, al prezzo di un maggiore tempo di esecuzione del processo di compressione, sarebbe possibile raggiungere risultati ancora superiori in termini di accuratezza, aumentando ulteriormente il numero di epoche.

Il confronto analitico con gli altri approcci è stato effettuato considerando i risultati riportati dagli autori nei rispettivi articoli, limitando l'analisi a quei metodi che hanno adottato il modello DeiT-Small come base di partenza e ImageNet-1K come dataset di test, al fine di garantire un confronto equo e diretto.

Analizzando le prestazioni del framework sviluppato rispetto ai metodi allo stato dell'arte, emergono risultati competitivi in diversi scenari. Confrontato con WDP [77] e X-Pruner [78], il nostro approccio si dimostra superiore sul piano dell'accuratezza: a parità di compressione, WDP accusa un degrado importante della Top-1 Acc. (-1.25%), mentre X-Pruner si arresta al 78.93% (nonostante non sia stato reso noto il numero di parametri per quest'ultimo). Sul fronte dell'efficienza hardware, il nostro metodo surclassa nettamente WDP, garantendo un incremento del *throughput* del $+27.4\%$ (contro $+10.7\%$) e una riduzione della latenza del -21.5% (contro -11.2%). Il raffronto con S2ViTE [9] evidenzia invece prestazioni molto simili, caratterizzate da un'accuratezza equivalente a fronte di una compressione leggermente minore del nostro metodo, che porta S2ViTE a mantenere un marginale vantaggio sulla riduzione della latenza (-22.6% contro il nostro -21.5%). Infine, approcci più recenti come GOHSP [76] e CAIT [70] si dimostrano superiori incrementando l'accuratezza rispetto alla baseline (rispettivamente del $+0.18\%$ e $+0.40\%$). CAIT, infatti, limita la riduzione dei parametri a un modesto -16.4% , compensando con un notevole aumento del *throughput* dovuto all'adozione di tecniche di *token merging* da noi non adottate. GOHSP, d'altro canto, ottiene un miglioramento della latenza comparabile ma lievemente superiore al nostro (-24.6%), risultato però giustificato da un livello di compressione più elevato (-35.0%).

6.9 Deploy su mobile

Al fine di testare i benefici del framework di compressione proposto in uno scenario applicativo reale, è stato eseguito il *deploy* locale su dispositivi mobili basati su sistema operativo Android. Si è scelto di adottare l'ecosistema **ONNX Runtime** per la gestione dell'esportazione dei modelli e l'esecuzione dell'inferenza *on-device*, sfruttando l'elevata compatibilità del framework con i diversi acceleratori hardware presenti sul mercato.



images/deploy.png

Figura 6.11: Processo di deploy adottato per il test di inferenza *on-device*.

Il processo di *deploy*, illustrato in Figura 6.11, ha interessato sia il modello proposto (Tabella 6.7) sia la controparte DeiT-S non compressa, utilizzate per confrontare le prestazioni in termini di latenza. I modelli sono stati inizialmente esportati nello standard ONNX, generando i rispettivi grafi computazionali, e successivamente convertiti ad una precisione di 16 bit dei parametri (*Float16*).

La riduzione della precisione (quantizzazione) dei pesi è una pratica consolidata nelle *pipeline* di *deploy* per dispositivi *edge*, in quanto consente un importante risparmio di memoria e un incremento del *throughput*. Per i modelli appartenenti alla famiglia dei Vision Transformer, tale scelta rappresenta un compromesso ottimale: come dimostrato da Liu et al. [43], i modelli ViT sono notoriamente sensibili alle tecniche di quantizzazione *post-training* tradizionali (come il passaggio a INT8), le quali introducono grossi cali (fino al 10%) delle metriche di accuratezza in assenza di un successivo processo di *fine-tuning*. Adottando invece una precisione a 16 bit, è possibile mantenere un'accuratezza invariata rispetto al modello originale, ma con un importante risparmio in termini di memoria e un incremento del *throughput* durante l'inferenza.



Figura 6.12: Esempio di inferenza su una immagine, è visibile la classificazione, con la rispettiva *confidence*, oltre alle latenze registrate con il corrispondente incremento percentuale.

Una volta ottenuti i modelli esportati a 16 bit, questi sono stati integrati in un'applicazione Android nativa, sviluppata appositamente in linguaggio **Kotlin** per consentire l'esecuzione dell'inferenza *on-device* e la misurazione della latenza. A tale scopo è stato utilizzato **ONNX Runtime Android SDK**, che si pone come intermediario tra i modelli esportati e le API di sistema per l'esecuzione dell'inferenza sui vari componenti hardware.

Al fine di sfruttare l'accelerazione hardware dei dispositivi mobili moderni, sono state adottate le **Android Neural Networks API (NNAPI)**. Le NNAPI agiscono come uno strato di controllo che direziona le operazioni matematiche del Vision Transformer (come i prodotti tensoriali) verso gli acceleratori dedicati presenti sul system-on-chip (SoC) del dispositivo, quali la NPU (Neural Processing Unit) o la GPU, ove supportati. Qualora l'hardware del dispositivo non disponga di acceleratori compatibili con le operazioni a 16 bit, l'infrastruttura esegue un *fallback* automatico sulla CPU.

Modello	Latenza Media (ms)	Std (ms)	Speedup (%)
DeiT-S (Originale)	296,6	16,2	Baseline
DeiT-S (Compresso)	222,1	7,2	+25,1%

Tabella 6.8: Analisi della latenza in fase di inferenza *on-device*, sulla base di 12 test.

Come evidenziato dai dati statistici raccolti in Tabella 6.8, i tempi di inferenza presentano una variabilità importante. Tale fenomeno è comune nei sistemi operativi come Android, dove lo *scheduler* alloca dinamicamente le risorse computazionali in base al carico istantaneo del sistema. L'analisi dei risultati mostra che il processo di compressione, eseguito dal *framework* proposto, permette di ottenere uno *speedup* medio del 25,1%, riducendo la latenza in media da 296,6 ms a 222,1 ms. Oltre all'incremento prestazionale, si osserva una riduzione significativa della deviazione standard, che passa da 16,2 ms a 7,2 ms. Questo decremento evidenzia come l'ottimizzazione e il minor transito di dati in memoria rendano l'esecuzione del modello compresso significativamente più stabile e meno soggetta alle fluttuazioni di carico del sistema operativo rispetto alla controparte non compressa.

Conclusioni

In questo lavoro è stato progettato, implementato e validato un framework automatico per la compressione e l'ottimizzazione di modelli *Vision Transformer* (ViT), con l'obiettivo di ridurre l'impatto computazionale e renderne possibile l'esecuzione efficiente su architetture hardware a risorse limitate, come i dispositivi mobili ed *edge*. La strategia proposta ha integrato una tecnica di *pruning* strutturato con gli algoritmi di ottimizzazione, unendo anche la *Knowledge Distillation*.

L'analisi sperimentale, condotta su benchmark standard quali *CIFAR-100* e *ImageNet-1K*, ha confermato l'efficacia dell'approccio e la sua competitività con lo stato dell'arte. I risultati quantitativi hanno evidenziato che sia possibile ottenere una riduzione di circa un terzo dei parametri e dei GFLOPs, con una perdita di accuratezza trascurabile, che corrisponde però ad un incremento significativo del throughput. Sono stati inoltre condotti dei test di inferenza su dispositivi mobili, dimostrando l'impatto reale della compressione sulle latenze registrato a seguito della procedura di *deploy*.

Gli studi di ablazione hanno validato l'architettura interna del framework, dimostrando che sia la metrica di importanza basata sul Fisher Score, sia la componente di ricerca contribuiscono ad evitare il decadimento delle prestazioni durante il processo di compressione.

Nell'analisi dei risultati ottenuti dai test sono emersi alcuni pattern strutturali interessanti che portano ad alcune considerazioni sulla natura dei modelli ViT e sul loro funzionamento. Da quanto osservato emerge una struttura ad "U", già teorizzata e studiata nella letteratura recente, secondo il quale i layer centrali contengono le rappresentazioni ad alto livello, i concetti semantici, e di conseguenza sono più sensibili al pruning.

6.10 Limitazioni e sviluppi futuri

Di seguito sono elencate alcune limitazioni del lavoro svolto e possibili sviluppi futuri che potrebbero essere intrapresi per migliorare ulteriormente il framework di compressione proposto:

- **Costo computazionale della compressione:** Il framework di compressione presentato, nonostante l'efficacia dimostrata, richiede un costo computazionale considerevole. Se in particolare si vuole recuperare la massima *accuracy* possibile, è necessario utilizzare un numero elevato di epoche di *fine tuning*, che, se unito ad un elevato numero di iterazioni di ricerca, può portare a costi paragonabili a quelli di un addestramento completo del modello originale. Nonostante ciò, è importante sottolineare che il processo di compressione rappresenta un investimento da eseguire in una singola occasione, che presenta benefici tangibili che potrebbero comunque risultare vantaggiosi in scenari di produzione.
- **Mancanza di test sui modelli di grandi dimensioni:** A causa dell'elevato costo computazionale prima esposto, i test sperimentali sono stati condotti su modelli di dimensioni ridotte, che rappresentano comunque un'ottima sfida per la compressione, tuttavia non è stato possibile estendere l'analisi a modelli di maggiori dimensioni, che potrebbero beneficiare maggiormente di un processo di questo tipo. Sarebbe inoltre auspicabile estendere l'analisi a modelli di maggiori dimensioni al fine di verificare la presenza degli stessi pattern strutturali osservati nel test condotti in questo lavoro, in maniera tale da confermare ulteriormente le considerazioni formulate attraverso l'analisi dei risultati.
- **Estensione a compiti di visione complessi:** Oltre a testare il framework su modelli di maggiori dimensioni, sarebbe utile eseguire dei test al di fuori del *task* di classificazione. Questo consentirebbe di verificare la bontà del processo su modelli utilizzati come *backbone* per compiti di *object detection*, *semantic segmentation* o modelli multimodali.
- **Confronto con metodo Greedy:** Dato l'impiego di un algoritmo di ricerca avanzato per l'individuazione delle migliori azioni di *pruning* da attuare, sarebbe necessario un confronto con un implementazione di tipo *greedy* del metodo. Tale comparazione permetterebbe di verificare una eventuale superiorità del processo di ricerca con algoritmo *Branch and Bound* rispetto a quanto proposto in questo lavoro, giustificando così l'incremento del costo computazionale.

- **Integrazione con tecniche di quantizzazione:** Il beneficio computazionale potrebbe essere ulteriormente amplificato combinando il *pruning* strutturato con tecniche di quantizzazione post-addestramento (PTQ) o quantizzazione di tipo *training-aware*(QAT). L'integrazione di queste tecniche, oggi molto diffuse nell'ambito dei *Large Language Models* (LLM) e pienamente supportate dall'hardware, permetterebbero di ottenere modelli ancora più performanti, al costo di un leggero decadimento delle prestazioni predittive.
- **Applicazione a modelli di tipo *Large Language Models* (LLM):** Un interessante sviluppo futuro potrebbe essere rappresentato dall'estensione del framework a modelli di tipo LLM, che presentano un numero di parametri estremamente elevato, e che potrebbero beneficiare in maniera significativa da un processo di compressione. L'applicazione del framework a questo tipo di modelli potrebbe richiedere alcune modifiche alla strategia di compressione, ma rappresenterebbe un importante passo avanti. La più grande limitazione in questo caso sarebbe costituita dal costo computazionale, che potrebbe essere proibitivo, tuttavia si potrebbe mitigarlo attraverso l'impiego di tecniche di *parameter efficient fine tuning* (PEFT), che permetterebbero di ridurre significativamente il numero di parametri da aggiornare durante la fase di *fine tuning*.
- **Utilizzo di tecniche di Token Pruning:** Un ulteriore direzione di ricerca potrebbe essere rappresentato dall'integrazione di tecniche di *token pruning*, che permetterebbero di ridurre in maniera significativa il costo computazionale durante la fase di inferenza, eliminando i token meno rilevanti per la predizione. Questo è facilmente visibile nei risultati ottenuti in [70], nel quale è stato adottato sia un processo di *pruning* strutturato, sia una tecnica di *token merging*, ottenendo un incremento del 90% di *throughput*.

Ringraziamenti

Desidero esprimere la mia più profonda gratitudine, in primo luogo, alla mia Relatrice, la Prof.ssa Liliana Lo Presti, per avermi guidato con costante disponibilità, pazienza e rigore scientifico lungo tutto lo sviluppo di questo lavoro di ricerca. I suoi preziosi consigli e la fiducia che ha riposto nelle mie intuizioni sono stati fondamentali per il superamento delle sfide incontrate lungo il percorso.

Successivamente desidero ringraziare tutti i miei colleghi di laboratorio, in particolare i due Gabriele, che mi hanno fornito tante opportunità di confronto e crescita, e mi hanno tirato su nei momenti in cui il morale poteva vacillare.

Un ringraziamento speciale va poi a Gianvito, con il quale ho condiviso questo percorso di studi, una casa e soprattutto un'amicizia sincera e fraterna.

Un grazie di cuore va alla mia famiglia e ai miei amici, che mi hanno sempre sostenuto, incoraggiato ma soprattutto sopportato, in questi anni di stress e sacrifici. Senza di loro, questo traguardo non sarebbe stato possibile.

Infine, grazie a Paola per essere stata al mio fianco, per tutto il suo amore, la sua pazienza e il suo supporto incondizionato. Senza di lei, questo percorso sarebbe stato molto più difficile e noioso. La sua presenza e il suo affetto sono stati una fonte inesauribile di forza e motivazione.

A tutti voi, il mio più sincero grazie.

Bibliografia

- [1] A. F. Agarap. *Deep Learning using Rectified Linear Units (ReLU)*. 2019. arXiv: 1803.08375 [cs.NE]. URL: <https://arxiv.org/abs/1803.08375>.
- [2] J. L. Ba, J. R. Kiros e G. E. Hinton. *Layer Normalization*. 2016. arXiv: 1607.06450 [stat.ML]. URL: <https://arxiv.org/abs/1607.06450>.
- [3] D. Bahdanau, K. Cho e Y. Bengio. *Neural Machine Translation by Jointly Learning to Align and Translate*. 2016. arXiv: 1409.0473 [cs.CL]. URL: <https://arxiv.org/abs/1409.0473>.
- [4] Y. Bengio, P. Simard e P. Frasconi. «Learning long-term dependencies with gradient descent is difficult». In: *IEEE Transactions on Neural Networks* 5.2 (1994), pp. 157–166.
- [5] E. Boros e P. L. Hammer. «Pseudo-Boolean optimization». In: *Discrete Applied Mathematics* 123.1 (2002), pp. 155–225. URL: <https://www.sciencedirect.com/science/article/pii/S0166218X01003419>.
- [6] C. Buciluă, R. Caruana e A. Niculescu-Mizil. *Model compression*. 2006.
- [7] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov e S. Zagoruyko. *End-to-End Object Detection with Transformers*. 2020. arXiv: 2005.12872 [cs.CV]. URL: <https://arxiv.org/abs/2005.12872>.
- [8] M. Chen, H. Peng, J. Fu e H. Ling. *AutoFormer: Searching Transformers for Visual Recognition*. 2021. arXiv: 2107.00651 [cs.CV]. URL: <https://arxiv.org/abs/2107.00651>.
- [9] T. Chen, Y. Cheng, Z. Gan, L. Yuan, L. Zhang e Z. Wang. *Chasing Sparsity in Vision Transformers: An End-to-End Exploration*. 2021. arXiv: 2106.04533 [cs.CV]. URL: <https://arxiv.org/abs/2106.04533>.

- [10] X. Chen, Q. Cao, Y. Zhong, J. Zhang, S. Gao e D. Tao. *DearKD: Data-Efficient Early Knowledge Distillation for Vision Transformers*. 2022. arXiv: 2204.12997 [cs.CV]. URL: <https://arxiv.org/abs/2204.12997>.
- [11] Y. Chen, X. Dai, D. Chen, M. Liu, X. Dong, L. Yuan e Z. Liu. *MobileFormer: Bridging MobileNet and Transformer*. 2022. arXiv: 2108.05895 [cs.CV]. URL: <https://arxiv.org/abs/2108.05895>.
- [12] K. Cho, B. van Merriënboer, Ç. Gülçehre, F. Bougares, H. Schwenk e Y. Bengio. «Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation». In: *CoRR* abs/1406.1078 (2014). arXiv: 1406.1078. URL: <http://arxiv.org/abs/1406.1078>.
- [13] J.-B. Cordonnier, A. Loukas e M. Jaggi. *On the Relationship between Self-Attention and Convolutional Layers*. 2020. arXiv: 1911.03584 [cs.LG]. URL: <https://arxiv.org/abs/1911.03584>.
- [14] Z. Dai, H. Liu, Q. V. Le e M. Tan. *CoAtNet: Marrying Convolution and Attention for All Data Sizes*. 2021. arXiv: 2106.04803 [cs.CV]. URL: <https://arxiv.org/abs/2106.04803>.
- [15] K. Deb, A. Pratap, S. Agarwal e T. Meyarivan. «A fast and elitist multiobjective genetic algorithm: NSGA-II». In: *IEEE Transactions on Evolutionary Computation* 6 (2002), pp. 182–197.
- [16] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li e L. Fei-Fei. «ImageNet: A large-scale hierarchical image database». In: *2009 IEEE Conference on Computer Vision and Pattern Recognition*. 2009, pp. 248–255.
- [17] J. Devlin, M.-W. Chang, K. Lee e K. Toutanova. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2019. arXiv: 1810.04805 [cs.CL]. URL: <https://arxiv.org/abs/1810.04805>.
- [18] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit e N. Houlsby. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. 2021. arXiv: 2010.11929 [cs.CV]. URL: <https://arxiv.org/abs/2010.11929>.
- [19] D. E. W. E. L. Lawler. *Branch-and-Bound Methods: A Survey*. 1966. eprint: <https://doi.org/10.1287/opre.14.4.699>.

- [20] J. Eckstein e D. P. Bertsekas. «On the Douglas—Rachford splitting method and the proximal point algorithm for maximal monotone operators». In: *Mathematical programming* 55.1 (1992), pp. 293–318.
- [21] M. Geva, R. Schuster, J. Berant e O. Levy. «Transformer feed-forward layers are key-value memories». In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 2021, pp. 5484–5495.
- [22] C. Gong, D. Wang, M. Li, X. Chen, Z. Yan, Y. Tian, V. Chandra et al. *Nasvit: Neural architecture search for efficient vision transformers with gradient conflict aware supernet training*. 2021.
- [23] Y. Gong, L. Liu, M. Yang e L. Bourdev. *Compressing Deep Convolutional Networks using Vector Quantization*. 2014. arXiv: 1412.6115 [cs.CV]. URL: <https://arxiv.org/abs/1412.6115>.
- [24] A. Gu e T. Dao. *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. 2024. arXiv: 2312.00752 [cs.LG]. URL: <https://arxiv.org/abs/2312.00752>.
- [25] S. Han, H. Mao e W. J. Dally. *Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding*. 2016. arXiv: 1510.00149 [cs.CV]. URL: <https://arxiv.org/abs/1510.00149>.
- [26] A. Hatamizadeh, G. Heinrich, H. Yin, A. Tao, J. M. Alvarez, J. Kautz e P. Molchanov. *FasterViT: Fast Vision Transformers with Hierarchical Attention*. 2024. arXiv: 2306.06189 [cs.CV]. URL: <https://arxiv.org/abs/2306.06189>.
- [27] K. He, X. Zhang, S. Ren e J. Sun. *Deep Residual Learning for Image Recognition*. 2015. arXiv: 1512.03385 [cs.CV]. URL: <https://arxiv.org/abs/1512.03385>.
- [28] D. Hendrycks e K. Gimpel. *Gaussian Error Linear Units (GELUs)*. 2023. arXiv: 1606.08415 [cs.LG]. URL: <https://arxiv.org/abs/1606.08415>.
- [29] G. Hinton, O. Vinyals e J. Dean. *Distilling the Knowledge in a Neural Network*. 2015. arXiv: 1503.02531 [stat.ML]. URL: <https://arxiv.org/abs/1503.02531>.
- [30] S. Hochreiter e J. Schmidhuber. «Long Short-Term Memory». In: *Neural Computation* 9.8 (1997), pp. 1735–1780.

- [31] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam e D. Kalenichenko. *Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference*. 2017. arXiv: 1712.05877 [cs.LG]. URL: <https://arxiv.org/abs/1712.05877>.
- [32] E. Jang, S. Gu e B. Poole. *Categorical Reparameterization with Gumbel-Softmax*. 2017. arXiv: 1611.01144 [stat.ML]. URL: <https://arxiv.org/abs/1611.01144>.
- [33] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. d. I. Casas, E. B. Hanna, F. Bressand et al. «Mixtral of experts». In: *arXiv preprint arXiv:2401.04088* (2024).
- [34] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollár e R. Girshick. *Segment Anything*. 2023. arXiv: 2304.02643 [cs.CV]. URL: <https://arxiv.org/abs/2304.02643>.
- [35] F. Kunstner, L. Balles e P. Hennig. *Limitations of the Empirical Fisher Approximation for Natural Gradient Descent*. 2020. arXiv: 1905.12558 [cs.LG]. URL: <https://arxiv.org/abs/1905.12558>.
- [36] e. G. H. LeCun Y. Bengio. *Deep Learning*. 2015. URL: <https://doi.org/10.1038/nature14539>.
- [37] Y. Li, H. Mao, R. Girshick e K. He. *Exploring Plain Vision Transformer Backbones for Object Detection*. 2022. arXiv: 2203.16527 [cs.CV]. URL: <https://arxiv.org/abs/2203.16527>.
- [38] Y. Li, S. Xu, B. Zhang, X. Cao, P. Gao e G. Guo. *Q-ViT: Accurate and Fully Quantized Low-bit Vision Transformer*. 2022. arXiv: 2210.06707 [cs.CV]. URL: <https://arxiv.org/abs/2210.06707>.
- [39] Y. Lin, T. Zhang, P. Sun, Z. Li e S. Zhou. *FQ-ViT: Post-Training Quantization for Fully Quantized Vision Transformer*. 2023. arXiv: 2111.13824 [cs.CV]. URL: <https://arxiv.org/abs/2111.13824>.
- [40] J. D. C. Little, K. G. Murty, D. W. Sweeney e C. Karel. «An Algorithm for the Traveling Salesman Problem». In: *Operations Research* 11.6 (1963), pp. 972–989. URL: <http://www.jstor.org/stable/167836> (visitato il giorno 29/03/2026).

- [41] H. Liu, K. Simonyan e Y. Yang. *DARTS: Differentiable Architecture Search*. 2019. arXiv: 1806.09055 [cs.LG]. URL: <https://arxiv.org/abs/1806.09055>.
- [42] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin e B. Guo. *Swin Transformer: Hierarchical Vision Transformer using Shifted Windows*. 2021. arXiv: 2103.14030 [cs.CV]. URL: <https://arxiv.org/abs/2103.14030>.
- [43] Z. Liu, Y. Wang, K. Han, S. Ma e W. Gao. *Post-Training Quantization for Vision Transformer*. 2021. arXiv: 2106.14156 [cs.CV]. URL: <https://arxiv.org/abs/2106.14156>.
- [44] W. Luo, Y. Li, R. Urtasun e R. Zemel. *Understanding the Effective Receptive Field in Deep Convolutional Neural Networks*. 2017. arXiv: 1701.04128 [cs.CV]. URL: <https://arxiv.org/abs/1701.04128>.
- [45] C. J. Maddison, A. Mnih e Y. W. Teh. *The Concrete Distribution: A Continuous Relaxation of Discrete Random Variables*. 2017. arXiv: 1611.00712 [cs.LG]. URL: <https://arxiv.org/abs/1611.00712>.
- [46] S. Markidis, S. W. D. Chien, E. Laure, I. B. Peng e J. S. Vetter. *NVIDIA Tensor Core Programmability, Performance & Precision*. 2018.
- [47] S. Mehta e M. Rastegari. *MobileViT: Light-weight, General-purpose, and Mobile-friendly Vision Transformer*. 2022. arXiv: 2110.02178 [cs.CV]. URL: <https://arxiv.org/abs/2110.02178>.
- [48] P. Michel, O. Levy e G. Neubig. «Are sixteen heads really better than one?» In: *Advances in neural information processing systems* 32 (2019).
- [49] P. Molchanov, A. Mallya, S. Tyree, I. Frosio e J. Kautz. *Importance Estimation for Neural Network Pruning*. 2019. arXiv: 1906.10771 [cs.LG]. URL: <https://arxiv.org/abs/1906.10771>.
- [50] W. Murray e K.-M. Ng. *An algorithm for nonlinear optimization problems with binary variables*. 2010. URL: <https://link.springer.com/article/10.1007/s10589-008-9218-1>.
- [51] J. Pan, A. Bulat, F. Tan, X. Zhu, L. Dudziak, H. Li, G. Tzimiropoulos e B. Martinez. *EdgeViTs: Competing Light-weight CNNs on Mobile Devices with Vision Transformers*. 2022. arXiv: 2205.03436 [cs.CV]. URL: <https://arxiv.org/abs/2205.03436>.

- [52] K. Persand, A. Anderson e D. Gregg. «Taxonomy of Saliency Metrics for Channel Pruning». In: *IEEE Access* 9 (2021), pp. 120110–120126. URL: <http://dx.doi.org/10.1109/ACCESS.2021.3108545>.
- [53] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger e I. Sutskever. *Learning Transferable Visual Models From Natural Language Supervision*. 2021. arXiv: 2103.00020 [cs.CV]. URL: <https://arxiv.org/abs/2103.00020>.
- [54] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey e I. Sutskever. *Robust Speech Recognition via Large-Scale Weak Supervision*. 2022. arXiv: 2212.04356 [eess.AS]. URL: <https://arxiv.org/abs/2212.04356>.
- [55] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever et al. «Language models are unsupervised multitask learners». In: *OpenAI blog* 1.8 (2019), p. 9.
- [56] M. Rastegari, V. Ordonez, J. Redmon e A. Farhadi. *XNOR-Net: ImageNet Classification Using Binary Convolutional Neural Networks*. 2016. arXiv: 1603.05279 [cs.CV]. URL: <https://arxiv.org/abs/1603.05279>.
- [57] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov e L.-C. Chen. *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. 2019. arXiv: 1801.04381 [cs.CV]. URL: <https://arxiv.org/abs/1801.04381>.
- [58] P. Shaw, J. Uszkoreit e A. Vaswani. *Self-Attention with Relative Position Representations*. 2018. arXiv: 1803.02155 [cs.CL]. URL: <https://arxiv.org/abs/1803.02155>.
- [59] A. Srinivas, T.-Y. Lin, N. Parmar, J. Shlens, P. Abbeel e A. Vaswani. *Bottleneck Transformers for Visual Recognition*. 2021. arXiv: 2101.11605 [cs.CV]. URL: <https://arxiv.org/abs/2101.11605>.
- [60] A. Steiner, A. Kolesnikov, X. Zhai, R. Wightman, J. Uszkoreit e L. Beyer. *How to train your ViT? Data, Augmentation, and Regularization in Vision Transformers*. 2022. arXiv: 2106.10270 [cs.CV]. URL: <https://arxiv.org/abs/2106.10270>.
- [61] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen e Y. Liu. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. 2023. arXiv: 2104.09864 [cs.CL]. URL: <https://arxiv.org/abs/2104.09864>.

- [62] X. Su, S. You, J. Xie, M. Zheng, F. Wang, C. Qian, C. Zhang, X. Wang e C. Xu. *ViTAS: Vision Transformer Architecture Search*. 2021. arXiv: 2106.13700 [cs.CV]. URL: <https://arxiv.org/abs/2106.13700>.
- [63] I. Sutskever, O. Vinyals e Q. V. Le. «Sequence to Sequence Learning with Neural Networks». In: *CoRR* abs/1409.3215 (2014). arXiv: 1409.3215. URL: <http://arxiv.org/abs/1409.3215>.
- [64] L. Theis, I. Korshunova, A. Tejani e F. Huszár. *Faster gaze prediction with dense networks and Fisher pruning*. 2018. arXiv: 1801.05787 [cs.CV]. URL: <https://arxiv.org/abs/1801.05787>.
- [65] H. Tian, B. Xu e S. Li. *Distillation Dynamics: Towards Understanding Feature-Based Distillation in Vision Transformers*. 2025. arXiv: 2511.06848 [cs.CV]. URL: <https://arxiv.org/abs/2511.06848>.
- [66] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles e H. Jégou. *Training data-efficient image transformers and distillation through attention*. 2021. arXiv: 2012.12877 [cs.CV]. URL: <https://arxiv.org/abs/2012.12877>.
- [67] P. K. A. Vasu, J. Gabriel, J. Zhu, O. Tuzel e A. Ranjan. *FastViT: A Fast Hybrid Vision Transformer using Structural Reparameterization*. 2023. arXiv: 2303.14189 [cs.CV]. URL: <https://arxiv.org/abs/2303.14189>.
- [68] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser e I. Polosukhin. «Attention Is All You Need». In: *CoRR* abs/1706.03762 (2017). arXiv: 1706.03762. URL: <http://arxiv.org/abs/1706.03762>.
- [69] A. Wang, H. Chen, Z. Lin, J. Han e G. Ding. *RepViT: Revisiting Mobile CNN From ViT Perspective*. 2024. arXiv: 2307.09283 [cs.CV]. URL: <https://arxiv.org/abs/2307.09283>.
- [70] A. Wang, H. Chen, Z. Lin, S. Zhao, J. Han e G. Ding. *CAIT: Triple-Win Compression towards High Accuracy, Fast Inference, and Favorable Transferability For ViTs*. 2025. arXiv: 2309.15755 [cs.CV]. URL: <https://arxiv.org/abs/2309.15755>.
- [71] S. Wang, T. Xie, J. Cheng, X. Zhang e H. Liu. *MDL-NAS: A Joint Multi-domain Learning Framework for Vision Transformer*. 2023.

- [72] H. Wu, B. Xiao, N. Codella, M. Liu, X. Dai, L. Yuan e L. Zhang. *CvT: Introducing Convolutions to Vision Transformers*. 2021. arXiv: 2103.15808 [cs.CV]. URL: <https://arxiv.org/abs/2103.15808>.
- [73] K. Wu, J. Zhang, H. Peng, M. Liu, B. Xiao, J. Fu e L. Yuan. *TinyViT: Fast Pretraining Distillation for Small Vision Transformers*. 2022. arXiv: 2207.10666 [cs.CV]. URL: <https://arxiv.org/abs/2207.10666>.
- [74] P. Wu, F. Neri e Z. Feng. *DASViT: Differentiable Architecture Search for Vision Transformer*. 2025. arXiv: 2507.13079 [cs.LG]. URL: <https://arxiv.org/abs/2507.13079>.
- [75] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang e T.-Y. Liu. *On Layer Normalization in the Transformer Architecture*. 2020. arXiv: 2002.04745 [cs.LG]. URL: <https://arxiv.org/abs/2002.04745>.
- [76] M. Yin, B. Uzkent, Y. Shen, H. Jin e B. Yuan. *GOHSP: A Unified Framework of Graph and Optimization-based Heterogeneous Structured Pruning for Vision Transformer*. 2023. arXiv: 2301.05345 [cs.AI]. URL: <https://arxiv.org/abs/2301.05345>.
- [77] F. Yu, K. Huang, M. Wang, Y. Cheng, W. Chu e L. Cui. *Width & depth pruning for vision transformers*. 2022.
- [78] L. Yu e W. Xiang. *X-Pruner: eXplainable Pruning for Vision Transformers*. 2023. arXiv: 2303.04935 [cs.CV]. URL: <https://arxiv.org/abs/2303.04935>.
- [79] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe e Y. Yoo. *CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features*. 2019. arXiv: 1905.04899 [cs.CV]. URL: <https://arxiv.org/abs/1905.04899>.
- [80] X. Zhai, B. Mustafa, A. Kolesnikov e L. Beyer. *Sigmoid Loss for Language Image Pre-Training*. 2023. arXiv: 2303.15343 [cs.CV]. URL: <https://arxiv.org/abs/2303.15343>.
- [81] H. Zhang, M. Cisse, Y. N. Dauphin e D. Lopez-Paz. *mixup: Beyond Empirical Risk Minimization*. 2018. arXiv: 1710.09412 [cs.LG]. URL: <https://arxiv.org/abs/1710.09412>.

- [82] L. Zhu, B. Liao, Q. Zhang, X. Wang, W. Liu e X. Wang. *Vision Mamba: Efficient Visual Representation Learning with Bidirectional State Space Model*. 2024. arXiv: 2401.09417 [cs.CV]. URL: <https://arxiv.org/abs/2401.09417>.